

GRAND CHALLENGE MONOGRAPH COLLECTION

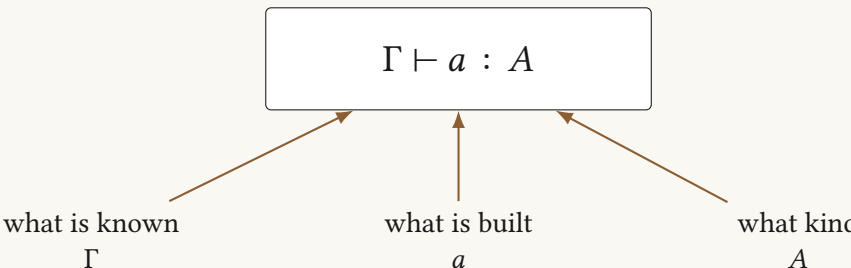
TYPE THEORY

The Grand Unified Theory of Computation

JUDGMENT

The Grammar of Computation

Volume I



Grand Challenge Labs

First Edition - Release Candidate 1 - 2026

Grand Challenge Monograph Collection

Type Theory - The Grand Unified Theory of Computation, Volume I

Judgment: The Grammar of Computation

First Edition, Release Candidate 1, 2026.

Prepared by Grand Challenge Labs.

This release candidate is intended for mathematical, pedagogical, bibliographic, and typographic review. Historical attributions are documented in the bibliography. The phrase “grand unified theory” names the series’ research thesis; it is not asserted as an established theorem.

The executable TinyTT companion is a pedagogical reference implementation, not a production proof kernel. Formal metatheoretic claims in the text state their scope explicitly.

Preface: Why Begin Before Programs?

Most introductions to computation begin with machines, algorithms, or programs. This book begins earlier. It asks what must already be in place before an expression can count as a program, before an operation can count as an application, and before a sequence of symbols can count as saying anything at all.

The answer developed here is not “types prevent errors.” That is true but shallow. The deeper claim is that computation depends on a discipline of meaningful distinctions. A system must distinguish an expression from noise, a number from a function, an assumption from a conclusion, a temporary name from a permanent definition, and a valid transformation from an arbitrary rewrite. Type theory gives us a language for making those distinctions explicit.

The central sentence of Volume I is

$$\Gamma \vdash a : A.$$

It says, roughly: under the information recorded in context Γ , the expression a is a meaningful construction of kind A . Every symbol in that sentence will be earned rather than imposed. By the end of the book, the reader should see contexts as informational worlds, terms as constructions, types as structured spaces of admissible constructions, substitution as the motion of information through dependency, and evaluation as the exposure of canonical computational content.

This is a first volume, not the whole theory. We deliberately postpone full dependent sums and products, universes, identity types, effects, linearity, session types, and higher structure. But we construct the conceptual machinery that makes those subjects feel inevitable.

Governing Principle

Formal notation should arrive after the problem it solves is visible. A rule is memorable when the reader knows what would go wrong without it.

How to use this book

Each chapter has four layers. The main text builds intuition and formal structure together. *Formal Interludes* isolate precise rules and metatheoretic claims. *Computational Laboratories* turn the mathematics into small executable systems. Exercises are divided into Checkpoint, Core, Synthesis, Proof Workshop, Design Clinic, and Challenge modes. The publication calibration is: Checkpoint = one-star recall/translation; Core = two-star construction; Synthesis = three-star explanation across viewpoints; Proof Workshop = three-star formal argument; Design Clinic = four-star language engineering; Challenge = four-to-five-star extension or research reading. A complete instructor/self-study solution companion accompanies this release candidate. The illustration plates are not decoration: each has a stated pedagogical burden, and the complete production specification appears in Appendix D.

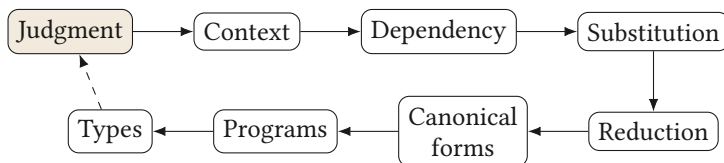
No proof assistant is required. The laboratories use a tiny Python implementation included with the manuscript. The goal is not to teach Python. The goal is to make every semantic claim operational enough that the reader can test it.

Metatheoretic convention. Named theorems in this volume refer, unless stated otherwise, to the small typed core implemented by `TinyTT`: booleans, naturals, unit, empty, functions, products, sums, explicit annotations, and a nondependent natural-number recursor, with capture-avoiding substitution and the call-by-value reduction relation used in the laboratory companion. The declarative metatheory uses the standard fresh-binder convention: bound variables may be alpha-renamed so that formal contexts contain distinct names. `TinyTT` permits lexical shadowing in source terms; this is an implementation convenience, not an additional declarative typing principle. Proofs are complete when they are short enough to illuminate the mechanism; longer standard inductions are given as structured proof sketches whose induction measure and critical cases are explicit. Later volumes will strengthen these results for dependent calculi.

Series Orientation

The larger collection advances a hypothesis: type theory is a candidate grammar of computational possibility. Volume I studies the grammar itself. Later volumes ask what happens when propositions become executable, when interaction replaces closed evaluation, when effects enter, when types become data, and when equality acquires higher structure.

The phrase “grand unified theory” is therefore a research thesis, not a ceremonial title. The formal lineage of this volume runs through typed lambda calculus, natural deduction, proof normalization, and constructive type theory; representative primary landmarks include Church’s simple type theory, Gentzen’s natural deduction, Curry’s functionality discipline, Howard’s formulae-as-types notes, and Martin-Löf’s intuitionistic type theory [2, 5, 4, 8, 9]. Untyped computation, general recursion, concurrency, stochasticity, physical interaction, and machine learning all place pressure on it. A serious unification must explain those cases or mark exactly where its reach ends.



Contents

Preface: Why Begin Before Programs?	iii
Series Orientation	v
1 Before Programs: What Does It Mean for an Expression to Make Sense?	1
1.1 The mistake of beginning with execution	1
1.2 Three layers: marks, expressions, judgments	1
1.3 Meaning as licensed use	2
1.4 The traffic-light analogy, and where it fails	2
1.5 A first reading of the central judgment	4
1.6 Why types are not sets, yet	4
1.7 The first design question	4
Exercises	6
2 Judgments: The Atomic Form of Formal Meaning	9
2.1 Statements about expressions, not expressions themselves	9
2.2 Rules as licenses to assert judgments	9
2.3 Derivation trees	10
2.4 Canonical and noncanonical forms	10
2.5 Introduction, elimination, computation	10
2.6 Why elimination rules matter	12
2.7 Derivability versus truth	13
3 Contexts: Worlds of Assumption	15
3.1 Closed terms are the exception	15

3.2	Context formation	15
3.3	The variable rule	17
3.4	Contexts are ordered when dependency matters	17
3.5	Weakening: unused information may be carried	17
3.6	Exchange and contraction as computational policies	18
3.7	Context as interface state	19
4	Variables and Dependency: Naming What the Future May Supply	22
4.1	A variable is a promise, not a mystery	22
4.2	Free and bound occurrences	22
4.3	Alpha-equivalence: names are not identities	23
4.4	Dependency before dependent types	23
4.5	Families as maps from information to possibility	25
4.6	Scope is a computational boundary	25
5	Substitution: Information in Motion	29
5.1	The central dynamic	29
5.2	A simple example	29
5.3	Substitution is not textual search-and-replace	30
5.4	Substitution as graph rewriting	30
5.5	Beta reduction is substitution exposed	32
5.6	The substitution lemma	32
5.7	Why the lemma matters for execution	33
5.8	Substitution beyond syntax	34
6	Rules: How a Language Grows	37
6.1	A language is a constitution	37
6.2	The harmony principle	37
6.3	Local soundness	39
6.4	Local completeness	39
6.5	Rules generate computation	39
6.6	An example: the unit type	40
6.7	An example: the empty type	40

6.8	Rule inversion as reasoning	40
7	Functions: Packaging Hypothetical Computation	43
7.1	A function begins as a hypothetical	43
7.2	The function type as a capability	43
7.3	Beta: opening a package	44
7.4	Eta: functions are determined by application behavior	44
7.5	Higher-order computation	44
7.6	Currying reveals dependency structure	45
7.7	Functions and implication	45
7.8	Functions are not all of computation	48
8	Products and Sums: The Algebra of Information	50
8.1	Two ways to combine possibilities	50
8.2	Products: information in parallel	50
8.3	Sums: information with a tag	51
8.4	Case analysis is exhaustive by type	51
8.5	Products and sums as logical connectives	51
8.6	Data types as sums of products	53
8.7	The distributive law becomes a program	53
8.8	Products and sums as communication shapes	55
9	Natural Numbers and Induction: Computation from Generation	57
9.1	A type can be specified by how its inhabitants are generated	57
9.2	Recursion as the computational eliminator	57
9.3	Addition emerges from the eliminator	58
9.4	Induction is dependent elimination	58
9.5	Why structural recursion tends to terminate	58
9.6	Canonical forms and canonicity	60
9.7	Induction as a comprehension principle	61
10	Equality and Conversion: When Are Two Programs the Same?	64
10.1	One question, several equalities	64

10.2	Syntactic identity is too strict	64
10.3	Definitional equality	65
10.4	Conversion	65
10.5	Normalization as an equality procedure	65
10.6	Confluence	66
10.7	Extensional equality is stronger	66
10.8	Equality is a budget	66
11	Normalization: Computation as the Exposure of Canonical Form	72
11.1	What does evaluation reveal?	72
11.2	Values, normal forms, canonical forms	72
11.3	Evaluation strategy	73
11.4	Strong normalization	73
11.5	Termination is an expressive constraint	74
11.6	Progress	74
11.7	Preservation	76
11.8	Normalization and proof	77
11.9	Normalization is not the whole of computation	78
12	Proof and Program: The Curry-Howard Lens	81
12.1	A correspondence that changes both sides	81
12.2	Conjunction as product	81
12.3	Disjunction as sum	83
12.4	Falsehood as empty type	83
12.5	Universal and existential quantification foreshadow dependency	83
12.6	Proof relevance	84
12.7	Programs from proofs, proofs from programs	84
12.8	Inhabitation as proof search	84
12.9	Where Curry-Howard is not enough	86
12.10	The first unification	88
13	Type Checking: Turning Judgments into an Algorithm	90
13.1	From declarative rules to a machine procedure	90

13.2	Inference and checking	90
13.3	A tiny synthesis algorithm	91
13.4	Error messages as failed derivations	91
13.5	Elaboration	93
13.6	The trusted computing base	93
13.7	Decidability	94
13.8	A minimal proof assistant architecture	95
14	Judgment as Communication: From Expressions to Protocol States	98
14.1	The turnstile as a communication boundary	98
14.2	A function call is a tiny protocol	98
14.3	Contexts as protocol states	99
14.4	APIs as partial type theories	99
14.5	The limits of the closed-term worldview	99
14.6	Language is not merely representation	101
14.7	Collected comprehension principles	101
14.8	What Volume I has established	102
14.9	The next necessity: types that depend on information	103
14.10	The thesis after one volume	104
A	A Compact Formal Reference	109
A.1	Notation guide	109
A.2	Types and terms	110
A.3	Contexts	110
A.4	Selected typing rules	110
A.5	Core reductions	111
A.6	Metatheoretic dependency graph	112
B	Computational Laboratory Guide	113
B.1	Purpose	113
B.2	Recommended sequence	113
B.3	Laboratory rubric	113
B.4	Experiments worth keeping	114

C	Selected Exercise Notes and Solutions	115
C.1	Function inhabitants	115
C.2	Commuting products and sums	115
C.3	Why an arbitrary atomic A has no closed inhabitant	116
C.4	The capture example	116
C.5	Weakening: the application case	116
C.6	Substitution: the binder case	116
C.7	Progress for applications	117
C.8	Bidirectional soundness: application	117
D	Illustration Register and Production Status	118
D.1	Visual doctrine	118
D.2	Figure-production constraints	123
E	Glossary of Core Terms	124
F	Further Reading: A Guided Path	126
	Bibliography	127
G	Instructor and Self-Study Roadmap	130
G.1	Fourteen-week course	130
G.2	Mastery criteria	130
G.3	Capstone project	131
	Closing Note: The Small Sentence	132

Before Programs: What Does It Mean for an Expression to Make Sense?

1.1 The mistake of beginning with execution

Imagine that a machine receives the string

`blue + 7.`

Should it execute? The question is premature. Before asking what the expression evaluates to, we must ask whether the expression belongs to a language in which the operation `+` may be applied to its two arguments. A processor can manipulate any bit pattern; computation, in the richer sense relevant to programming and reasoning, requires a distinction between *well-formed use* and arbitrary symbol motion.

This is the first reversal in the book. Meaning does not arrive after execution as a story we tell about machine behavior. Some account of admissible expression must precede any claim that a machine behavior implements a program.

A calculator illustrates the point. The button sequence `2 + 3 =` is admitted by its interface and yields a number. The sequence `+ = sqrt +` may be physically pressable, yet it is not thereby a mathematical expression. The device embodies a grammar of acceptable states and transitions. Programming languages make such grammars explicit.

Intuition

A type system is often described as a fence that keeps bad programs out. A better first picture is a grammar that makes certain computational sentences possible to say at all.

1.2 Three layers: marks, expressions, judgments

We will distinguish three levels that are commonly collapsed.

At the first level are *marks*: characters, tokens, syntax trees, bytes. They are physical or representational objects. At the second level are *expressions*: marks interpreted according to a grammar. At the third level are *judgments*: claims that some expression has a particular status.

For example, the string $3 + 4$ may parse as an expression. The claim

$$\vdash 3 + 4 : \mathbb{N}$$

says more: the expression is a construction of natural-number kind. A parser can establish the first fact. A type checker is concerned with the second.

This distinction matters because grammatical shape alone is weak. Both $3 + 4$ and `true + 4` may have the same parse tree. The second can be syntactically legal and still fail to have the required meaning.

1.3 Meaning as licensed use

The word *meaning* is dangerous because it invites philosophical inflation. We use it operationally. To say that an expression is meaningful in a formal system is to say that the rules of the system license certain uses of it.

If $n : \mathbb{N}$, we may ask for its successor. If $f : A \rightarrow B$ and $a : A$, we may apply f to a . If $p : A \times B$, we may project its first component. These permissions are not comments attached to terms. They determine what further constructions are available.

Meaning is therefore revealed by *consequences of use*. A type is valuable because it tells us what operations are legitimate and what guarantees those operations preserve.

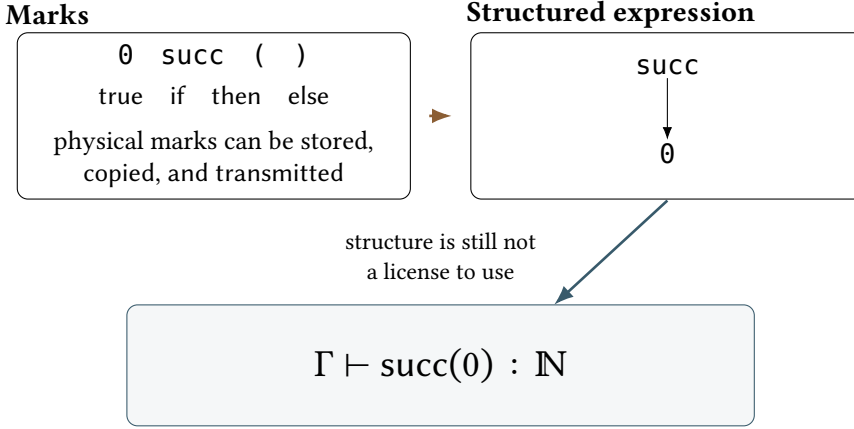
1.4 The traffic-light analogy, and where it fails

A traffic light classifies situations into states such as red, amber, and green, and each state licenses different actions. Types behave similarly: they classify terms and constrain what can happen next.

But types are richer than labels. A function type $A \rightarrow B$ does not merely attach the label “function.” It states a behavioral contract: supply an acceptable A and the construction must produce a B . A product type says what observations can be made. A dependent type will later say that the very set of legal future operations can depend on information obtained now.

The analogy is useful only up to the point where the classification becomes generative. Types do not merely sort existing objects. Type formers create new spaces of admissible computation.

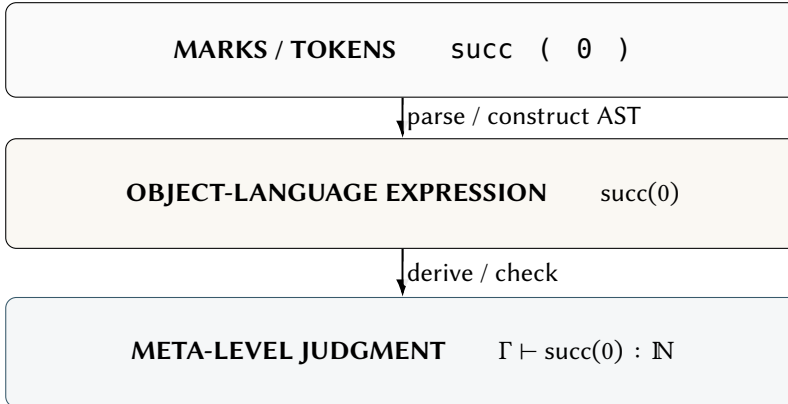
Before execution



The added layer: a formal system licenses a claim about the expression.
 Parsing organizes marks; typing situates the expression in a discipline of admissible use.

Figure 1.1: **Plate 1. Before execution.** Symbol manipulation, parsing, and typing are distinct layers. The diagram does not claim that every meaningful computation must be typed; it isolates the additional structure a typed formal system contributes.

Marks, expressions, judgments



Each downward move adds commitments.
 A term is not itself a claim that the term is well formed, typable, equal, or true.

Figure 1.2: **Plate 3. Three levels that must not be collapsed.** The stack is conceptual, not an implementation mandate: a compiler may interleave parsing and checking, but the distinctions between concrete marks, object-language syntax, and meta-level assertions remain useful.

1.5 A first reading of the central judgment

We now display the central form without yet formalizing it:

$$\Gamma \vdash a : A.$$

Read it as:

Given the information recorded in Γ , a is a legitimate construction of kind A .

The left side records what is available. The right side records what has been constructed and how it may be used. The turnstile separates assumptions from the claim they license.

The empty context is often omitted. Thus

$$\vdash 2 : \mathbb{N}$$

says that 2 needs no local assumptions to count as a natural-number construction.

1.6 Why types are not sets, yet

A familiar mathematical reading treats a type as a set and a term as an element. This can be a useful model, but we will not make it our definition. If we prematurely identify types with sets, we smuggle in an entire foundation and obscure the operational questions that type theory is designed to answer.

For now, a type is specified by rules: how we know it is a type, how its canonical inhabitants are introduced, how inhabitants may be used, and how computations involving them reduce. Later semantic models may interpret those rules as sets, domains, spaces, games, categories, or something else.

Do not confuse

Do not read $a : A$ as ordinary set membership $a \in A$ unless a particular semantics has justified that interpretation. Type membership is a judgment internal to the formal language; set membership belongs to one possible external model.

1.7 The first design question

Suppose we want a language containing booleans and natural numbers. Before we define any algorithm, we must answer:

1. Which expressions count as types?
2. Which expressions are canonical booleans and naturals?

3. Which operations are permitted on each?
4. Which compound expressions compute to which canonical forms?
5. Which equalities are visible to the type checker without running arbitrary external reasoning?

These are language-design questions, but they are also questions about the structure of computation. A type theory answers them by inference rules.

Formal Interlude I: Four primitive judgment forms

A minimal intensional type theory often distinguishes at least:

$$\begin{aligned} \Gamma \vdash A \text{ type}, \quad \Gamma \vdash a : A, \\ \Gamma \vdash A \equiv B \text{ type}, \quad \Gamma \vdash a \equiv b : A. \end{aligned}$$

The first two concern formation and inhabitation. The latter two concern *judgmental* or *definitional* equality: equality recognized by the rules of computation themselves.

The notation varies across authors. The important point is conceptual separation. “*A* is a legitimate type” and “*a* inhabits *A*” are different claims, as are “these expressions compute to the same thing” and “there exists a proposition asserting their equality.”

Worked Example 1.1: One parseable phrase, two different questions

The surface phrase `succ true` has an unambiguous abstract syntax tree:

$$\text{Succ}(\text{True}).$$

The parser therefore succeeds. Typing asks a different question. The successor rule requires a premise of the form

$$\Gamma \vdash t : \mathbb{N}.$$

But `true` synthesizes `Bool`, not `ℕ`. The derivation fails at a specific premise; no change to the parse tree is required to explain the failure.

By contrast, `if true then 0 else succ 0` parses and synthesizes `ℕ`: the guard has type `Bool`, and both branches have type `ℕ`. The example isolates three facts: parseability, typability, and result type.

Computational Laboratory 1: Syntax is not typing

Open `labs/tinytt.py`. Run the parser on the expressions

```
parse("succ 0")
parse("succ true")
```

```
parse("if true then 0 else succ 0")
```

All three can be represented as syntax trees. Then call the type checker. The second expression is rejected even though it parses. Record the distinction between a *syntactic constructor* and a *typing rule*.

Exercises

Checkpoint.

1. Explain, without using the word “error,” why `true + 3` may be grammatical but not typable.
2. Give an example of a physical action on a calculator that does not correspond to a meaningful mathematical expression.
3. In the judgment $x : \mathbb{N} \vdash x + 1 : \mathbb{N}$, identify what is assumed and what is claimed.

Core.

4. Invent a tiny language with two base types. State three expressions that parse but should not type-check.
5. Explain why interpreting every type as a set at the outset may hide distinctions relevant to computation.
6. Write four different English readings of $\Gamma \vdash a : A$: one as a programmer, one as a logician, one as a protocol designer, and one as a compiler engineer.

Synthesis.

7. A spreadsheet cell may contain a number, formula, date, or text. Analyze a spreadsheet as a primitive typed language. Where does the analogy succeed, and where does it fail?
8. Argue for or against: “A parser decides whether a sentence exists; a type checker decides whether it says something.” Refine the sentence until you are satisfied with it.

Proof Workshop

- P1. Formalize a grammar in which both `succ 0` and `succ true` parse. Then state the typing premise that distinguishes them.
- P2. Prove that “parses” does not imply “has type $\mathbb{N}$ ” by giving a concrete counterexample in TinyTT.

Design Clinic

Design a three-stage diagnostic for a tiny language that separately reports tokenization failure, parse failure, and typing failure. Give one input that reaches each stage.

Challenge.

9. Can an untyped lambda term be meaningful? Give an answer that preserves the usefulness of type theory without declaring untyped computation meaningless by fiat.

Anatomy of a judgment

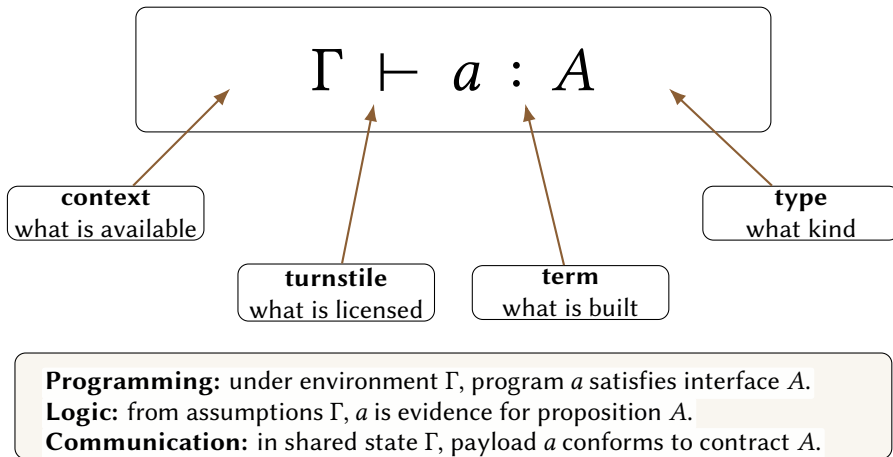


Figure 1.3: **Plate 2. Anatomy of a judgment.** The four visible parts of $\Gamma \vdash a : A$ answer different questions. The three readings at the bottom are interpretations of one formal shape, not assertions that programming, logic, and protocol semantics are identical in every respect.

Judgments: The Atomic Form of Formal Meaning

2.1 Statements about expressions, not expressions themselves

A judgment is something we assert about expressions. This distinction is easy to miss because mathematical notation often suppresses the speech act. The expression 3 is not true or false. The judgment

$$\vdash 3 : \mathbb{N}$$

may be derivable or not derivable in a formal system.

Similarly, $A \rightarrow B$ is an expression. The claim

$$\Gamma \vdash A \rightarrow B \text{ type}$$

is a judgment saying that the expression denotes a well-formed type under context Γ .

This separation is one of type theory's deepest acts of hygiene. Object language and metalanguage must not be blurred casually.

2.2 Rules as licenses to assert judgments

Judgments become useful only when governed by rules. A rule has premises above a line and a conclusion below it:

$$\frac{J_1 \quad J_2}{J_3}.$$

Read it as a license: whenever J_1 and J_2 have been established, J_3 may be asserted.

For a simple natural-number type, we might have

$$\frac{}{\Gamma \vdash \mathbb{N} \text{ type}} \quad \frac{}{\Gamma \vdash 0 : \mathbb{N}} \quad \frac{\Gamma \vdash n : \mathbb{N}}{\Gamma \vdash \text{succ}(n) : \mathbb{N}}.$$

The first two rules have no premises. They are primitive commitments of the language. The third is generative: from any known natural number, we may construct another.

Intuition

An inference rule is a small machine for manufacturing justified statements. A derivation is the record of repeatedly operating those machines.

2.3 Derivation trees

To prove that $\text{succ}(\text{succ}(0))$ has type $\mathbb{N}$, we build a tree:

$$\frac{\frac{\overline{\vdash 0 : \mathbb{N}}}{\vdash \text{succ}(0) : \mathbb{N}}}{\vdash \text{succ}(\text{succ}(0)) : \mathbb{N}}.$$

The tree is not merely a proof that the final judgment is valid. It records the construction history that licenses the claim.

Different type theories differ by the rules they admit. Changing a rule can change not only which theorems are provable but which programs exist and which computations terminate.

2.4 Canonical and noncanonical forms

The term 0 is a canonical natural number. So is $\text{succ}(0)$. But an expression such as

$$(\lambda x.x) 0$$

may have type $\mathbb{N}$ without being in canonical form. It contains computational work. Evaluation exposes the canonical content:

$$(\lambda x.x) 0 \longrightarrow 0.$$

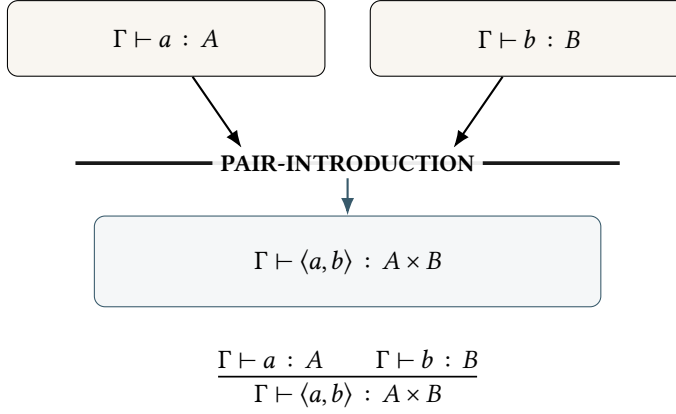
This distinction between *having a type* and *being a canonical inhabitant* will become central. Types tell us what a computation promises; normalization reveals the form of the delivered result.

2.5 Introduction, elimination, computation

For each type former, we will repeatedly ask three questions.

1. **Formation:** under what conditions is this a legitimate type?
2. **Introduction:** how are canonical inhabitants constructed?

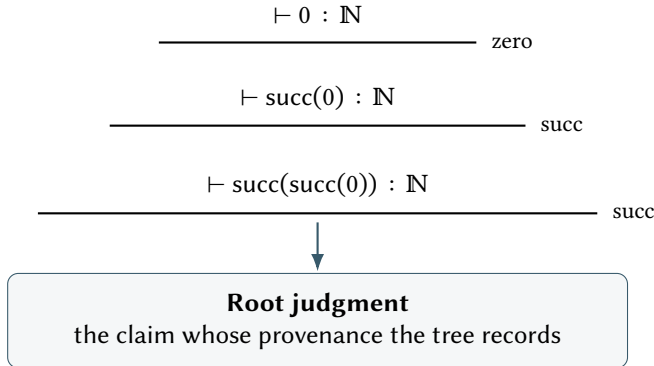
A rule manufactures a licensed judgment



The rule bar is not causal machinery. It records a license:
if the premises are derivable, so is the conclusion.

Figure 2.1: **Plate 4. Inference as licensed construction.** The press-like layout is mnemonic only. An inference rule need not describe temporal execution; it specifies a valid step in a derivation.

Derivation tree for two



Every edge can be audited against a rule.
A correct-looking conclusion with a forged premise is not a derivation.

Figure 2.2: **Plate 5. Derivation as provenance.** The tree records why a judgment is licensed. Its vertical orientation follows proof-theory convention rather than the chronological order in which an implementation must construct the tree.

3. **Elimination:** how may inhabitants be used or observed?

A fourth question connects construction to use:

4. **Computation:** what happens when an eliminator immediately consumes an introduced form?

These four questions are the recurring grammar of the book.

For products, introduction builds a pair and elimination projects a component. The computation rule says that projecting the first component of a freshly built pair returns the first thing we put in. For functions, introduction is lambda abstraction, elimination is application, and computation is beta reduction.

2.6 Why elimination rules matter

It is tempting to define a type by listing its inhabitants. But a computational type is equally characterized by what one may do with its inhabitants. If someone hands us an opaque value of product type $A \times B$, we know we may obtain an A and a B . That observational power is part of the meaning of the type.

This principle has a subtle consequence. If introduction rules say too much and elimination rules permit too much, the language may become inconsistent or computationally incoherent. Well-designed rules must fit together.

Formal Interlude II: Rule schemas and metavariables

In a rule such as

$$\frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B}{\Gamma \vdash \langle a, b \rangle : A \times B},$$

Γ , a , b , A , and B are metavariables ranging over contexts, terms, and types. The rule is a schema representing infinitely many concrete instances.

The distinction between variables inside the object language and metavariables used to describe the language is essential. We will use ordinary letters in both roles when conventional, but the reader should track the level by context.

Proposition 2.1 (Inversion for natural-number constructors). *In the natural-number fragment generated only by the zero and successor introduction rules, if*

$$\Gamma \vdash \text{succ}(n) : \mathbb{N}$$

is derivable, then $\Gamma \vdash n : \mathbb{N}$ is derivable.

Proof. Inspect the last rule of the derivation. The conclusion has outer term constructor `succ`. Among the rules of this fragment, only successor introduction can produce such a conclusion, and its unique premise is exactly $\Gamma \vdash n : \mathbb{N}$. This is a one-step inversion argument. $\square$

2.7 Derivability versus truth

A judgment can be derivable in one formal system and not another. This does not make formal reasoning arbitrary. It means that derivability is relative to explicit rules. Semantics then asks whether those rules are sound with respect to some intended interpretation.

Type theory therefore has two complementary directions:

rules $\longrightarrow$ derivations

and

models $\longrightarrow$ validation of rules.

Volume I mostly moves in the first direction. Volume VIII of the series will examine the second in depth.

Worked Example 2.1: Reading a derivation by its last rule

Suppose the conclusion is

$$\vdash \text{succ}(\text{succ}(0)) : \mathbb{N}.$$

The only rule in the natural-number fragment whose conclusion has outer constructor `succ` is successor introduction. Therefore a valid derivation must contain the immediate premise

$$\vdash \text{succ}(0) : \mathbb{N}.$$

Applying the same reasoning again forces $\vdash 0 : \mathbb{N}$. This backward reading of a derivation is called *inversion*: the shape of a derivable conclusion constrains the rule that could have produced it.

Computational Laboratory 2: A derivation checker

Implement a function `check_nat_derivation(tree)` that verifies derivation trees for zero and successor. Do not merely inspect the final term. Require each node to name the rule it claims to use and recursively verify its premises. Then construct one forged tree whose conclusion is correct but whose premise is invalid. Observe the difference between *having a true-looking conclusion* and *possessing a valid derivation*.

Exercises

Checkpoint.

1. What is the difference between the expression $A \rightarrow B$ and the judgment $\Gamma \vdash A \rightarrow B$ type?

2. Draw a derivation of $\vdash \text{succ}^3(0) : \mathbb{N}$.
3. Name the four recurring questions associated with a type former.

Core.

4. Design introduction and elimination rules for a two-valued boolean type.
5. State a plausible computation rule for your boolean eliminator.
6. Give an example showing that a well-typed term need not be canonical.

Synthesis.

7. Explain why an API is partly characterized by its elimination rules: the observations or operations clients are permitted to perform.
8. Compare a derivation tree with a provenance graph in a data-processing pipeline.

Proof Workshop

- P1. Prove Proposition 2.1 again by induction on derivation height rather than last-rule inspection. Which proof is more informative here?
- P2. Define a derivation-height function and show that every premise of a finite derivation has strictly smaller height than its conclusion node.

Design Clinic

Construct a deliberately bad natural-number rule that makes inversion ambiguous. Explain which checker diagnostic becomes less precise as a result.

Challenge.

9. Suppose we add a rule with no premises concluding $\Gamma \vdash a : A$ for every a and A . What collapses? Give the computational as well as logical consequences.

Contexts: Worlds of Assumption

3.1 Closed terms are the exception

The numeral 0 can be understood without local assumptions. Most interesting programs cannot. Consider

$$x + 1.$$

The expression is meaningful only if something is known about x . We might record

$$x : \mathbb{N} \vdash x + 1 : \mathbb{N}.$$

The context $x : \mathbb{N}$ is not a comment. It is part of the judgment.

A context records the local commitments under which an expression is meaningful. It is therefore useful to think of a context as a small world.

3.2 Context formation

Contexts themselves must be well formed. The empty context is legitimate:

$$\frac{}{\cdot \text{ ctx }}.$$

If Γ is a context and A is a type under Γ , then we may extend the world with a fresh variable:

$$\frac{\Gamma \text{ ctx} \quad \Gamma \vdash A \text{ type}}{\Gamma, x : A \text{ ctx}}.$$

The freshness condition matters: in the declarative metatheory we choose x fresh, using alpha-renaming when necessary. Concrete programming syntax may permit shadowing; TinyTT does so, with lookup selecting the nearest binder. The proofs below use the alpha-renamed, distinct-name presentation.

This is our first genuine comprehension principle. From a world Γ and a type meaningful in that world, we form a larger world in which an arbitrary inhabitant of that type is available.

Context as an informational room

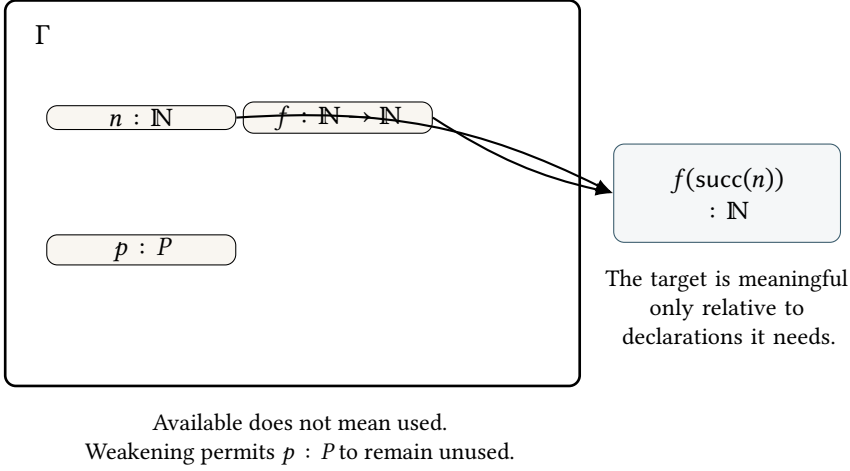
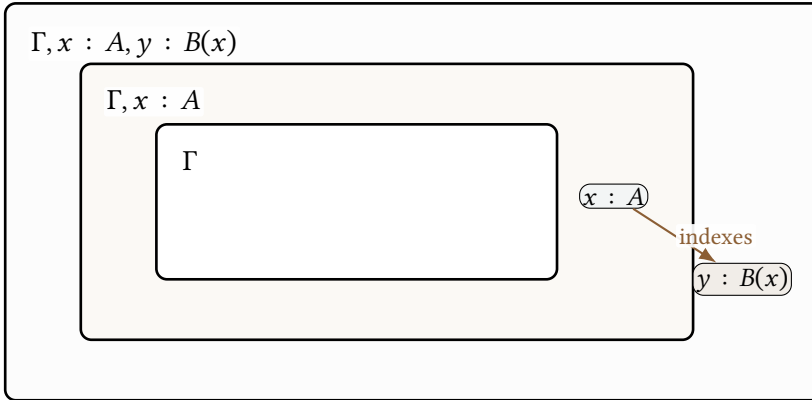


Figure 3.1: **Plate 6. A context as available information.** The room metaphor emphasizes locality and availability. Contexts are formal sequences of declarations, not literal containers, and richer type theories impose dependency-sensitive ordering.

Context extension is world extension



Later declarations may be meaningful only because earlier information is already available.

Figure 3.2: **Plate 7. Context extension.** The nested frames depict inclusion of available assumptions. They should not be read as set inclusion in the dependent case: order and dependency are part of the formal structure.

3.3 The variable rule

Once a declaration appears in the context, we may use it:

$$\overline{\Gamma, x : A, \Delta \vdash x : A}.$$

The notation allows declarations after x because later assumptions may depend on earlier ones in richer theories. In the simple fragment developed here, we can read the context as a typed environment.

The variable rule seems trivial, but conceptually it converts a hypothetical commitment into a usable term. By extending the context with $x : A$, we say: “reason as if an arbitrary A has been supplied.” The variable x is our local handle on that hypothetical information.

3.4 Contexts are ordered when dependency matters

If declarations never depend on one another, a context may look like a set of assumptions. Dependent type theory reveals why that view is incomplete. Consider

$$n : \mathbb{N}, xs : \text{Vec}(A, n).$$

The declaration of xs mentions n . We cannot in general swap the declarations:

$$xs : \text{Vec}(A, n), n : \mathbb{N}$$

because n has not yet been introduced when the first declaration is checked.

Thus a context is more like an ordered construction history than an unordered bag.

3.5 Weakening: unused information may be carried

Suppose

$$\Gamma \vdash a : A.$$

If we extend the context with a declaration unrelated to the derivation, the old term should remain valid:

$$\Gamma, x : B \vdash a : A.$$

This principle is called **weakening**. It says that merely learning an additional fact does not invalidate an already meaningful construction, provided the extension is well formed.

Not every type system accepts unrestricted weakening. Linear and relevance-sensitive systems deliberately constrain whether assumptions may go unused. The ordinary structural rule therefore encodes a computational policy: information may be ignored.

3.6 Exchange and contraction as computational policies

Two other familiar structural rules are **exchange**, which permits reordering independent assumptions, and **contraction**, which permits one assumption to be used more than once.

These rules are easy to treat as invisible bureaucracy. They are better understood as resource assumptions.

Contraction licenses copying. If a variable $x : A$ may occur twice in a term, the information denoted by x has been duplicated at the level of use. Linear logic and linear type systems remove unrestricted contraction precisely to model resources that cannot be copied freely.

Weakening licenses disposal. If $x : A$ need not occur in the result, information may be discarded.

Ordinary programming languages usually grant both privileges silently. Type theory can surface them.

Formal Interlude III: Structural rules

For suitable well-formed contexts, the classical structural principles can be displayed schematically as:

$$\frac{\Gamma \vdash J}{\Gamma, x : A \vdash J} \quad (\text{weakening}),$$

$$\frac{\Gamma, x : A, y : B, \Delta \vdash J}{\Gamma, y : B, x : A, \Delta \vdash J} \quad (\text{exchange, when dependencies permit}),$$

$$\frac{\Gamma, x : A, y : A, \Delta \vdash J}{\Gamma, x : A, \Delta[x/y] \vdash J[x/y]} \quad (\text{contraction}).$$

The exact formulations vary with the theory. Their conceptual significance is stable: they control disposal, reordering, and duplication of assumptions.

Theorem 3.1 (Weakening for the simple core). *Assume formal contexts have distinct variable names. If $\Gamma \vdash t : T$ and Γ' is obtained from Γ by inserting fresh, well-formed declarations without changing any existing binding, then $\Gamma' \vdash t : T$.*

Proof sketch. Induct on the derivation of $\Gamma \vdash t : T$. In the variable case, the declaration used by the variable remains present with the same type. Every nonbinding constructor case follows by applying the induction hypotheses to the premises and reconstructing the same rule. In a binder case, alpha-rewrite the bound variable if necessary so that it is fresh for the inserted declarations; then apply the induction hypothesis beneath the binder and rebuild the rule. Because the simple core has no term-dependent types, inserted declarations do not alter the types of existing declarations. $\square$

3.7 Context as interface state

A context resembles more than a list of variable declarations. It can be viewed as a snapshot of an interaction state. Before a function receives its argument, the argument is unavailable. After binding $x : A$, the local computational world changes.

This anticipates session types and protocol systems. There, the “context” may record which channels exist and which messages are legal next. The general idea is the same: the meaning of an expression is relative to a state of available capabilities.

Worked Example 3.1: A judgment uses only part of its world

Let

$$\Gamma = n : \mathbb{N}, \quad f : \mathbb{N} \rightarrow \mathbb{N}, \quad p : P.$$

The judgment $\Gamma \vdash fn : \mathbb{N}$ is derived from the declarations for f and n . The assumption $p : P$ is carried but unused. Weakening explains why adding $p : P$ does not invalidate the earlier derivation. Contraction is not needed because neither declaration is duplicated, and exchange is harmless only for declarations that do not depend on one another.

Computational Laboratory 3: Context lookup

In `tinytt.py`, contexts are represented as ordered lists of pairs. Implement or inspect:

```
def lookup(ctx, name):
    for n, ty in reversed(ctx):
        if n == name:
            return ty
    raise TypeError(f"unbound variable: {name}")
```

Experiment with shadowing by adding two declarations with the same printed name. Then modify the representation so that variables carry unique internal identifiers. The exercise demonstrates why human-readable names are not reliable identities.

Exercises

Checkpoint.

1. Explain why $x + 1$ cannot be assigned a type without information about x .
2. State the context-formation rule in words.
3. What computational privileges correspond to weakening and contraction?

Core.

4. Give a context whose declarations cannot be reordered once dependent types are admitted.
5. Construct a term that uses a variable twice and explain why contraction is implicated.
6. Construct a term whose type derivation uses weakening.

Synthesis.

7. Treat a shell session as a context. Which facts and capabilities accumulate as commands execute?
8. Explain how a security capability system can be seen as a context discipline.

Proof Workshop

- P1. Prove weakening for the variable and application cases of the core calculus in full detail.
- P2. Give an example showing why exchange needs a dependency side condition once declarations such as $y : B(x)$ are allowed.

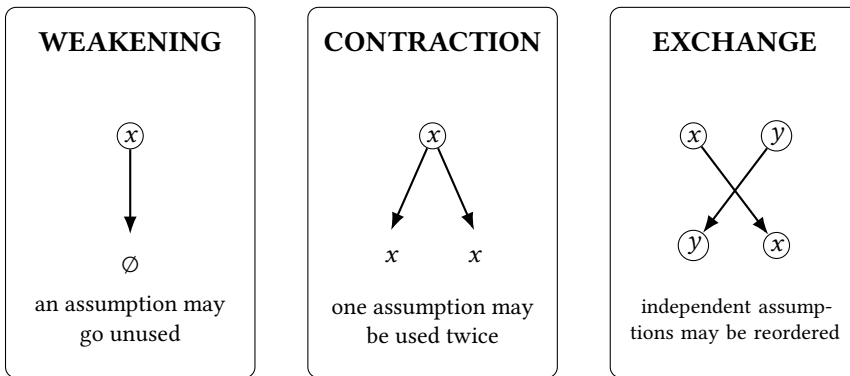
Design Clinic

Redesign TinyTT contexts so shadowing is impossible internally while user-facing names remain readable. State the invariant your representation enforces.

Challenge.

9. Imagine a language in which variables must be used exactly once. Which familiar programming patterns become awkward? Which resource-sensitive patterns become easier to reason about?

Structural rules are resource policies



Linear and relevance-sensitive systems deliberately revoke some of these privileges.

Figure 3.3: **Plate 8. Structural rules as computational assumptions.** Disposal, duplication, and reordering are represented as resource operations. The picture does not imply physical copying costs; it identifies permissions encoded by ordinary structural typing.

Variables and Dependency: Naming What the Future May Supply

4.1 A variable is a promise, not a mystery

Beginners often encounter variables as unknown numbers. In type theory, that picture is too narrow. A variable is a local name for information whose kind is known even when its particular value is not.

In

$$x : A \vdash t : B,$$

x should be read as “an arbitrary future inhabitant of A .” The term t is constructed under that promise. If t works only for special x , then its derivation must make those restrictions explicit.

This interpretation changes the emotional character of symbolic reasoning. Variables are not holes caused by ignorance. They are instruments for expressing uniformity.

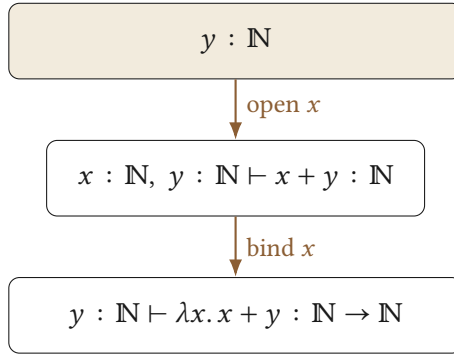
4.2 Free and bound occurrences

Consider

$$\lambda x. x + y.$$

The occurrence of x is bound by the lambda; the occurrence of y is free. The term may be meaningful under a context containing y , whereas it supplies its own local assumption for x .

Binding is the formal mechanism by which a language opens a temporary hypothetical world.



The variable is introduced locally, used in a construction, and then discharged into the function type. This pattern recurs across logic: assume something, reason under the assumption, then package the dependency.

4.3 Alpha-equivalence: names are not identities

The terms

$$\lambda x.x \quad \text{and} \quad \lambda z.z$$

represent the same binding structure. Renaming a bound variable should not alter the program. This principle is called **alpha-equivalence**.

Printed names are conveniences for humans. A rigorous implementation may use de Bruijn indices, globally unique identifiers, or locally nameless representations so that accidental name collisions cannot change meaning.

Do not confuse

The expression $\lambda x.y$ is not alpha-equivalent to $\lambda y.y$. Renaming the binder from x to y captured a formerly free occurrence. Alpha-renaming must avoid changing binding structure.

4.4 Dependency before dependent types

Even in simply typed lambda calculus, terms depend on variables. The term

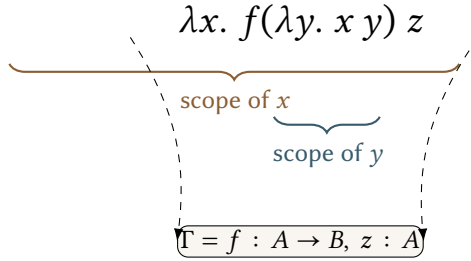
$$f(x)$$

depends on both f and x . What simple types do not yet permit is for a *type itself* to depend on a term.

Dependent type theory allows judgments such as

$$n : \mathbb{N} \vdash \text{Vec}(A, n) \text{ type.}$$

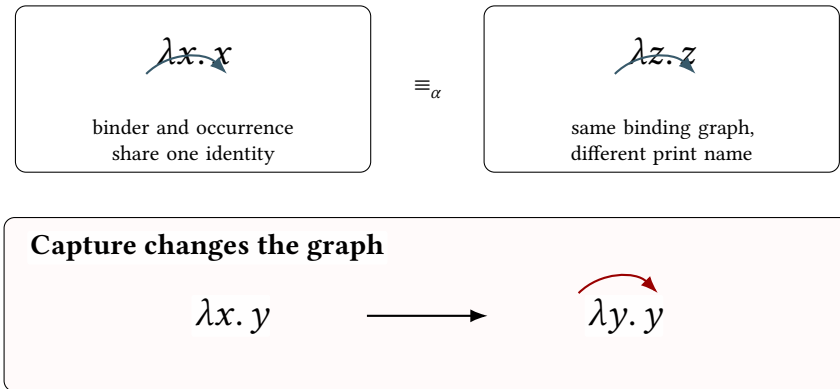
Bound and free variables



x and y are licensed by binders; f and z must be licensed by the surrounding context.

Figure 4.1: **Plate 9. Scope is an information boundary.** Braces and tethers show which occurrences are locally bound and which remain obligations on the external context. Printed position is incidental; binding structure is the invariant.

Alpha-equivalence preserves binding identity



Renaming is valid only when it leaves the binder-occurrence relation unchanged.

Figure 4.2: **Plate 10. Alpha-equivalence.** Variable names are replaceable labels on binding identities. The lower panel shows why freshness conditions matter: a textual rename can turn an external dependency into a local one.

This says that the type of vectors is indexed by a value. Different information about n changes which type is under discussion.

We will not fully develop dependent types in Volume I, but we introduce the idea now because it reveals what contexts and substitution are really for.

4.5 Families as maps from information to possibility

Think of a type family $B(x)$ as assigning a type to every possible $x : A$:

$$x \mapsto B(x).$$

The base A indexes a collection of fibers. For every point x in the base, there is a corresponding space of possible constructions $B(x)$.

A dependent function will later choose an inhabitant in every fiber. A dependent pair will choose a base point together with an inhabitant of the corresponding fiber. The notation will look more sophisticated, but the underlying idea is simple: information received now determines which possibilities are meaningful next.

4.6 Scope is a computational boundary

A binder creates a scope. Inside it, a variable is available. Outside it, the variable is not. This is often taught as punctuation. More usefully, scope marks a boundary of available information.

When a function returns, its local variables disappear unless their information has been encoded into the result. When a proof discharges an assumption, the conclusion can no longer rely on the assumption except through the structure of the proposition proved. Scope therefore implements a form of information hygiene.

Formal Interlude IV: Free-variable discipline

Let $\text{FV}(t)$ denote the free variables of a term. A basic well-scopedness invariant is:

$$\Gamma \vdash t : A \implies \text{FV}(t) \subseteq \text{dom}(\Gamma).$$

This is not yet the full typing theorem. It states that every free name used by a well-typed term must be accounted for by its context.

For a binder,

$$\text{FV}(\lambda x.t) = \text{FV}(t) \setminus \{x\}.$$

For application,

$$\text{FV}(t u) = \text{FV}(t) \cup \text{FV}(u).$$

These equations already predict how context requirements compose.

Theorem 4.1 (Free-variable adequacy). *For the simple core, if $\Gamma \vdash t : T$, then every free variable of t is declared in Γ :*

$$\text{FV}(t) \subseteq \text{dom}(\Gamma).$$

Proof sketch. Induct on the typing derivation. In the variable case the claim is exactly context membership. Applications and pairs use set union together with the induction hypotheses. In a lambda case, the body is checked under $\Gamma, x : A$; the induction hypothesis therefore accounts for all free variables of the body by $\text{dom}(\Gamma) \cup \{x\}$, and removing the bound variable x leaves only declarations from Γ . Case binders are analogous. $\square$

Worked Example 4.1: Renaming that changes a program

In $\lambda x.y$, the occurrence of y is free. Renaming the binder x to y naively gives $\lambda y.y$, where the occurrence is now bound. The printed change looks local, but the dependency graph changed globally.

A legal alpha-renaming first chooses a fresh name, for example

$$\lambda x.y \equiv_{\alpha} \lambda z.y.$$

The criterion is therefore not “the letters look different” but “the binder-occurrence relation is unchanged.”

Computational Laboratory 4: Capture-avoiding renaming

Implement `free_vars(term)` and `rename_bound(term, old, new)`. Test the dangerous input

```
Lam("x", Var("y"))
```

when renaming `x` to `y`. A correct implementation must first choose a fresh name or use an internal representation where capture is impossible.

Exercises

Checkpoint.

1. Identify the free variables of $(\lambda x.f x) y$.
2. Why are $\lambda x.x$ and $\lambda z.z$ considered the same binding structure?
3. Explain a variable as a promise rather than an unknown.

Core.

4. Give an example of variable capture caused by careless renaming.

5. Draw scopes for $\lambda x.\lambda y.x$ and $\lambda x.\lambda y.y$.
6. Describe, in plain language, what a type family $B(x)$ does.

Synthesis.

7. Compare lexical scope in programming languages with the visibility of assumptions inside a mathematical proof.
8. Explain why dependency makes context order significant.

Proof Workshop

- P1. Prove the lambda case of Theorem 4.1 using the equation for free variables of a binder.
- P2. Show that alpha-equivalence preserves the set of free variables.

Design Clinic

Represent the same term once with names and once with de Bruijn indices. Compare which representation makes alpha-equivalence trivial and which makes error messages easier.

Challenge.

9. Research or derive the de Bruijn representation of $\lambda x.\lambda y.x$. Explain what problem the representation solves and what readability cost it introduces.

A type family assigns a space of possibilities

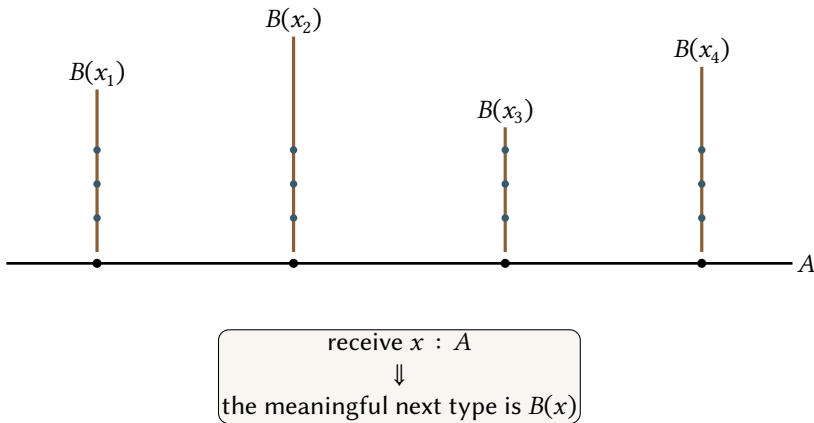


Figure 4.3: **Plate 11. Fibres of a type family.** Geometry supplies an intuition: a base point chooses a corresponding fibre. At this stage the picture is mnemonic; Volume II will make dependent formation and substitution precise, and later semantic volumes will say when spatial language is literal.

Substitution: Information in Motion

5.1 The central dynamic

If context extension creates a hypothetical world, substitution closes the loop by supplying actual information.

Suppose

$$\Gamma, x : A \vdash t : B$$

and

$$\Gamma \vdash a : A.$$

Then we should be able to replace the hypothetical x by the actual construction a :

$$\Gamma \vdash t[a/x] : B[a/x].$$

In the simply typed setting B does not depend on x , so the conclusion is just

$$\Gamma \vdash t[a/x] : B.$$

The dependent notation reveals the more general principle: substitution propagates newly supplied information through both programs and the types that describe them.

Governing Principle

Context extension is the act of saying “suppose an x is available.” Substitution is the act of saying “now here is the x .” Together they form the basic open-close cycle of hypothetical computation.

5.2 A simple example

Take

$$x : \mathbb{N} \vdash \text{succ}(x) : \mathbb{N}$$

and

$$\vdash 0 : \mathbb{N}.$$

Substituting 0 for x gives

$$\vdash \text{succ}(0) : \mathbb{N}.$$

Nothing surprising occurs. That lack of surprise is exactly what the substitution theorem formalizes. Replacing a variable by a term of the type promised for that variable preserves meaningfulness.

5.3 Substitution is not textual search-and-replace

Consider

$$(\lambda y.x)[y/x].$$

Naive textual replacement yields

$$\lambda y.y,$$

but that changes the meaning. The supplied y was free before substitution and becomes captured by the binder.

We must first rename the binder:

$$\lambda z.x$$

and then substitute:

$$(\lambda z.x)[y/x] = \lambda z.y.$$

This is **capture-avoiding substitution**.

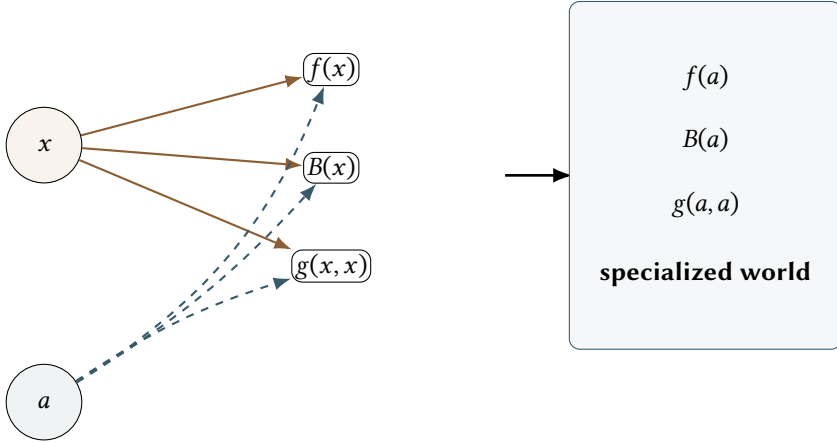
The bookkeeping may appear technical. Conceptually it enforces an information invariant: supplying a value for one dependency must not accidentally alter which other dependencies are local or external.

5.4 Substitution as graph rewriting

A useful mental picture is a dependency graph. The variable x is a source node whose value flows to all places that refer to it. Substitution rewires those references to a concrete construction.

This picture scales. In compiler optimization, substitution resembles inlining. In symbolic algebra, it resembles instantiation. In logic, it specializes a hypothetical derivation. In dependent types, it changes the very proposition or type being discussed.

Substitution is information in motion



Supplying $a : A$ discharges the hypothetical source $x : A$ and specializes every genuine dependency.

Figure 5.1: **Plate 12. Substitution as dependency rewiring.** The graph picture tracks dependency, not machine memory addresses. Capture avoidance ensures that replacing one dependency does not silently alter the binding identity of another.

Capture avoidance preserves external dependencies

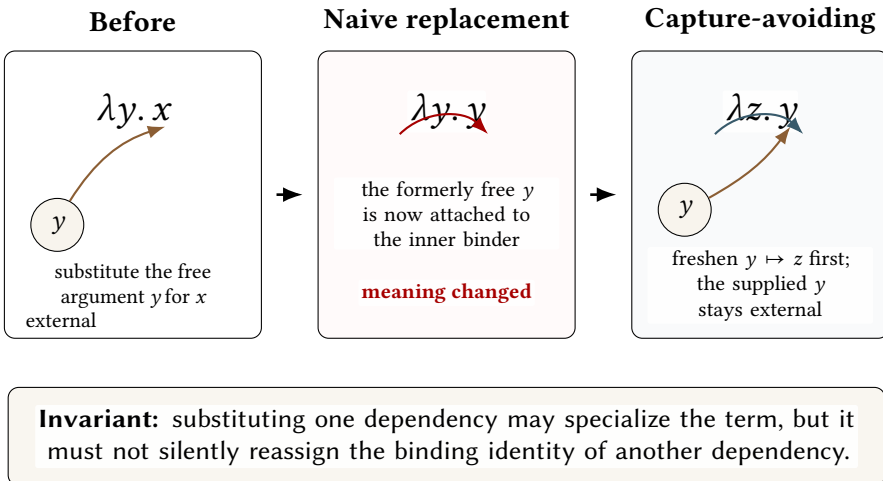


Figure 5.2: **Plate 13. Capture-avoiding substitution.** The tether picture records binding identity, not pointer structure in an implementation. Freshening a binder is required exactly when a free variable in the incoming term would otherwise become captured.

5.5 Beta reduction is substitution exposed

Function application makes substitution computationally visible:

$$(\lambda x.t) a \longrightarrow t[a/x].$$

The left side packages a hypothetical computation and immediately supplies the promised input. The right side is the resulting specialization.

This is beta reduction. But the name can hide the idea. Beta reduction is simply the computational event in which an abstract dependency receives concrete information.

5.6 The substitution lemma

The structural heart of most type systems is a statement of the form:

Formal Interlude V: Substitution Lemma

If

$$\Gamma, x : A, \Delta \vdash t : T$$

and

$$\Gamma \vdash a : A,$$

then, under the usual well-formedness conditions,

$$\Gamma, \Delta[a/x] \vdash t[a/x] : T[a/x].$$

The proof is generally by induction on the typing derivation of t . Each typing rule must be shown stable under substitution. This is one reason substitution is a design test for type systems: rules that fail to compose with substitution usually signal a deeper incoherence.

Theorem 5.1 (Substitution for the simple core). *Assume formal contexts have distinct names. If*

$$\Gamma, x : A, \Delta \vdash t : T \quad \text{and} \quad \Gamma \vdash a : A,$$

then

$$\Gamma, \Delta \vdash t[a/x] : T.$$

Here Δ is unchanged because types in the simple core do not contain term variables.

Proof sketch. Induct on the typing derivation of t , with the trailing context Δ carried as part of the induction statement. The critical variable case splits on whether the variable is x . If it is x , weakening lifts $\Gamma \vdash a : A$ to $\Gamma, \Delta \vdash a : A$; otherwise the variable's unchanged declaration remains in Γ, Δ . Constructor cases commute substitution through their subterms and use the induction hypotheses. For a binder $\lambda y.u$, alpha-rename so that

y is fresh for a , x , and the surrounding context. The body is typed under $\Gamma, x : A, \Delta, y : C$, so the induction hypothesis applies with trailing context $\Delta, y : C$, yielding $\Gamma, \Delta, y : C \vdash u[a/x] : D$; rebuild the lambda. Case-expression binders are analogous. This generalized statement is exactly what licenses substitution beneath newly introduced binders. $\square$

Corollary 5.2 (Beta preservation). *If $\Gamma \vdash (\lambda x.t) a : B$, then the beta reduct $t[a/x]$ also has type B .*

Proof. Inversion of application and lambda typing gives $\Gamma, x : A \vdash t : B$ and $\Gamma \vdash a : A$ for some A . Apply Theorem 5.1. $\square$

5.7 Why the lemma matters for execution

Suppose a well-typed application reduces:

$$(\lambda x.t) a \longrightarrow t[a/x].$$

The typing rule for application tells us that the left side has some type B . The substitution lemma tells us that the right side still has type B . Thus the fundamental computational step does not break the typing guarantee.

This is a prototype of **preservation**: evaluation preserves types.

The other half of type safety is **progress**: a closed well-typed term is either already a value or can take a computational step. Together, preservation and progress yield the familiar slogan “well-typed programs do not get stuck” for the language fragment under consideration.

Intuition

Preservation says the train stays on the track when it moves. Progress says a correctly placed train is either at a station where it is supposed to stop or has some legal next segment of track.

Worked Example 5.1: Beta reduction with a free argument

Consider

$$(\lambda x.\lambda y.x) y.$$

The argument y is free. Direct replacement would produce $\lambda y.y$ and capture it. Capture-avoiding substitution first alpha-renames the inner binder:

$$(\lambda x.\lambda z.x) y \longrightarrow \lambda z.y.$$

The result still depends on the external y , exactly as the application did before reduction.

Computational Laboratory 5: Capture-avoiding substitution

Use the implementation in `tinytt.py` to evaluate

```
App(Ann(Lam("x", Succ(Var("x"))), Arrow(NAT, NAT)), Zero())
```

and verify that it becomes `Succ(Zero())`. Then construct a term where the argument contains a free variable whose printed name conflicts with an inner binder. Inspect the fresh-name step performed by substitution.

5.8 Substitution beyond syntax

The same pattern appears in many guises:

- A generic function is specialized to a concrete type parameter.
- A theorem with a universally quantified variable is instantiated at a particular value.
- A template receives configuration data.
- A query plan receives runtime parameters.
- A protocol state receives a message and transitions to a more specialized future state.

Not all of these are literally syntactic substitution. The analogy is useful because they share a form: an open construction acquires information, and its remaining possibilities are recomputed in light of that information.

Exercises

Checkpoint.

1. Compute $(x + x)[3/x]$.
2. Explain why $(\lambda y.x)[y/x]$ cannot be implemented as naive replacement.
3. State beta reduction in symbols and in words.

Core.

4. Perform capture-avoiding substitution for $(\lambda y.\lambda z.x\ y)[y/x]$.
5. Explain how the substitution lemma supports preservation.
6. Give an example of substitution into a type family such as $\text{Vec}(A, n)$.

Synthesis.

7. Compare beta reduction with function inlining in a compiler. Which properties are shared? Which are not?
8. Describe a database prepared statement as an open term awaiting substitution.

Proof Workshop

- P1. Complete the abstraction case of Theorem 5.1, explicitly stating the freshness condition and where alpha-renaming is used.
- P2. Prove the variable cases $x[a/x] = a$ and $y[a/x] = y$ for $y \neq x$ preserve typing under the theorem's hypotheses.

Design Clinic

Instrument **subst** to emit a trace whenever it freshens a binder. Create the smallest term that triggers the trace and explain why freshening is semantically necessary.

Challenge.

9. Sketch an induction proof of substitution for variables, abstraction, and application in simply typed lambda calculus.

Beta reduction closes a hypothetical world

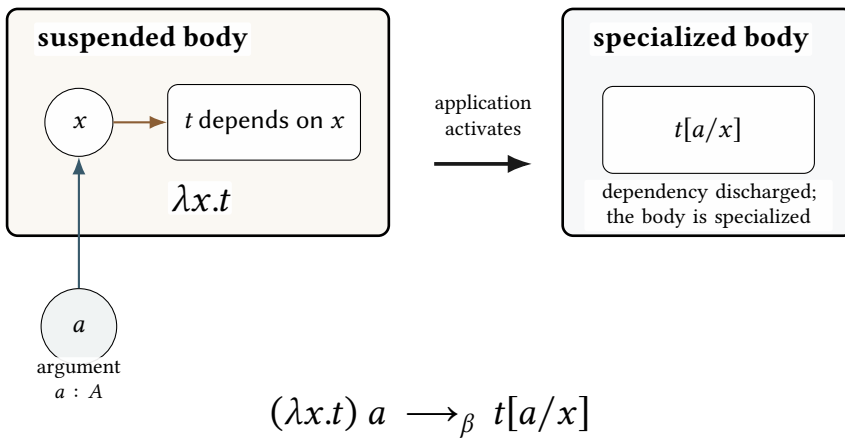


Figure 5.3: **Plate 14. Beta reduction.** The socket metaphor emphasizes discharge of a hypothetical dependency. It does not prescribe an evaluator's runtime representation: closures, environments, graph reduction, and substitution machines can realize the same rule differently.

Rules: How a Language Grows

6.1 A language is a constitution

A formal language is not merely a vocabulary plus grammar. Its inference rules determine which constructions are legitimate and how those constructions behave. Adding a type former is therefore closer to amending a constitution than adding a word to a dictionary.

For every new type constructor we must ask whether its formation, introduction, elimination, and computation rules fit the existing system. Do they preserve substitution? Do they create ambiguous reductions? Do they preserve normalization? Are equalities decidable? Do they permit inconsistent judgments?

The design discipline matters because local-looking rules can have global consequences.

6.2 The harmony principle

Suppose a type's introduction rules tell us how to create inhabitants, and its elimination rules tell us how to consume them. The two should be in harmony. The proof-theoretic background is Gentzen's natural deduction and its later normalization analysis, especially Prawitz's systematic study [5, 13].

For a product, if the canonical way to create an inhabitant is

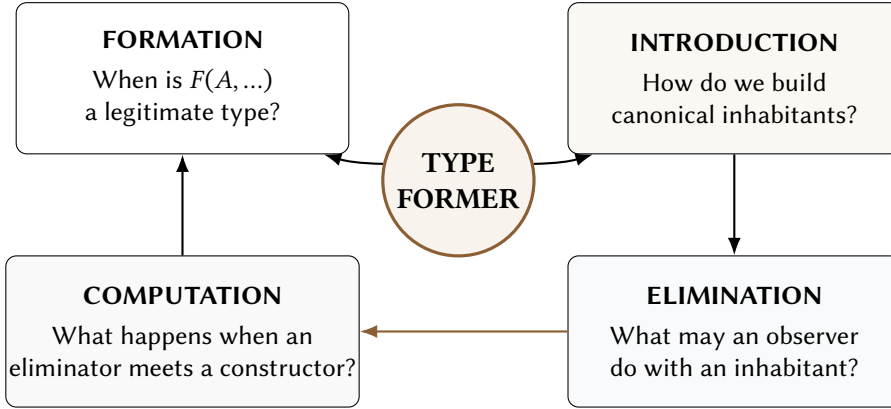
$$\langle a, b \rangle : A \times B,$$

then the eliminators should reveal exactly the information introduced: first and second projections.

If an eliminator could magically produce a C unrelated to either component, it would extract information that was never put in. If the eliminators revealed less than the introduction supplied, the type could carry observationally inaccessible structure.

This “no more and no less” intuition underlies proof-theoretic accounts of logical connectives and guides type-former design.

The four questions for every type former



A new connective is not specified by syntax alone. These four questions determine its admissible objects, its evidence, its observations, and the reductions generated where evidence meets observation.

Figure 6.1: **Plate 15. The type-former compass.** Formation, introduction, elimination, and computation are a design discipline rather than four universally independent rule schemas. Some systems package or derive parts of the quartet differently, but the four questions remain a useful audit.

Harmony: observation recovers construction

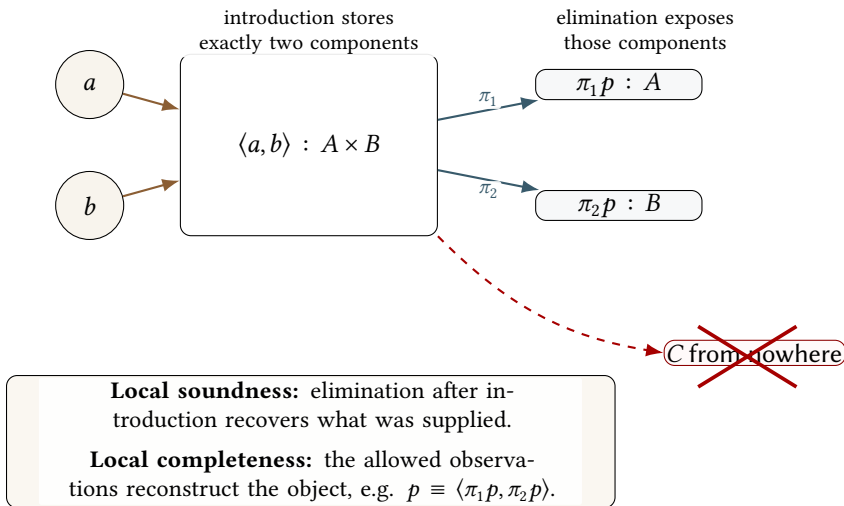


Figure 6.2: **Plate 16. Proof-theoretic harmony.** The product container is mnemonic. Harmony relates introduction and elimination rules; not every type former has literal fields or projections.

6.3 Local soundness

One direction of harmony asks whether elimination after introduction gives back what we expect. For products:

$$\pi_1\langle a, b \rangle \longrightarrow a, \quad \pi_2\langle a, b \rangle \longrightarrow b.$$

For functions:

$$(\lambda x.t) a \longrightarrow t[a/x].$$

These reductions express *local soundness*: the eliminator does not obtain more information than the constructor supplied.

6.4 Local completeness

The converse asks whether every inhabitant can be reconstructed from the observations its eliminators permit. For products, one expects an eta principle such as

$$p \equiv \langle \pi_1 p, \pi_2 p \rangle.$$

For functions, an eta principle says roughly

$$f \equiv \lambda x.f x$$

when x is not free in f .

Eta laws say that the observations provided by the eliminators are complete enough to characterize the object.

6.5 Rules generate computation

The moment we add introduction and elimination forms, we have created redexes: places where an eliminator meets a constructor. Computation rules resolve those meetings.

This suggests a powerful design heuristic:

Governing Principle

To understand a type, ask what counts as a canonical value and what an observer is allowed to do with an unknown value. Computation is the conversation between those two answers.

6.6 An example: the unit type

The unit type **1** has one canonical inhabitant, written $\star$:

$$\frac{}{\Gamma \vdash \mathbf{1} \text{ type}} \quad \frac{}{\Gamma \vdash \star : \mathbf{1}}.$$

What does it mean to eliminate a unit value? Since there is no informative choice to inspect, elimination says: if we can build $c : C$ assuming the unit value is $\star$, then we can build c from any $u : \mathbf{1}$. In a simple nondependent presentation, unit carries no data.

Computationally, **1** is useful precisely because it represents the presence of a result without informative payload: acknowledgments, completed actions, trivial proofs, and placeholder values.

6.7 An example: the empty type

The empty type **0** has no introduction rule. There is no canonical inhabitant.

Its eliminator says that from an inhabitant of **0**, anything follows:

$$\frac{\Gamma \vdash e : \mathbf{0}}{\Gamma \vdash \text{absurd}_C(e) : C}.$$

This is computationally sensible because the eliminator can never be invoked on a closed canonical value: there is none.

Logically, **0** behaves like falsehood. Computationally, a function $\mathbf{0} \rightarrow C$ is total because it never receives an argument.

6.8 Rule inversion as reasoning

Typing rules can be read backward. If we know

$$\Gamma \vdash \langle a, b \rangle : A \times B,$$

we can often infer that

$$\Gamma \vdash a : A \quad \text{and} \quad \Gamma \vdash b : B.$$

This technique is called inversion. It is central to metatheoretic proofs and type-checker implementation.

The visual idea is simple: the outermost syntactic constructor limits which typing rule could have produced the judgment.

Formal Interlude VI: Syntax-directedness

A typing relation is especially convenient for implementation when the outer form of the term determines which rule to inspect. Such a system is called *syntax directed*. Not every elegant declarative type system is syntax directed. Subsumption, implicit coercions, polymorphism, and definitional equality can introduce nondeterminism. Practical type checkers often transform declarative rules into algorithmic or bidirectional forms.

Proposition 6.1 (Uniqueness of synthesized types). *For TinyTT’s annotated simple core, if the same term synthesizes both A and B in the same context, then $A = B$ syntactically. In particular, the synthesized types are definitionally equal.*

Proof sketch. Induct on the syntax of synthesizing terms. A variable’s type is determined by context lookup; an annotation exposes exactly its written type; application exposes the unique codomain of the synthesized function type; projections expose fixed components of a synthesized product; zero and successor synthesize $\mathbb{N}$; and the nondependent recursor synthesizes the type of its zero case after checking the step at $C \rightarrow C$. Terms such as bare lambdas and injections are intentionally excluded from synthesis and must be checked against an expected type. $\square$

Worked Example 6.1: Deriving a boolean instead of postulating one

A two-valued boolean can be represented by $\mathbf{1} + \mathbf{1}$. Define

$$\text{false} = \text{inl}(\star), \quad \text{true} = \text{inr}(\star).$$

An `if` expression is then sum elimination. The two branches must return the same result type because the eliminator must provide one contract for the whole case analysis. This construction demonstrates the design test for a new type former: constructors determine what information is present; eliminators may consume exactly that information.

Computational Laboratory 6: Add a type former

Extend `tinytt.py` with `UnitTy` and `UnitVal`. Add formation/typing support and confirm that `UnitVal()` checks against `UnitTy()`. Then add an empty type with no value constructor and an `Absurd` eliminator. Notice that the implementation mirrors the formation/introduction/elimination analysis almost line for line.

Exercises

Checkpoint.

1. What are the introduction forms for product, function, unit, and empty types?
2. What is meant by “elimination after introduction”?
3. Why is there no introduction rule for $\mathbf{0}$?

Core.

4. Propose computation rules for booleans with an **if** eliminator.
5. Explain the product eta law in plain language.
6. Give an example of a type former whose eliminator would be too strong and explain the resulting problem.

Synthesis.

7. Analyze a file handle type in terms of introduction and elimination. Which operations should be legal? What resource rules might be needed?
8. Compare an inference-rule constitution with an API specification. What does each omit that the other usually includes?

Proof Workshop

- P1. For products, derive both beta laws and the eta law from the formation/introduction/elimination/computation template. Mark which are reduction rules and which may instead be definitional equalities.
- P2. Give a type former whose proposed eliminator extracts information not supplied by any constructor. Show the resulting collapse with a concrete derivation.

Design Clinic

Add a `Maybe(A)` surface form and elaborate it to $A + \mathbf{1}$. Specify which layer—surface language, elaborator, or kernel—needs to know the new notation.

Challenge.

9. Investigate the phrase “proof-theoretic harmony.” Explain why beta and eta principles can be viewed as two directions of harmony.

Functions: Packaging Hypothetical Computation

The typed lambda-calculus lineage used here is historically anchored by Church’s integration of lambda abstraction with simple types [2]; our notation and operational presentation are modernized.

7.1 A function begins as a hypothetical

Suppose that under the assumption $x : A$ we can construct a term $t : B$:

$$\Gamma, x : A \vdash t : B.$$

If the construction works for an arbitrary x , we may package the entire hypothetical computation into a function:

$$\Gamma \vdash \lambda x. t : A \rightarrow B.$$

This is the introduction rule for function types.

Notice what has happened. We have not built a table of input-output pairs. We have built a method that remains suspended until information of type A arrives. Lambda abstraction is therefore a mechanism for closing over a hypothetical world.

7.2 The function type as a capability

The judgment

$$f : A \rightarrow B$$

should be read behaviorally: f is a capability that can transform any admissible A into a B . The elimination rule makes that capability usable:

$$\frac{\Gamma \vdash f : A \rightarrow B \quad \Gamma \vdash a : A}{\Gamma \vdash fa : B}.$$

Application is thus an interface match. The caller provides exactly the kind of evidence the function contract requests.

7.3 Beta: opening a package

When the function was just introduced and is immediately eliminated,

$$(\lambda x.t) a,$$

we know exactly what should happen:

$$(\lambda x.t) a \longrightarrow t[a/x].$$

Beta reduction is not an arbitrary operational convention. It is forced by the meaning of lambda abstraction as a packaged hypothetical derivation.

7.4 Eta: functions are determined by application behavior

Suppose $f : A \rightarrow B$. Then

$$\lambda x.f x$$

does nothing except accept an argument and pass it to f . Extensionally, this should be the same function:

$$f \equiv \lambda x.f x.$$

Eta expresses observational completeness: a function has no hidden observable content beyond what it does when applied.

Different intensional type theories treat eta differently: as definitional equality, propositional equality, or a theorem under extensionality principles. The distinction becomes important later.

7.5 Higher-order computation

Once functions are terms, they can themselves be inputs and outputs:

$$(A \rightarrow B) \rightarrow C, \quad A \rightarrow (B \rightarrow C).$$

This single move yields higher-order programming, proof combinators, callbacks, iterators, middleware, and much of the compositional vocabulary of modern software.

The familiar composition operator has type

$$(B \rightarrow C) \rightarrow (A \rightarrow B) \rightarrow A \rightarrow C.$$

Its implementation

$$\lambda g.\lambda f.\lambda x.g(f(x))$$

is almost a direct reading of the type from left to right.

This illustrates a general skill: a sufficiently informative type can act as a search specification for a program.

7.6 Currying reveals dependency structure

A two-argument function is commonly written

$$f : A \times B \rightarrow C.$$

But it may also be represented as

$$f : A \rightarrow (B \rightarrow C).$$

The latter says: receive an A , then return a capability waiting for a B . Operationally, partial application becomes natural.

This staged reading matters because dependent function types generalize it. After receiving $x : A$, the type of the remaining computation may itself depend on x .

7.7 Functions and implication

Under Curry-Howard, $A \rightarrow B$ can be read as implication $A \Rightarrow B$. A term of type $A \rightarrow B$ is then a method that transforms evidence of A into evidence of B .

The introduction rule becomes familiar mathematical reasoning:

Assume A . Under that assumption derive B . Discharge the assumption and conclude $A \Rightarrow B$.

The lambda binder records precisely which assumption is discharged.

Formal Interlude VII: Function rules

Formation:

$$\frac{\Gamma \vdash A \text{ type} \quad \Gamma \vdash B \text{ type}}{\Gamma \vdash A \rightarrow B \text{ type}}.$$

Introduction:

$$\frac{\Gamma, x : A \vdash t : B}{\Gamma \vdash \lambda x.t : A \rightarrow B}.$$

A function is a staged capability

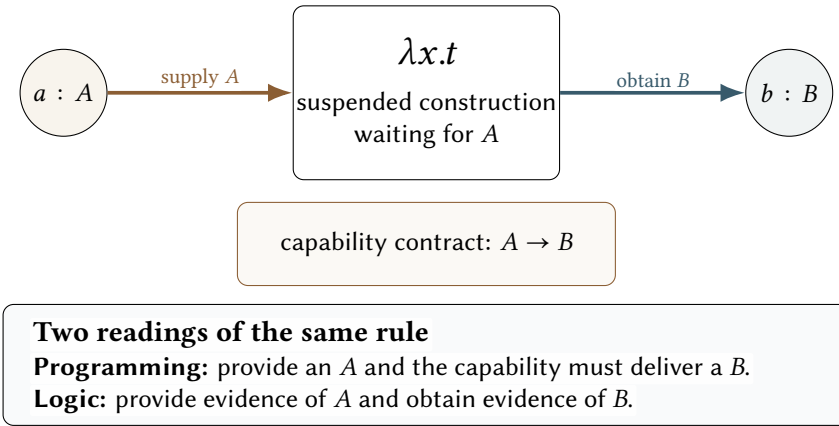
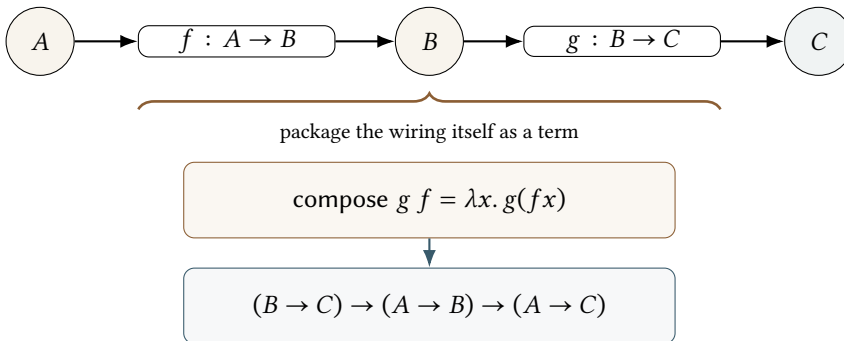


Figure 7.1: **Plate 17. Function as staged capability.** The diagram emphasizes the behavioral contract of $A \rightarrow B$. It does not imply eagerness, a particular calling convention, or that all computations are best modeled as total functions.

Higher-order composition: capabilities become data



The striking step is not composition itself; it is that f and g are ordinary inhabitants that may be accepted, returned, stored, and composed by another program.

Figure 7.2: **Plate 18. Higher-order composition.** The wiring diagram depicts extensional flow from A through B to C . It suppresses evaluation strategy and internal effects; two functions with the same arrow types need not have identical operational behavior.

Elimination:

$$\frac{\Gamma \vdash f : A \rightarrow B \quad \Gamma \vdash a : A}{\Gamma \vdash fa : B}.$$

Computation:

$$(\lambda x.t) a \longrightarrow t[a/x].$$

A common eta principle:

$$\lambda x.fx \equiv f \quad (x \notin \text{FV}(f)).$$

Theorem 7.1 (Typing is stable under the fundamental function computation). *If $\Gamma \vdash (\lambda x.t) a : B$, then*

$$(\lambda x.t) a \longrightarrow t[a/x]$$

and $\Gamma \vdash t[a/x] : B$.

Proof. The reduction statement is the beta rule. The typing statement is Corollary 5.2. The theorem is worth isolating because it shows the exact meeting point of static structure and dynamic computation: the rule that executes function application is justified by the structural theorem about substitution. $\square$

Worked Example 7.1: Composition is a three-stage hypothetical derivation

To construct

$$(A \rightarrow B) \rightarrow (B \rightarrow C) \rightarrow (A \rightarrow C),$$

assume $f : A \rightarrow B$, then $g : B \rightarrow C$, then $x : A$. Application gives $fx : B$, and a second application gives $g(fx) : C$. Discharging the assumptions in reverse order yields

$$\lambda f.\lambda g.\lambda x.g(fx).$$

The term is not guessed from the target type; it is the packaged derivation demanded by the target's nested function structure.

Computational Laboratory 7: A lambda evaluator

Construct the term

```
compose = Ann(
  Lam("f", Lam("g", Lam("x",
    App(Var("g"), App(Var("f"), Var("x"))))),
    Arrow(Arrow(A, B), Arrow(Arrow(B, C), Arrow(A, C)))
)
synth([], compose)
```

Use the checker to derive its type. Then apply it to concrete functions in the tiny language and watch beta reduction specialize one layer at a time. Compare

normal-order and call-by-value evaluation on a term where the argument contains reducible work.

7.8 Functions are not all of computation

This chapter makes functions look extraordinarily powerful because they are. Yet the series must resist a common overreach: not every computation is naturally modeled as a total map from an input value to an output value. Servers interact indefinitely; programs mutate state; randomized algorithms produce distributions; concurrent processes negotiate with one another.

Function types are the first major comprehension principle, not the final ontology of computation. Later volumes will widen the frame.

Exercises

Checkpoint.

1. Derive the type of $\lambda x.x$.
2. Reduce $(\lambda x.\text{succ}(x))\ 0$.
3. Explain function eta in words.

Core.

4. Find a term of type $A \rightarrow B \rightarrow A$.
5. Find a term of type $(A \rightarrow B) \rightarrow (B \rightarrow C) \rightarrow A \rightarrow C$.
6. Explain why the rules introduced so far provide no closed *constructor* for an arbitrary atomic type A . What additional metatheoretic fact is needed to conclude that no closed inhabitant exists at all?

Synthesis.

7. Read the type $(A \rightarrow B) \rightarrow A \rightarrow B$ as an English protocol.
8. Compare partial application with a conversation in which information arrives in stages.

Proof Workshop

- P1. Give a complete typing derivation for the composition term in Worked Example 7.1.

P2. Use Theorem 7.1 to show that applying the identity function to any well-typed argument preserves its type after one beta step.

Design Clinic

Compare two interfaces for lambdas: annotated binders versus bidirectional checking against an expected arrow type. Identify what information each interface asks the programmer to provide.

Challenge.

9. Derive a term witnessing $((A \rightarrow B) \rightarrow A) \rightarrow A$ if classical control operators are admitted. Explain why the pure intuitionistic lambda calculus does not give this inhabitant for arbitrary A, B .

Products and Sums: The Algebra of Information

8.1 Two ways to combine possibilities

Functions describe transformation. Products and sums describe two elementary ways information can be assembled.

A product $A \times B$ says: both pieces are present. A sum $A + B$ says: one of two alternatives is present, together with evidence of which one.

These two forms already generate a surprisingly rich algebra of data.

8.2 Products: information in parallel

If

$$\Gamma \vdash a : A \quad \text{and} \quad \Gamma \vdash b : B,$$

we may construct

$$\Gamma \vdash \langle a, b \rangle : A \times B.$$

The eliminators are projections:

$$\pi_1 p : A, \quad \pi_2 p : B.$$

Their computation rules are immediate:

$$\pi_1 \langle a, b \rangle \longrightarrow a, \quad \pi_2 \langle a, b \rangle \longrightarrow b.$$

A product is not merely “two values.” It is a contract that both projections are available.

8.3 Sums: information with a tag

A sum has two introduction forms:

$$\frac{\Gamma \vdash a : A}{\Gamma \vdash \text{inl}(a) : A + B}, \quad \frac{\Gamma \vdash b : B}{\Gamma \vdash \text{inr}(b) : A + B}.$$

A consumer of $A + B$ must be prepared for either case. Its eliminator is case analysis:

$$\text{case } s \text{ of } \text{inl}(x) \Rightarrow t \mid \text{inr}(y) \Rightarrow u.$$

This is tagged information. An inhabitant carries both a payload and evidence specifying which branch contains the payload.

8.4 Case analysis is exhaustive by type

The value of a sum type can arise only through one of its introductions. Therefore a complete eliminator has exactly the corresponding branches.

This is the type-theoretic root of exhaustive pattern matching. A compiler can warn about a missing case because the type describes the universe of canonical possibilities.

For a boolean type defined as $\mathbf{1} + \mathbf{1}$, exhaustive branching becomes ordinary `if`. For optional data, $A + \mathbf{1}$ represents either an A or absence. For error handling, $A + E$ represents success or a typed error.

8.5 Products and sums as logical connectives

Under Curry-Howard,

$$A \times B$$

corresponds to conjunction $A \wedge B$: evidence consists of evidence for both components.

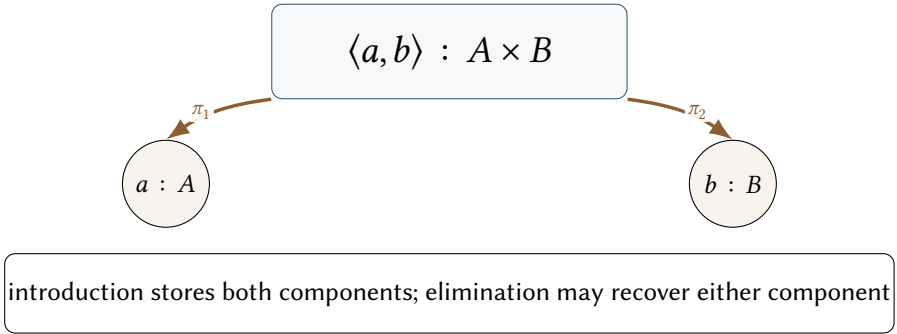
Similarly,

$$A + B$$

corresponds to disjunction $A \vee B$: evidence must identify which disjunct is supported and provide evidence for it.

This explains why a constructive proof of disjunction must tell us which side holds. Merely proving that assuming neither leads to contradiction may require classical principles not computationally informative enough to choose a branch.

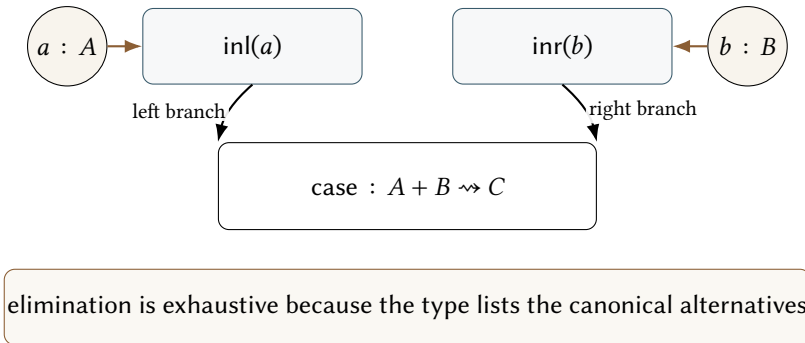
Product: both pieces are available



A product is stronger than “two possible values.” It promises simultaneous possession: every inhabitant supports both observations.

Figure 8.1: **Plate 19. Product.** The two projection arrows expose the observations guaranteed by $A \times B$. The picture suppresses sharing and evaluation cost: projecting a component need not imply that both components were eagerly computed.

Sum: one payload, with its branch tag



The tag is computational evidence. A consumer may not discard the distinction until it has supplied behavior for both constructors.

Figure 8.2: **Plate 20. Sum.** A sum contains exactly one branch’s payload together with a constructor tag. The diagram shows the canonical two-constructor case; larger variants are repeated sums or primitive variants with more constructors.

8.6 Data types as sums of products

Many algebraic data types can be decomposed into products and sums. A simple syntax tree might be

$$\text{Expr} = \mathbb{N} + (\text{Expr} \times \text{Expr}) + (\text{Expr} \times \text{Expr}).$$

The three alternatives might represent a numeral, addition, and multiplication. Recursive types require machinery we have not formalized yet, but the shape is already visible: constructors are alternatives, and constructor arguments are products of fields.

This is why the phrase **algebraic data type** is apt. Products behave like multiplication of possibilities; sums behave like addition.

If A has m finite inhabitants and B has n , then

$$|A \times B| = mn, \quad |A + B| = m + n.$$

The arithmetic analogy is exact for finite sets and remains structurally suggestive beyond them.

8.7 The distributive law becomes a program

The algebraic identity

$$A \times (B + C) \cong (A \times B) + (A \times C)$$

corresponds to explicit programs in both directions.

Given $(a, \text{inl}(b))$, produce $\text{inl}(a, b)$. Given $(a, \text{inr}(c))$, produce $\text{inr}(a, c)$. The inverse pattern matches on the outer sum and reconstructs the product.

An isomorphism of types is therefore witnessed by computations.

Formal Interlude VIII: Product and sum rules

Product introduction and eliminations:

$$\frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B}{\Gamma \vdash \langle a, b \rangle : A \times B},$$

$$\frac{\Gamma \vdash p : A \times B}{\Gamma \vdash \pi_1 p : A}, \quad \frac{\Gamma \vdash p : A \times B}{\Gamma \vdash \pi_2 p : B}.$$

Sum introductions:

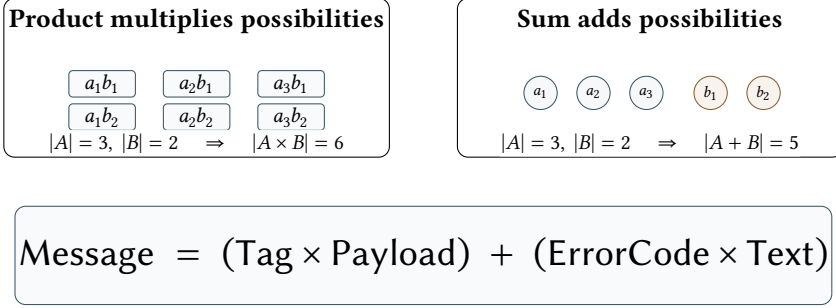
$$\frac{\Gamma \vdash a : A}{\Gamma \vdash \text{inl}(a) : A + B}, \quad \frac{\Gamma \vdash b : B}{\Gamma \vdash \text{inr}(b) : A + B}.$$

Sum elimination requires a branch for each constructor and a common result type C :

$$\frac{\Gamma \vdash s : A + B \quad \Gamma, x : A \vdash t : C \quad \Gamma, y : B \vdash u : C}{\Gamma \vdash \text{case}(s; x.t; y.u) : C}.$$

The beta laws select the corresponding branch.

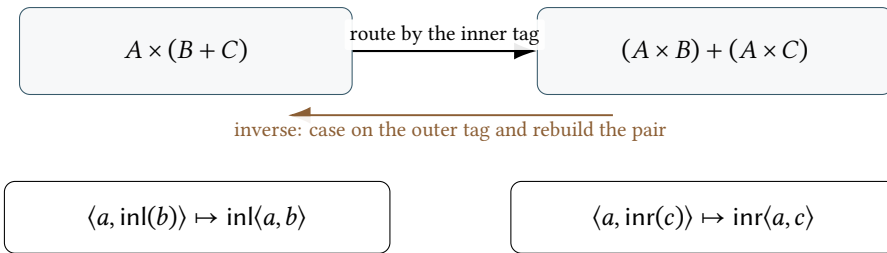
The algebra of data



Sums choose constructors; products collect fields. Nested together, they form the grammar of ordinary algebraic data types. For finite types the arithmetic is literal cardinality arithmetic.

Figure 8.3: **Plate 21. Algebra of data.** Products and sums behave like multiplication and addition of finite cardinalities, and the same constructors organize richer data shapes. The arithmetic analogy is exact for finite sets, but cardinality alone forgets typing, labels, recursion, and computational behavior.

Distributivity is executable rewiring



The isomorphism is not a symbolic identity imposed from outside. It is witnessed by programs whose composites normalize to the corresponding canonical inputs.

Figure 8.4: **Plate 22. Distributivity as program.** The two sides contain the same information organized differently. The arrows denote explicit total functions, not definitional equality: an implementation may store or evaluate the two representations differently.

Theorem 8.1 (Canonical forms for products and sums). *Let v be a closed value in the TinyTT core.*

- (a) *If $\vdash v : A \times B$, then $v = \langle v_1, v_2 \rangle$ with $\vdash v_1 : A$ and $\vdash v_2 : B$.*
- (b) *If $\vdash v : A + B$, then either $v = \text{inl}(v_1)$ with $\vdash v_1 : A$, or $v = \text{inr}(v_2)$ with $\vdash v_2 : B$.*

Proof. Inspect the value forms and invert their typing derivations. Lambdas, unit, booleans, and numerals cannot receive product or sum types under the core rules. The only value constructor that introduces a product is pairing; the only value constructors that introduce a sum are the two injections. Inverting the corresponding introduction rule yields the component typings. $\square$

Worked Example 8.1: Distributivity is executable rewiring

An inhabitant of

$$A \times (B + C) \rightarrow (A \times B) + (A \times C)$$

can be built by first projecting $a : A$ and the tagged alternative $s : B + C$. Case analysis on s preserves its tag:

$$\langle a, \text{inl}(b) \rangle \mapsto \text{inl}\langle a, b \rangle,$$

$$\langle a, \text{inr}(c) \rangle \mapsto \text{inr}\langle a, c \rangle.$$

The familiar algebraic identity is therefore witnessed by a concrete program.

Computational Laboratory 8: Exhaustive structure

Construct a sum term in `tinytt.py` and a case expression whose two branches both return naturals. Then intentionally make one branch return a function. The checker rejects the case because the entire elimination must have one result type. Next, encode booleans as a sum of units and implement `not` using case analysis.

8.8 Products and sums as communication shapes

A product says “I will send both.” A sum says “I will send one of these alternatives and tell you which.” These are already protocol fragments.

A response type

$$\text{Result} = \text{Payload} + \text{Error}$$

forces the receiver to acknowledge failure as part of the communication contract. A pair

$$\text{Token} \times \text{Expiry}$$

forces both pieces to travel together.

Types are becoming message grammars.

Exercises

Checkpoint.

1. Give the canonical introduction form of $A \times B$.
2. Why must case analysis for $A + B$ have two branches?
3. Encode a boolean using unit and sum.

Core.

4. Write programs witnessing $A \times B \cong B \times A$.
5. Write programs witnessing $A + B \cong B + A$.
6. Show that $A \times \mathbf{1} \cong A$ and $A + \mathbf{0} \cong A$.

Synthesis.

7. Design the type of a parser that may either return an AST or structured parse error information.
8. Explain how exhaustive pattern matching is a direct computational consequence of the introduction rules for sums.

Proof Workshop

- P1. Prove the product clause of Theorem 8.1 by enumerating every TinyTT value form and excluding the impossible cases by typing inversion.
- P2. Construct programs witnessing both directions of $A \times (B + C) \cong (A \times B) + (A \times C)$, then normalize both composites on canonical inputs.

Design Clinic

Encode a small JSON-like message schema as a sum of products. Explain which malformed payload shapes become unrepresentable after checking.

Challenge.

9. Give explicit functions in both directions for the distributive isomorphism $A \times (B + C) \cong (A \times B) + (A \times C)$ and reason informally that the composites reduce to identities.

Natural Numbers and Induction: Computation from Generation

9.1 A type can be specified by how its inhabitants are generated

Natural numbers have two canonical forms:

$$0 : \mathbb{N}$$

and

$$\text{succ}(n) : \mathbb{N} \quad \text{whenever } n : \mathbb{N}.$$

These constructors do more than list examples. They define the smallest class of canonical values closed under zero and successor.

This is an **inductive type**. Its elimination rule follows from the same generative description: to define something for every natural number, explain what happens at zero and how to extend the construction from n to $\text{succ}(n)$.

9.2 Recursion as the computational eliminator

To compute a value of some fixed type C from $n : \mathbb{N}$, provide

$$c_0 : C$$

and

$$s : C \rightarrow C.$$

Then define

$$\text{rec}_{\mathbb{N}}(n; c_0; s) : C.$$

The computation equations are

$$\text{rec}_{\mathbb{N}}(0; c_0; s) \longrightarrow c_0,$$

$$\text{rec}_{\mathbb{N}}(\text{succ}(n); c_0; s) \longrightarrow s(\text{rec}_{\mathbb{N}}(n; c_0; s)).$$

The eliminator follows the construction history of the input. Zero selects the base case; successor exposes one layer and recurses on the predecessor.

9.3 Addition emerges from the eliminator

Define addition by recursion on the first argument:

$$\text{add}(0, m) = m,$$

$$\text{add}(\text{succ}(n), m) = \text{succ}(\text{add}(n, m)).$$

This is not an extra magical feature. It is an instance of the natural-number elimination principle.

Multiplication, exponentiation, iteration, list-length computations, and finite loops all fit the same pattern.

9.4 Induction is dependent elimination

Recursion returns a result in one fixed type C . Mathematical induction is stronger because the claim may depend on the number:

$$n : \mathbb{N} \vdash P(n) \text{ type.}$$

To construct an inhabitant of $P(n)$ for every n , provide

$$p_0 : P(0)$$

and a step

$$n : \mathbb{N}, p : P(n) \vdash p_s : P(\text{succ}(n)).$$

Then the eliminator yields

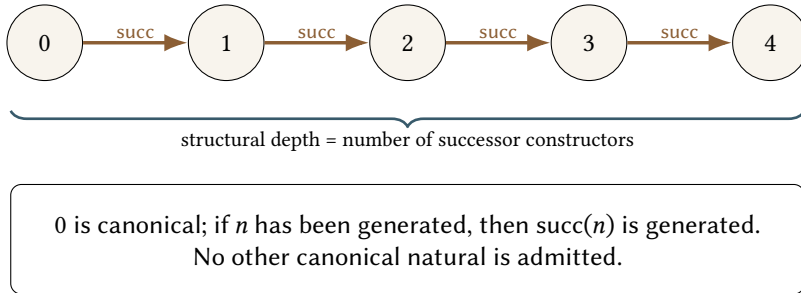
$$\text{ind}_{\mathbb{N}}(n; p_0; p_s) : P(n).$$

This is one of the points where “proof” and “program” become difficult to separate. The induction principle is a dependent program whose output type changes with its input.

9.5 Why structural recursion tends to terminate

Each recursive call in the natural-number recursor is made on a structurally smaller input. The computation follows the finite constructor tree until it reaches zero.

Natural numbers are generated, layer by layer



The eliminator mirrors this generation rule: one case for 0, one step for succ .
Generation and elimination are two faces of the same inductive structure.

Figure 9.1: **Plate 23. Natural numbers as generated structure.** The ladder depicts constructor depth rather than numerical distance in a metric space. The ellipsis beyond the shown numerals is implicit: the inductive rule permits arbitrarily many finite successor layers.

The recursor follows construction history

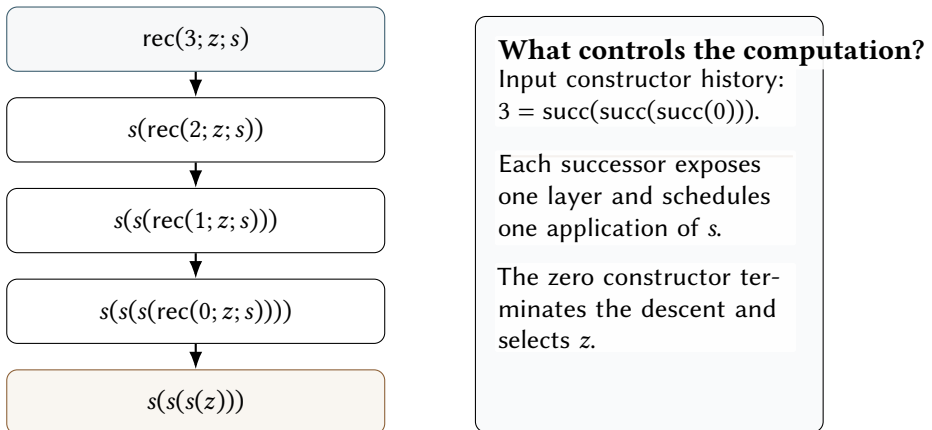


Figure 9.2: **Plate 24. Recursor follows construction history.** The reduction trace is for the numeral 3 and a unary step $s : C \rightarrow C$. It illustrates structural recursion, not every possible evaluation strategy or optimized implementation of primitive recursion.

This is the first glimpse of a tradeoff that will matter greatly later. A type theory can guarantee normalization by restricting recursion to structurally justified forms. A general-purpose language may permit arbitrary recursion and thereby gain expressive convenience at the cost of guaranteed termination.

The boundary is not merely theoretical. Total languages can make strong promises precisely because some programs are excluded.

9.6 Canonical forms and canonicity

A desirable property of a type theory is that every closed normal term of natural-number type is a numeral:

$$0, \text{ succ}(0), \text{ succ}(\text{succ}(0)), \dots$$

This is a **canonicity** statement. It connects typing and computation: a closed proof that “this is a natural number” ultimately computes to recognizably numerical data.

Canonicity is one reason constructive type theory feels computational. Proofs of existential or disjunctive statements can often normalize to explicit witnesses or branch choices.

Formal Interlude IX: Natural-number elimination

A nondependent recursor can be typed as

$$\frac{\Gamma \vdash n : \mathbb{N} \quad \Gamma \vdash c_0 : C \quad \Gamma \vdash s : C \rightarrow C}{\Gamma \vdash \text{rec}_{\mathbb{N}}(n; c_0; s) : C}.$$

A dependent eliminator has the more expressive schematic form

$$\frac{\Gamma, n : \mathbb{N} \vdash P(n) \text{ type} \quad \Gamma \vdash p_0 : P(0) \quad \Gamma, n : \mathbb{N}, p : P(n) \vdash p_s : P(\text{succ}(n))}{\Gamma, n : \mathbb{N} \vdash \text{ind}_{\mathbb{N}}(n; p_0; p_s) : P(n)}.$$

The type of the result is allowed to vary with the number being eliminated.

Theorem 9.1 (The recursor iterates its step along numeral structure). *Let $\bar{n} = \text{succ}^n(0)$, with $\Gamma \vdash z : C$ and $\Gamma \vdash s : C \rightarrow C$. In the congruence closure of TinyTT’s computation rules,*

$$\Gamma \vdash \text{natrec } \bar{n} \ z \ s \equiv s^n(z) : C,$$

where $s^0(z) = z$ and $s^{n+1}(z) = s(s^n(z))$.

Proof. By induction on n . For $n = 0$, the zero computation rule gives $\text{natrec } 0 \ z \ s \rightarrow z$, hence definitional equality with $s^0(z)$. For $n + 1$, the successor equation gives

$$\text{natrec } (\text{succ}(\bar{n})) \ z \ s \rightarrow s(\text{natrec } \bar{n} \ z \ s).$$

By the induction hypothesis, the recursive argument is definitionally equal to $s^n(z)$; congruence for application therefore yields equality with $s(s^n(z)) = s^{n+1}(z)$. Notice that this

is an equality statement, not a claim that call-by-value evaluation must pass through the literal intermediate syntax $s^n(z)$. $\square$

Worked Example 9.1: Addition follows construction history

Define addition by recursion on its first argument:

$$\text{add } n \ m = \text{rec } n \ m \ (\lambda a.\text{succ}(a)).$$

For $2 + 3$, the recursor peels two successor constructors from the first numeral, applying the step function once per layer:

$$\text{rec } 2 \ 3 \ s \longrightarrow s(\text{rec } 1 \ 3 \ s) \longrightarrow s(s(\text{rec } 0 \ 3 \ s)) \longrightarrow s(s(3)) = 5.$$

The control flow is obtained from the data's constructor history; no external loop counter is required.

Computational Laboratory 9: Build arithmetic from the recursor

Use `NatRec` in `tinytt.py` to define addition and multiplication. Do not add primitive evaluator cases for either operation. They must be ordinary terms assembled from the eliminator. Evaluate $2 + 3$ and 2×3 and inspect the reduction trace. Count how the number of reduction steps depends on which argument is chosen for recursion.

9.7 Induction as a comprehension principle

The natural numbers are not just a collection of values. Their constructors and eliminator jointly say what computations are possible *because* values were generated in a particular way.

This generalizes. Lists are generated by `nil` and `cons`, so their eliminator has empty and cons cases. Trees are generated by node constructors, so their eliminators mirror those constructors. Syntax itself can be represented inductively, making interpreters and compilers instances of elimination over syntax trees.

Data generation and computation are dual views of one structure.

Exercises

Checkpoint.

1. State the canonical constructors of $\mathbb{N}$.
2. State the two computation equations of the natural-number recursor.

3. Explain the difference between recursion and dependent induction.

Core.

4. Define predecessor using natural-number recursion, choosing a suitable state type if needed.
5. Define a boolean test for zero.
6. Explain why structural recursion on naturals terminates.

Synthesis.

7. Describe the constructors and eliminator shape for a binary tree type.
8. Explain how an AST interpreter is structurally similar to a proof by induction on syntax.

Proof Workshop

- P1. Expand the $n = 2$ case of Theorem 9.1 as a sequence of recursor equations and congruence/definitional-equality steps. Explain why, when the step and base are open terms, this need not be a literal call-by-value execution trace.
- P2. Define multiplication using the recursor and prove by induction on the first input that $\text{mul } \bar{n} \bar{m}$ reduces to the numeral for nm , assuming the corresponding theorem for addition.

Design Clinic

Add a predecessor function to TinyTT without adding a primitive predecessor constructor. Derive it from natural-number recursion and test it on zero and three.

Challenge.

9. Formulate a dependent predicate $P(n)$ expressing “ $n + 0 = n$ ” and describe the base and induction-step inhabitants that a full dependent type theory would require.

Induction transports evidence through a family

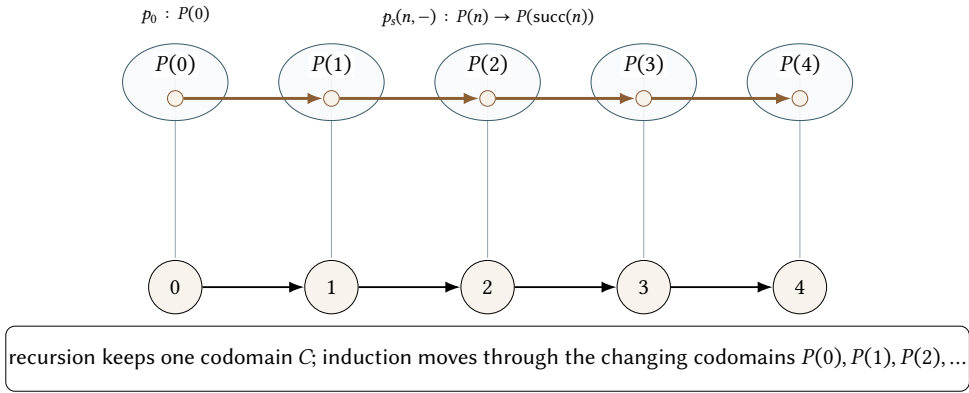


Figure 9.3: **Plate 25. Induction fibers.** The vertical fibres depict a type family $P(n)$ indexed by naturals, and the warm points and arrows depict evidence transported by the induction step between adjacent fibres. The geometry is schematic: no metric or literal spatial embedding of types is assumed.

Equality and Conversion: When Are Two Programs the Same?

10.1 One question, several equalities

The expression “these programs are equal” can mean radically different things.

Two programs may be character-for-character identical. They may differ only by names of bound variables. They may reduce to the same normal form. They may return the same outputs for all inputs. They may be related by a proof inside the language. They may have indistinguishable effects in every program context.

A type theory must decide which of these equalities the type checker is allowed to use automatically.

10.2 Syntactic identity is too strict

The terms

$$(\lambda x.x) 0 \quad \text{and} \quad 0$$

are visibly different syntax. Yet beta computation tells us that the first reduces to the second. If the type system treated them as entirely unrelated, type checking would become needlessly brittle.

Similarly,

$$\lambda x.x \quad \text{and} \quad \lambda y.y$$

should not be distinguished merely because of binder spelling.

Thus type theory ordinarily works modulo at least some notion of computational equality.

10.3 Definitional equality

We write

$$\Gamma \vdash t \equiv u : A$$

when the theory itself recognizes t and u as definitionally equal at type A .

Typical sources of definitional equality include alpha-renaming and computation rules such as beta reduction. Some theories also include eta laws or unfold named definitions automatically.

The essential feature is that definitional equality is not normally proved by constructing an inhabitant of an equality type. It is part of the machinery used to determine whether expressions fit together.

Intuition

Definitional equality is the equality the language is willing to see without being persuaded. Propositional equality, introduced later in the series, is equality for which evidence may have to be supplied.

10.4 Conversion

Suppose

$$\Gamma \vdash t : A$$

and the type checker knows

$$\Gamma \vdash A \equiv B \text{ type.}$$

Then it should accept

$$\Gamma \vdash t : B.$$

This is the conversion principle.

Without conversion, a function expecting a type written in one computational form could reject a value whose type normalizes to the same thing. With conversion, type checking becomes stable under the theory's own notion of computation.

10.5 Normalization as an equality procedure

A common strategy for deciding definitional equality is to normalize both sides and compare their canonical forms. If

$$t \longrightarrow^* n \quad \text{and} \quad u \longrightarrow^* n,$$

then t and u are candidates for definitional equality.

This works cleanly only when normalization terminates and normal forms are sufficiently unique. Those are metatheoretic properties, not free gifts.

10.6 Confluence

Suppose a term can reduce in two different ways:

$$t \longrightarrow u, \quad t \longrightarrow v.$$

A reduction system is confluent when the branches can ultimately be joined:

$$u \longrightarrow^* w \quad \text{and} \quad v \longrightarrow^* w.$$

Confluence says that local choices about which redex to evaluate need not alter the final normal form, when one exists. Plate 10.2 makes the claim concrete by enumerating the complete reduction graph of a term with two independent beta-redexes. It is one mathematical reason an equational theory based on reduction can be coherent.

10.7 Extensional equality is stronger

Two functions may produce equal outputs for every input while failing to be definitionally equal. For example, one may contain additional redexes, or the theory may deliberately keep intensional distinctions.

Function extensionality states, roughly, that if

$$\forall x : A, f(x) = g(x),$$

then

$$f = g.$$

Whether this principle is definitional, propositional, axiomatic, or derivable depends on the type theory.

This distinction matters computationally. Making an equality judgment more extensional can improve mathematical convenience while complicating decidable type checking.

10.8 Equality is a budget

Every equality the checker recognizes automatically consumes algorithmic budget. If definitional equality requires solving arbitrary mathematical theorems, type checking ceases to be a predictable compilation phase.

Language designers therefore negotiate a boundary:

$$\text{automatic computation} \quad | \quad \text{explicit proof obligation}.$$

Where that boundary is drawn shapes the entire programming experience.

Formal Interlude X: Equivalence laws for definitional equality

Definitional equality is normally at least an equivalence relation:

$$t \equiv t, \quad t \equiv u \Rightarrow u \equiv t,$$

$$t \equiv u \wedge u \equiv v \Rightarrow t \equiv v.$$

It is also expected to be a congruence: equal subterms may be replaced in larger well-formed expressions. Computation laws such as beta reduction generate additional equalities.

A conversion rule can be presented schematically as

$$\frac{\Gamma \vdash t : A \quad \Gamma \vdash A \equiv B \text{ type}}{\Gamma \vdash t : B}.$$

Theorem 10.1 (Normalization decides reduction-generated definitional equality under standard hypotheses). *Assume the typed reduction relation is strongly normalizing and confluent, alpha-equivalence of normal forms is decidable, and definitional equality is the congruence/equivalence closure generated by that reduction relation and alpha-equivalence. Then definitional equality is decidable by normalizing both terms and comparing their normal forms up to alpha-equivalence.*

Proof. Strong normalization guarantees that normalization terminates. Confluence guarantees uniqueness of normal form up to the chosen harmless presentation equivalences, here alpha-renaming. If t and u are connected by the symmetric-transitive closure of reduction, confluence gives the same normal form. Conversely, if they normalize to alpha-equivalent normal forms, each is definitionally equal to that common form and hence to the other. The final comparison is decidable by hypothesis. $\square$

Remark 10.2. The theorem is conditional on metatheory, not a license to define equality by running arbitrary programs. The point of normalization guarantees is precisely that they make this decision procedure safe.

Worked Example 10.1: Different text, one computational result

The terms

$$(\lambda x.x) 0 \quad \text{and} \quad 0$$

are neither textually identical nor alpha-equivalent. Yet the first beta-reduces to the second. In a theory whose definitional equality contains beta computation, the checker may recognize

$$(\lambda x.x) 0 \equiv 0 : \mathbb{N}.$$

This is stronger than comparing syntax and weaker than asserting every mathematically provable equality automatically.

Computational Laboratory 10: Equality by normalization

Use `normalize(term)` and `defeq(t,u)`. Test pairs that differ by beta reduction and alpha-renaming. Then add an eta expansion and observe whether the current checker identifies it automatically. Modify `defeq` to include eta normalization for functions and note how the equality algorithm becomes more involved.

Exercises

Checkpoint.

1. Give three distinct meanings of “program equality.”
2. Why should $(\lambda x.x) 0$ and 0 normally be definitionally equal?
3. State the conversion principle in words.

Core.

4. Draw a confluence diamond for a term containing two independent beta redexes.
5. Give two extensionally equal functions that might not be syntactically identical.
6. Explain why unrestricted theorem proving inside definitional equality threatens decidable type checking.

Synthesis.

7. Compare definitional equality with compiler constant folding.
8. Explain why equality policy is part of language ergonomics, not merely foundations.

Proof Workshop

- P1. Identify exactly where confluence and strong normalization are used in Theorem 10.1. Give a failure mode if each assumption is removed.
- P2. Show that alpha-equivalence is an equivalence relation on the named lambda terms used in TinyTT.

Design Clinic

Design an “equality budget” for a language: list three equalities you would make automatic and two you would require explicit proof for. Justify the checker-cost tradeoff.

Challenge.

9. Investigate normalization-by-evaluation at a conceptual level. Explain why evaluating into a semantic domain and reifying back to syntax can decide equality without performing naive source-level rewriting.

One word, several equality judgments

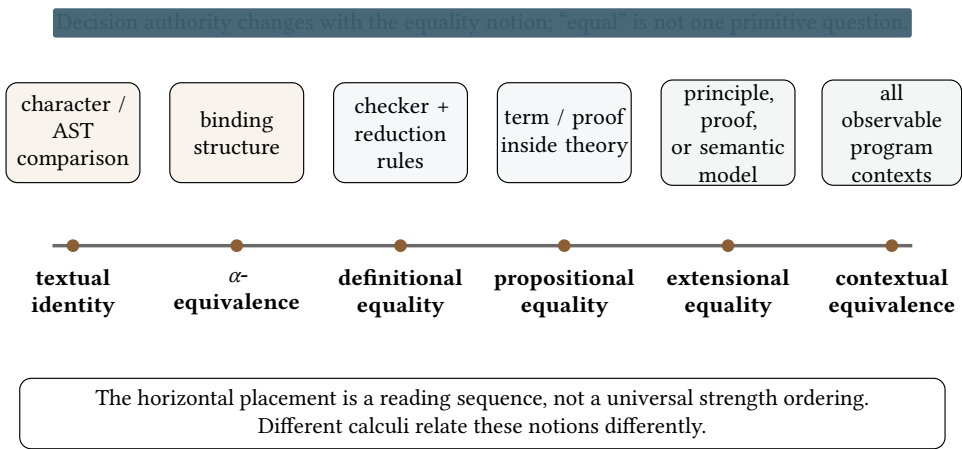


Figure 10.1: **Plate 26. Equality spectrum.** The same phrase—“these programs are equal”—can invoke syntax, binding structure, the checker’s conversion algorithm, internal proof, an extensional principle, or contextual observation. The sequence is deliberately not drawn as implication arrows: there is no theory-independent total ordering of these notions.

Confluence is a claim about an actual reduction graph

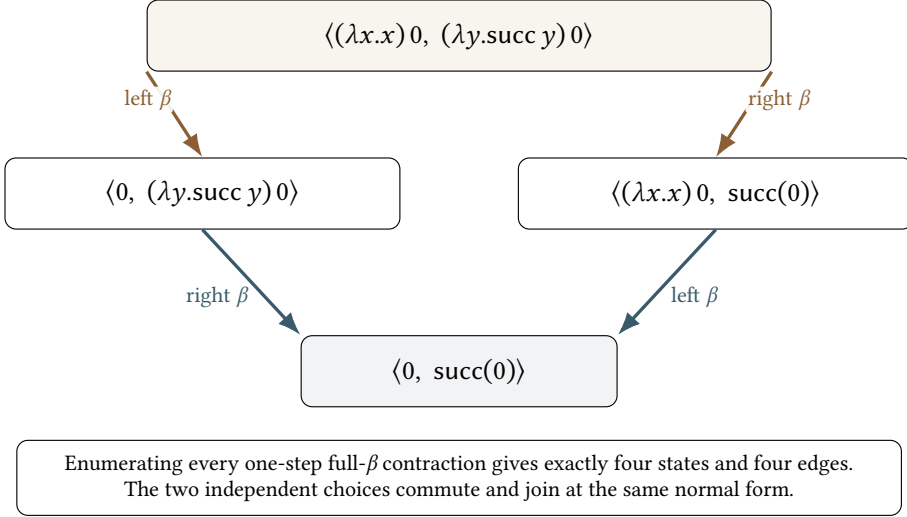


Figure 10.2: **Plate 27. A computed confluence diamond.** This is the complete full- β reduction graph of the displayed product term, not a generic diamond icon. Reducing either independent redex first produces a distinct intermediate term, and the remaining contraction joins both paths at $\langle 0, \text{succ}(0) \rangle$. The machine-readable edge list is shipped with the laboratory sources.

Equality has an automation boundary

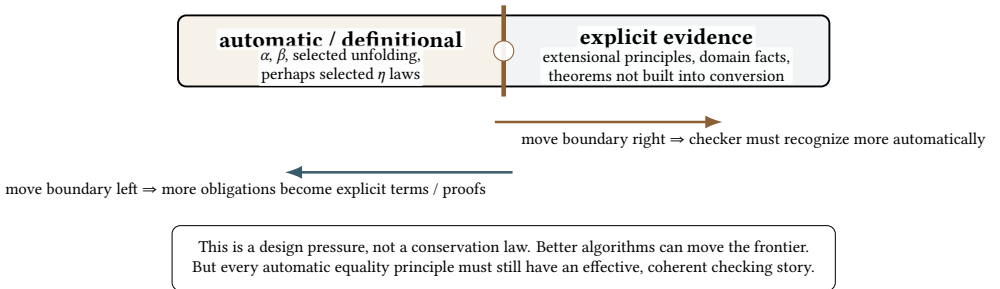


Figure 10.3: **Plate 28. Equality budget.** A language designer chooses how much equality the checker recognizes without explicit evidence. Moving the boundary can trade proof burden against conversion complexity, but the picture is not a claim of a strict zero-sum law: algorithmic advances and restricted principles can change the feasible frontier.

Normalization: Computation as the Exposure of Canonical Form

11.1 What does evaluation reveal?

Execution is often described as changing machine state. Type theory offers another lens: computation exposes the canonical information already implicit in a construction.

The term

$$(\lambda x. \text{succ}(x))\ 0$$

contains the answer $\text{succ}(0)$ in a noncanonical arrangement. Beta reduction removes the administrative layer. Evaluation is a process of making computational content explicit.

This view is especially natural for pure terms. Later volumes will examine where it must be broadened for interaction and effects.

11.2 Values, normal forms, canonical forms

These terms are related but should not be casually identified.

A **value** is an expression considered finished by a chosen operational semantics. A **normal form** contains no applicable reduction rule. A **canonical form** is a recognizable introduction form appropriate to its type.

For a well-designed strongly normalizing calculus, these notions line up in powerful ways. A closed normal term of function type is a lambda. A closed normal term of product type is a pair. A closed normal natural number is a numeral.

Such statements are called canonical-forms lemmas and are essential ingredients in progress proofs.

11.3 Evaluation strategy

Consider

$$(\lambda x.0) ((\lambda y.y) 0).$$

A call-by-value evaluator first reduces the argument, then applies the outer function. A call-by-name evaluator substitutes the unreduced argument immediately; because x is unused, the inner computation never occurs.

Both strategies reach 0 in this example. Their cost and behavior differ.

Type theory therefore does not uniquely determine an operational semantics. The typing rules constrain meaningful programs, while the evaluator determines a particular execution discipline among them.

11.4 Strong normalization

A calculus is **strongly normalizing** if every reduction sequence from every well-typed term eventually terminates. Classical reference points for normalization and typed lambda calculi include Barendregt and Girard–Lafont–Taylor [1, 6].

Remark 11.1 (Scope of the normalization claim). Volume I does not supply a full logical-relations or reducibility proof of strong normalization for the entire TinyTT core. The normalization-based decision theorem below therefore states normalization as an explicit hypothesis. The progress and preservation arguments do not depend on that omitted normalization proof.

This is far stronger than saying a particular evaluator terminates. Strong normalization says there is no infinite sequence of legal reductions at all.

The simply typed lambda calculus enjoys strong normalization. The untyped lambda calculus does not: the familiar term

$$\Omega = (\lambda x.xx)(\lambda x.xx)$$

reduces to itself forever.

Why can we not assign Ω an ordinary simple type? Typing xx would require x simultaneously to be an argument of itself and a function accepting its own type, producing a recursive type equation that simple types do not admit.

The exclusion of certain self-applications is one source of normalization. Plate 11.2 records the exact one-step reduction graph of Ω : contracting its root redex returns the same term, so the graph has a literal self-loop.

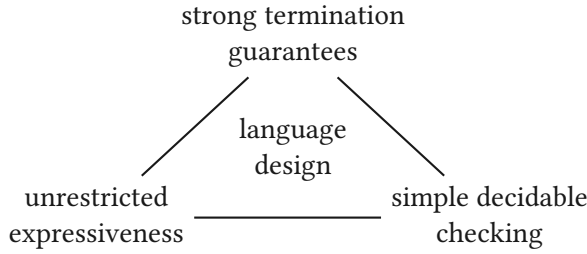
Proposition 11.2 (The usual Ω is not simply typable). *Let $\delta = \lambda x.xx$ and $\Omega = \delta\delta$. In the ordinary simply typed lambda calculus with finite types generated from base types and $\rightarrow$, there is no type T such that $\vdash \Omega : T$.*

Proof. It suffices to show that δ has no simple type. Suppose $\delta : S \rightarrow T$. By inversion of lambda typing, under $x : S$ the body xx has type T . Inversion of application typing then requires some U such that the function occurrence of x has type $U \rightarrow T$ while the argument occurrence has type U . The context gives both occurrences the single type S , hence $S = U$ and $S = U \rightarrow T$. Thus $S = S \rightarrow T$. But finite simple types have a strictly increasing size under arrow formation: $|S \rightarrow T| = 1 + |S| + |T| > |S|$. Contradiction. $\square$

11.5 Termination is an expressive constraint

Strong normalization is valuable because it yields decidable normalization and supports logical consistency. But it comes at a price: arbitrary general recursion cannot be admitted in the same unrestricted way as in a Turing-complete programming language.

This creates a recurring design triangle:



There is no universal law saying only two vertices are possible, but tensions are real. Sophisticated languages use guarded recursion, sized types, termination checkers, partiality monads, or effect systems to negotiate them.

11.6 Progress

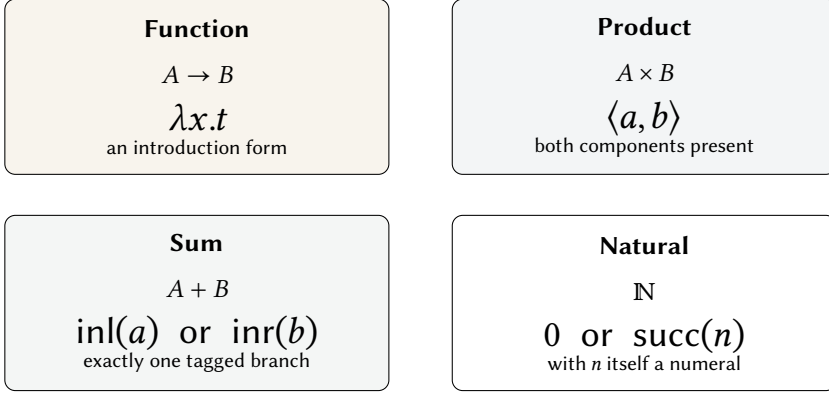
For a simple closed language, progress says:

If $\vdash t : T$, then either t is a value or there exists t' such that $t \longrightarrow t'$.

The theorem rules out stuck states such as attempting to apply a numeral as a function or projecting from a boolean. Those malformed configurations cannot be reached by closed well-typed terms.

Progress is proved by induction on typing derivations together with canonical-forms lemmas.

A type constrains the final shape of a closed normal term



Canonical-forms lemmas turn typing information into structural information about values.

Figure 11.1: **Plate 29. Canonical forms.** In the closed pure fragment taught here, a normal inhabitant's type sharply restricts its possible introduction form. This gallery is type-directed, not merely syntactic. Richer calculi can have additional canonical forms, so the lemma must always be stated for a particular language.

The normalization boundary: a loop that simple types refuse to admit

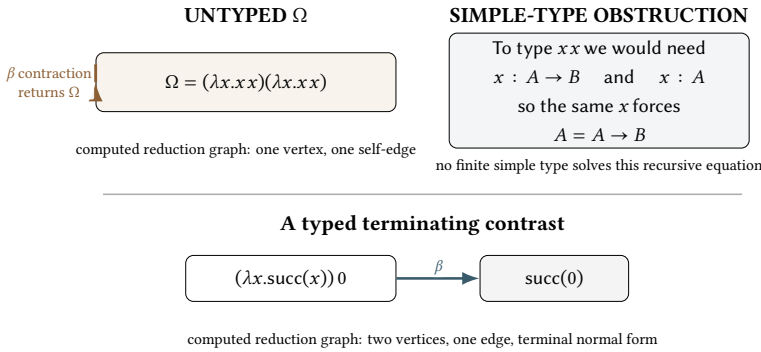


Figure 11.2: **Plate 30. Divergence boundary.** Contracting the root beta-redex of Ω reproduces exactly the same syntax, yielding a genuine self-loop in the reduction graph. In the ordinary simply typed lambda calculus, typing the subterm xx would force a recursive type equation $A = A \rightarrow B$, which finite simple types do not admit. Recursive or richer type systems change this particular argument; the plate illustrates one mechanism behind strong normalization of the simple calculus, not a universal prohibition on self-reference.

11.7 Preservation

Preservation says:

If $\Gamma \vdash t : T$ and $t \longrightarrow t'$, then $\Gamma \vdash t' : T$.

The function beta case relies on substitution. Product projections rely on inversion of the pair typing rule. Case reduction relies on the branch typing assumptions.

The proof is a map of the language's internal engineering. If preservation is unexpectedly hard, a typing or reduction rule may have been designed poorly.

Formal Interlude XI: Type safety as progress plus preservation

For the closed pure calculus in this volume:

$$\vdash t : T \implies t \text{ is a value or } t \rightarrow t' \text{ for some } t'$$

and

$$\vdash t : T \wedge t \rightarrow t' \implies \vdash t' : T.$$

Therefore evaluation cannot move a well-typed closed term into an ill-typed state, and a non-value well-typed closed term is not stuck.

This theorem does *not* say the program satisfies its user's intent, avoids logical bugs, is secure, terminates in every richer language, or avoids resource exhaustion. Type safety is precise and limited.

Theorem 11.3 (Canonical forms for closed TinyTT values). *Let v be a closed TinyTT value.*

- (a) *If $\vdash v : A \rightarrow B$, then $v = \lambda x.t$.*
- (b) *If $\vdash v : A \times B$, then $v = \langle v_1, v_2 \rangle$.*
- (c) *If $\vdash v : A + B$, then $v = \text{inl}(v_1)$ or $v = \text{inr}(v_2)$.*
- (d) *If $\vdash v : \mathbf{Bool}$, then $v = \text{true}$ or $v = \text{false}$.*
- (e) *If $\vdash v : \mathbb{N}$, then $v = \text{succ}^n(0)$ for some $n \geq 0$.*
- (f) *If $\vdash v : \mathbf{1}$, then $v = \star$. There is no closed value of type $\mathbf{0}$.*

Proof. Inspect TinyTT's value grammar and invert the final typing rule for each candidate value. Each outer type constructor has exactly the listed compatible value constructors. For naturals, induction on the value grammar shows that a natural value is zero or a finite successor chain. No value constructor has conclusion type $\mathbf{0}$. $\square$

Theorem 11.4 (Preservation for the TinyTT core). *If $\Gamma \vdash t : T$ and $t \longrightarrow t'$, then $\Gamma \vdash t' : T$.*

Proof sketch. Induct on the derivation of the reduction step. Congruence cases follow from the typing derivation and the induction hypothesis. The computational cases use inversion plus the relevant structural fact: beta reduction uses substitution (Theorem 5.1); product projection uses inversion of pair typing; sum case reduction uses substitution into the selected branch; boolean conditionals select a branch already checked at the common result type; and natural recursion uses the typing of the zero and step cases. $\square$

Remark 11.5 (Declarative preservation versus algorithmic re-checking). The theorem is about the declarative judgment. TinyTT’s evaluator may erase an annotation that the bidirectional surface checker originally used to synthesize a type. Consequently, an intermediate erased term can remain declaratively well typed while no longer being re-synthesizable by the surface algorithm without restoring an expected type or elaboration information. Release-candidate regression tests record these two properties separately; they must not be conflated.

Theorem 11.6 (Progress for closed TinyTT terms). *If $\vdash t : T$, then either t is a value or there exists t' such that $t \longrightarrow t'$.*

Proof sketch. Induct on the typing derivation. For constructor terms whose subterms are evaluated by call-by-value, apply the induction hypothesis to the next required subterm: if it steps, lift that step through the constructor; if all required components are values, the constructor itself is a value. For an eliminator, apply the induction hypothesis to its principal subterm. If that subterm steps, the eliminator takes the corresponding congruence step. If it is already a value, Theorem 11.3 identifies the only value constructors compatible with its type, so the matching computation rule applies. The empty eliminator’s value case is impossible because there is no closed value of type $\mathbf{0}$. Thus every closed well-typed non-value term has an immediate step; the proof never assumes normalization in advance. $\square$

Corollary 11.7 (No stuck closed well-typed states). *A closed well-typed TinyTT term cannot reach a non-value state with no reduction rule.*

Proof. Repeatedly apply preservation along the reduction sequence. At every reachable closed term, progress applies. $\square$

11.8 Normalization and proof

Under Curry-Howard, normalization of programs corresponds to simplification of proofs. A proof containing a detour – introduce an implication and immediately eliminate it – can be normalized by removing the detour. Beta reduction is proof normalization made executable.

This historical and structural connection is one reason type theory repeatedly joins logic and programming language theory. The same reduction relation can be read as executing code or simplifying evidence.

Worked Example 11.1: A typed step cannot change the promised kind

Let

$$t = \pi_1\langle(\lambda x.x) \ 0, \star\rangle.$$

The pair has type $\mathbb{N} \times \mathbf{1}$, so $t : \mathbb{N}$. Call-by-value first reduces the left component, giving $\pi_1\langle 0, \star\rangle$, and projection then gives 0. Every intermediate term still has type $\mathbb{N}$. This tiny trace is preservation made visible one step at a time.

Computational Laboratory 11: Reduction traces

Run the evaluator with tracing enabled on a nested composition term. Compare call-by-value and normal-order traces. For each reduction, identify the introduction form and eliminator that meet. Then attempt to represent Ω in the typed AST. The type checker should reject it before evaluation begins.

11.9 Normalization is not the whole of computation

An operating system is not usefully understood as a term whose purpose is to reach normal form. Neither is a web server, a control system, or an interactive agent. This does not invalidate normalization; it marks its domain.

For closed pure computation, normalization is an extraordinarily powerful model. For open interactive computation, we will need traces, protocols, coalgebraic structure, effects, and observations over time. A grand theory must contain both.

Exercises

Checkpoint.

1. Distinguish value, normal form, and canonical form.
2. State progress and preservation.
3. Give the untyped diverging term Ω and explain its behavior.

Core.

4. Give a term on which call-by-value performs work that call-by-name avoids.
5. Explain why preservation's beta case requires substitution.
6. State a canonical-forms lemma for products.

Synthesis.

7. Explain why a server may be correct precisely because it does not terminate.
8. Distinguish type safety from memory safety, total correctness, and semantic correctness.

Proof Workshop

- P1. Complete one congruence case and one computational case in the proof of preservation, showing the typing derivation before and after the step.
- P2. Prove progress for closed applications from the induction hypotheses and the canonical-form fact for arrow values.

Design Clinic

Modify the evaluator to return a structured reason when `step` returns `None`: value, impossible stuck state, or unsupported form. Explain how progress predicts which category closed well-typed terms can occupy.

Challenge.

9. Sketch the progress proof for application: if tu is closed and well typed, analyze the possible states of t and u until a reduction rule or value case is forced.

Type safety is two invariants working together

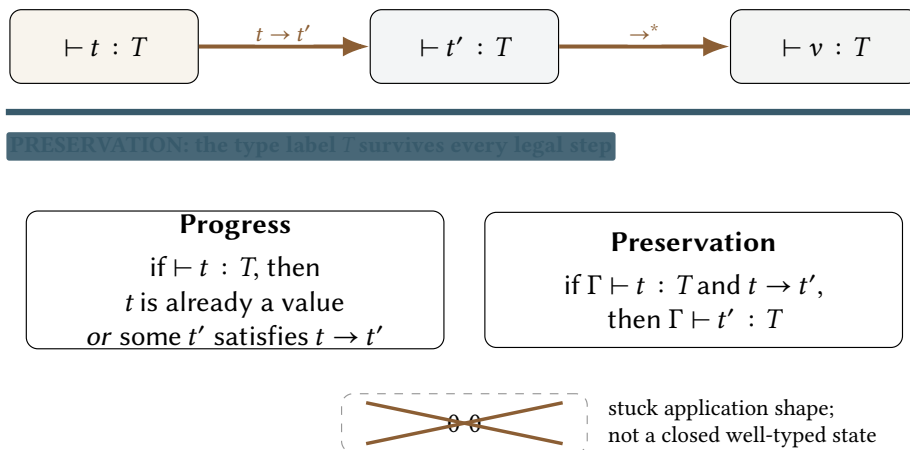


Figure 11.3: **Plate 31. Progress and preservation.** Progress excludes closed well-typed non-values with no legal move; preservation keeps the type invariant along every reduction step. Together they rule out stuck states for this pure core. They do not imply termination, correctness with respect to user intent, security, or resource bounds.

Proof and Program: The Curry-Howard Lens

Historical note: why “Curry–Howard”?

Curry’s 1934 work on functionality in combinatory logic supplied an early type-assignment/proof-theoretic bridge [4]. Howard’s notes, circulated in 1969 and published in 1980, made the formulae-as-types correspondence explicit for natural deduction [8]. The logical side of the presentation descends from Gentzen’s natural-deduction calculi [5]. The modern phrase names a lineage rather than a single simultaneous discovery.

12.1 A correspondence that changes both sides

The Curry-Howard correspondence is often summarized by the table

propositions $\leftrightarrow$ types, proofs $\leftrightarrow$ programs.

The table is useful but too static. The deeper correspondence matches *rules of reasoning* with *rules of construction* and *proof normalization* with *program evaluation*.

Implication is not merely labeled by a function type. Its introduction rule is lambda abstraction; its elimination rule is application; the logical detour elimination is beta reduction.

12.2 Conjunction as product

To prove $A \wedge B$, provide evidence for both A and B . That is exactly product introduction:

$$a : A, \quad b : B \quad \rightsquigarrow \quad \langle a, b \rangle : A \times B.$$

Curry–Howard lives in the rules, not in a slogan

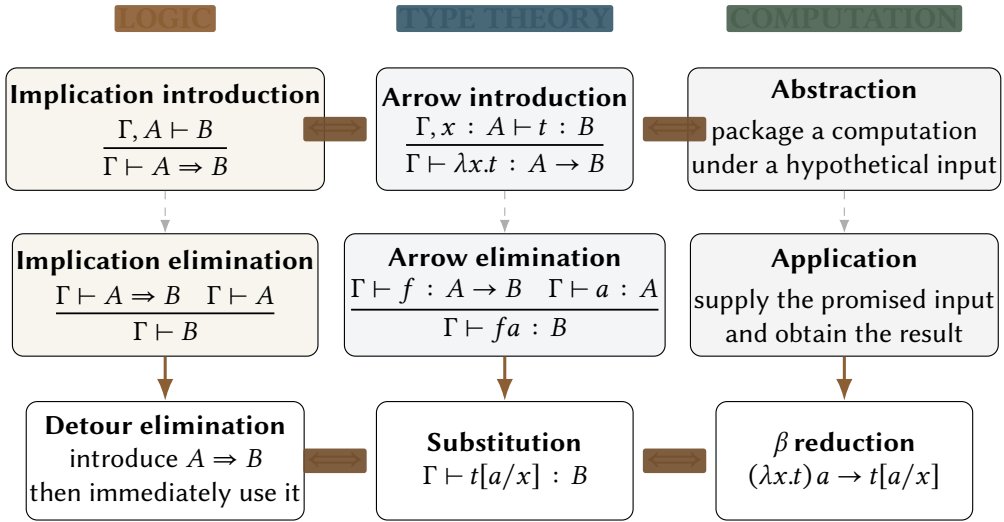


Figure 12.1: **Plate 32. Curry–Howard at rule level.** Implication introduction, arrow introduction, and lambda abstraction are three readings of the same structural move; implication elimination, arrow elimination, and application align likewise. The final row is the decisive dynamic correspondence: proof detour removal is substitution, and on terms that substitution is exposed by beta reduction. The plate concerns the constructive fragment developed here; richer logics and effects require additional structure.

To use evidence of $A \wedge B$, extract either component. That is projection.

The logical connective is therefore characterized by the same information-flow rules as the data structure.

12.3 Disjunction as sum

To prove $A \vee B$ constructively, we must know which side holds and have evidence for that side. That is a tagged sum.

To reason from $A \vee B$, provide a continuation for the A case and one for the B case. That is exhaustive case analysis.

Constructive disjunction is computationally informative because normalized evidence reveals the chosen branch.

12.4 Falsehood as empty type

A proof of falsehood would be an inhabitant of $\mathbf{0}$, but there is no constructor for such an inhabitant. From a hypothetical inhabitant of $\mathbf{0}$, any conclusion may be derived using empty elimination.

Negation becomes

$$\neg A \stackrel{\text{def}}{=} A \rightarrow \mathbf{0}.$$

To prove $\neg A$, write a function that turns any alleged evidence of A into impossibility.

12.5 Universal and existential quantification foreshadow dependency

The simple function type $A \rightarrow B$ corresponds to implication. A universal statement

$$\forall x : A. P(x)$$

requires a stronger type because the result proposition depends on the particular x :

$$\Pi_{x:A} P(x).$$

An existential statement

$$\exists x : A. P(x)$$

requires a witness $a : A$ paired with evidence of $P(a)$:

$$\Sigma_{x:A} P(x).$$

This is the conceptual bridge to Volume II. Quantification becomes computational dependency.

12.6 Proof relevance

In ordinary mathematics one often cares only that a proposition is true. In type theory, different inhabitants can represent different evidence. The distinction may matter computationally.

A proof of an existential statement can contain a witness. A proof of a disjunction can contain a branch choice. A proof of a specification can contain an algorithm certified to satisfy it.

This does not mean every proof should carry runtime data. Proof irrelevance, erasure, and extraction policies allow implementations to separate computationally significant content from certificates used only during checking.

12.7 Programs from proofs, proofs from programs

Suppose the type

$$(A \rightarrow B) \rightarrow A \rightarrow B$$

is presented as a proposition. Constructing a proof amounts to writing

$$\lambda f.\lambda a.f a.$$

Conversely, a program with that type is already a proof term for the corresponding intuitionistic implication.

This creates a practical methodology: write a specification as a type, then search for an inhabitant. Proof assistants mechanize precisely this activity at much greater expressive depth.

12.8 Inhabitation as proof search

A type can therefore be used operationally as a search objective. For an arrow goal, backward search may introduce a hypothetical argument. For a product goal, it may split the task into two component goals. For an atomic goal, it may inspect the context for a matching assumption or for a function whose codomain is the required type and whose arguments can themselves be constructed.

This perspective turns theorem proving into program synthesis in a literal sense: the search state is a typing judgment with a hole, and a successful search returns a term that fills the hole. But two distinctions matter. First, unrestricted inhabitation is not uniformly easy across type theories; complexity and decidability depend on the fragment. Second, search and verification need not have the same trust status. A heuristic search procedure can propose a candidate that a smaller checker independently verifies.

A type can be treated as a construction search problem

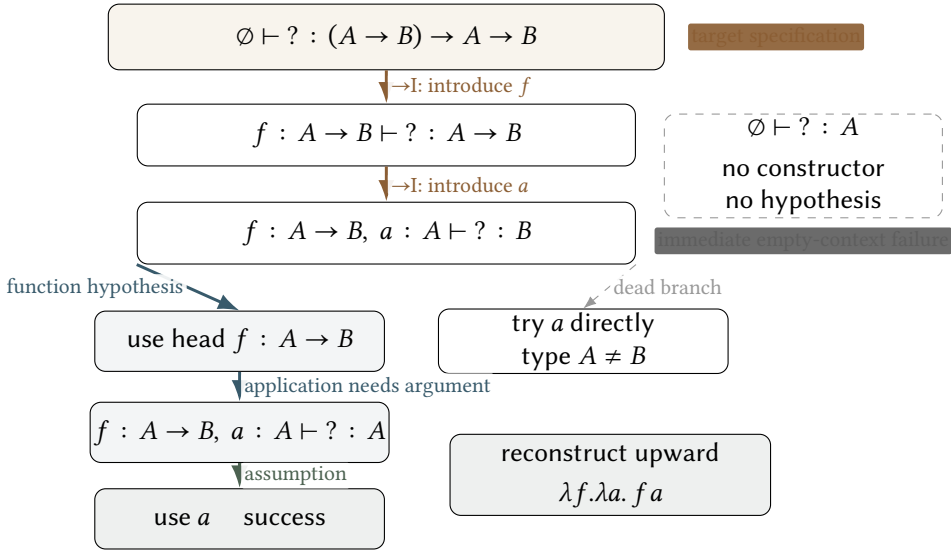


Figure 12.2: **Plate 33. Inhabitation as bounded search.** Backward proof search decomposes the target type into obligations, consults the context as an inventory of usable hypotheses, and reconstructs a term from a successful branch. The shipped search program finds $\lambda f. \lambda a. fa$ for the displayed target and rechecks the result with TinyTT. Failure of a bounded search is not, in general, a proof of uninhabitability; the atomic empty-context example is included because its failure is immediate from the fragment’s constructors.

Worked Example 12.2: Backward search reconstructs application

Consider the target

$$\emptyset \vdash ? : (A \rightarrow B) \rightarrow A \rightarrow B.$$

Arrow introduction first makes $f : A \rightarrow B$ available, then makes $a : A$ available. The remaining goal is B . The context contains no direct B , but it contains $f : A \rightarrow B$. Using f creates a subgoal A , which is discharged by a . Reconstructing the introductions upward yields

$$\lambda f. \lambda a. fa.$$

The companion program `inhabitation_search.py` performs this bounded search and then submits every reported solution to TinyTT. The search procedure proposes; the kernel-style checker decides whether the proposal actually inhabits the target.

12.9 Where Curry-Howard is not enough

The correspondence does not imply that every useful program is a proof in a strongly normalizing constructive logic without further machinery. General recursion, exceptions, state, nondeterminism, probability, and concurrency require enriched logical or type-theoretic structures.

Nor does it imply that ordinary mathematical propositions should always be identified with runtime data. The correspondence is structural, and its engineering realization depends on the theory.

Formal Interlude XII: The constructive fragment

The fragment developed so far supports the correspondences

logic	types
$A \Rightarrow B$	$A \rightarrow B$
$A \wedge B$	$A \times B$
$A \vee B$	$A + B$
$\top$	1
$\perp$	0

with introduction and elimination rules aligned on both sides. Full first-order quantifiers motivate dependent products and sums.

Proposition 12.1 (Implication detour elimination is beta reduction). *Under the Curry–Howard reading, introducing an implication and immediately eliminating it corresponds exactly to a beta redex:*

$$\frac{\frac{\Gamma, x : A \vdash t : B}{\Gamma \vdash \lambda x. t : A \rightarrow B} \quad \Gamma \vdash a : A}{\Gamma \vdash (\lambda x. t) a : B} \rightsquigarrow \Gamma \vdash t[a/x] : B.$$

Proof. The proof-theoretic detour first packages a derivation that assumes A , then immediately supplies evidence $a : A$. Eliminating the detour therefore substitutes the evidence for the discharged assumption. On terms this is precisely $(\lambda x. t) a \rightarrow t[a/x]$. The typing of the reduced proof term is guaranteed by substitution. $\square$

Worked Example 12.1: Commuting conjunction is swapping data

The constructive theorem

$$A \wedge B \Rightarrow B \wedge A$$

corresponds to the type

$$A \times B \rightarrow B \times A.$$

Given $p : A \times B$, projections provide $\pi_1 p : A$ and $\pi_2 p : B$. Pair introduction then yields

$$\lambda p. \langle \pi_2 p, \pi_1 p \rangle : A \times B \rightarrow B \times A.$$

The proof and the data-transforming program are the same construction viewed through two readings of the rules.

Computational Laboratory 12: Inhabit the proposition

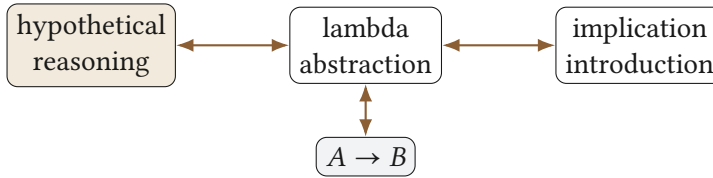
Use the term constructors only – no primitive constants – to inhabit each type if possible:

```
A -> A
A -> B -> A
(A -> B) -> (B -> C) -> A -> C
A * B -> B * A
A + B -> B + A
```

Ask the checker to verify each term. Then try to inhabit **A** in an empty context. The failure is the computational face of the fact that arbitrary propositions are not provable. Finally run `inhabitation_search.py`: inspect the bounded search trace for $(A \rightarrow B) \rightarrow A \rightarrow B$, then compare its candidate with the judgment independently accepted by TinyTT.

12.10 The first unification

At this point, several ideas introduced independently have collapsed onto one structure:



This is not yet the grand unification promised by the series. It is the first hard evidence that apparently different practices – proving, programming, and reasoning under assumptions – may share a common grammar.

Exercises

Checkpoint.

1. Match implication, conjunction, disjunction, truth, and falsehood to their type constructors.
2. Why is constructive evidence for a disjunction computationally informative?
3. Define negation using the empty type.

Core.

4. Find inhabitants of $(A \times B) \rightarrow A$ and $(A \times B) \rightarrow B$.
5. Find an inhabitant of $A \rightarrow (B \rightarrow A \times B)$.
6. Explain why an existential proposition naturally corresponds to a dependent pair rather than an ordinary product.

Synthesis.

7. Explain program extraction from a constructive proof to a software engineer.
8. Explain proof erasure to a mathematician concerned that proofs will make programs inefficient.

Proof Workshop

- P1. Translate the proof of $(A \wedge B) \Rightarrow (B \wedge A)$ into a typing derivation and then into a TinyTT term.

P2. Exhibit an implication-introduction/elimination detour and perform Proposition 12.1 at both the derivation and term levels.

Design Clinic

Choose a constructive theorem with nontrivial computational content. Identify which parts of its proof should remain after proof erasure and which parts are logically necessary but operationally irrelevant.

Challenge.

9. Determine which classical tautologies fail to have inhabitants in the pure intuitionistic fragment and identify what additional control or logical principle would be required.

Type Checking: Turning Judgments into an Algorithm

The algorithmic viewpoint developed here belongs to a long programming-language tradition: Milner’s polymorphic type discipline made compile-time inference operationally central [11], while modern metatheoretic presentations such as Pierce and Harper separate static judgments, dynamics, and implementation architecture with particular clarity [12, 7].

13.1 From declarative rules to a machine procedure

A declarative typing rule tells us what counts as valid. A compiler or proof assistant needs something stronger: an algorithm that decides, or at least effectively searches for, a derivation.

For simple terms the declarative rules are almost executable. To type-check an application fa , infer the type of f , confirm it is a function type $A \rightarrow B$, then check that a has type A . To type-check a pair, type-check both components. To type-check a variable, look it up in the context.

The difficulties begin when syntax does not determine enough information. A lambda $\lambda x.t$ does not, by itself, tell us the type of x . We can annotate the binder, search for a type, or adopt a discipline that separates checking from inference.

13.2 Inference and checking

A **bidirectional** type checker uses two mutually recursive judgments:

$$\Gamma \vdash t \Rightarrow A$$

meaning “from t , synthesize its type A ,” and

$$\Gamma \vdash t \Leftarrow A$$

meaning “check that t has the already known expected type A .”

Variables and applications naturally synthesize. Lambdas naturally check against a known function type. An annotation can turn a checking problem into a synthesis problem.

This simple distinction scales remarkably far. Modern elaborators use bidirectionality together with metavariables, constraints, unification, implicit arguments, and normalization.

13.3 A tiny synthesis algorithm

For variables:

$$\frac{x : A \in \Gamma}{\Gamma \vdash x \Rightarrow A}.$$

For annotated terms:

$$\frac{\Gamma \vdash t \Leftarrow A}{\Gamma \vdash (t : A) \Rightarrow A}.$$

For applications:

$$\frac{\Gamma \vdash f \Rightarrow A \rightarrow B \quad \Gamma \vdash a \Leftarrow A}{\Gamma \vdash fa \Rightarrow B}.$$

For lambda checking:

$$\frac{\Gamma, x : A \vdash t \Leftarrow B}{\Gamma \vdash \lambda x.t \Leftarrow A \rightarrow B}.$$

The rules look almost like code because they have been oriented algorithmically.

13.4 Error messages as failed derivations

A useful type error should answer not merely “expected A , got B ” but where the attempted derivation failed.

Suppose an application expects a natural number but receives a function. The checker can report:

While checking the argument of this application, the function parameter required $\mathbb{N}$, but the supplied expression synthesized $\mathbb{N} \rightarrow \mathbb{N}$.

The structure of the derivation provides a causal path. Good diagnostics preserve that path rather than flattening every failure into an equality mismatch.

Worked Example 13.2: A pair error is a failed premise, not a mysterious verdict

Suppose the checker is asked to verify $\langle 0, \lambda x.x \rangle$ against $\mathbb{N} \times \mathbb{N}$. Pair checking creates two premises. The first succeeds immediately. The second asks the lambda to check

Bidirectional typing routes information instead of guessing globally

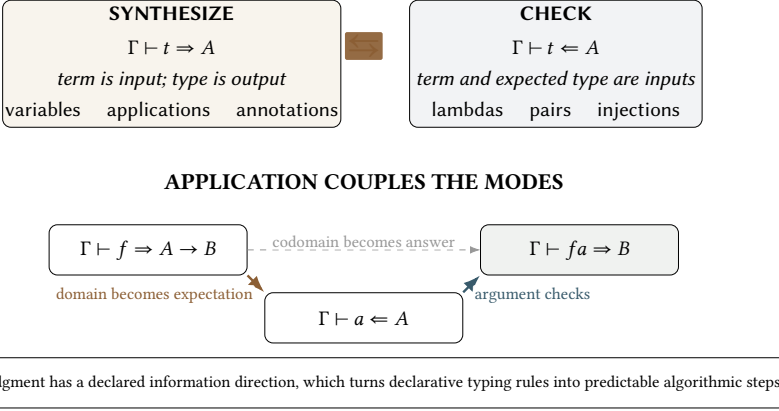


Figure 13.1: **Plate 34. Bidirectional typing.** Synthesis pushes information from syntax toward a type; checking pushes an expected type toward syntax. Application couples the modes: the function synthesizes an arrow, its domain becomes the expected type for the argument, and its codomain becomes the synthesized result. Bidirectionality does not change the declarative meaning of typing; it orients the rules into a predictable algorithmic discipline.

A useful type error preserves the failed derivation path

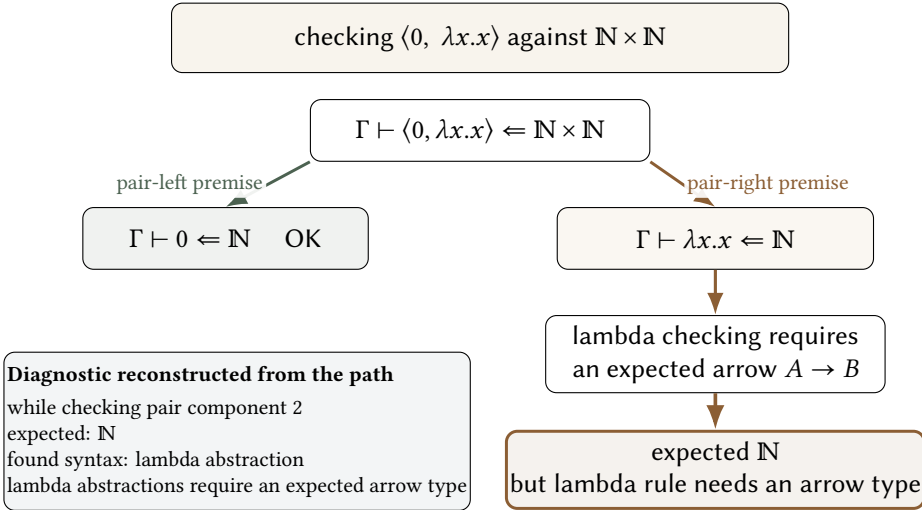


Figure 13.2: **Plate 36. Error as failed derivation.** The useful diagnostic is not merely the final inequality “expected $\mathbb{N}$, got something else.” It records which typing premise was being established, where that premise came from, and which rule became inapplicable. A derivation-oriented checker can therefore turn formal structure directly into causal user-facing diagnostics.

against $\mathbb{N}$, but the lambda rule can proceed only when the expected type exposes an arrow $A \rightarrow B$.

A good diagnostic can therefore report the entire local explanation: “while checking component 2 of a pair expected at $\mathbb{N} \times \mathbb{N}$, expected $\mathbb{N}$; encountered a lambda, whose checking rule requires an arrow type.” The error message is a compressed failed derivation.

13.5 Elaboration

Surface languages are usually more convenient than their cores. Users omit type arguments, use notation, overload operators, write pattern matching, and rely on inference. An **elaborator** translates this friendly language into a smaller explicit core whose terms are easier to check.

For example, a surface expression

$$f \ x$$

may elaborate to a core term carrying inferred type arguments and coercions. A pattern match may elaborate into primitive eliminators. Named notation may elaborate into nested applications.

The kernel then checks the explicit result independently.

This architecture separates convenience from trust:

$$\text{rich surface syntax} \rightarrow \text{elaboration} \rightarrow \text{small trusted core.}$$

13.6 The trusted computing base

Proof assistants often keep the kernel deliberately small. Sophisticated automation may search for proofs, but the final proof term is checked by a minimal trusted implementation of the judgment rules.

This realizes an important epistemic pattern:

Governing Principle

Search may be complicated; verification should be simpler than search whenever possible.

The same principle appears in compilers, cryptographic proofs, SAT certificates, and scientific computation. Type theory gives it a particularly explicit form: an artifact is accepted because the kernel can reconstruct or verify its judgment.

Proposition 13.1 (Independent checking localizes trust for the core judgment). *Let a search or elaboration procedure emit a core term t and claimed type A . If an independently*

implemented sound kernel accepts $\Gamma \vdash t : A$, then the validity of that core typing judgment does not depend on the internal correctness of the search heuristic that proposed t .

Proof. By kernel soundness, acceptance entails the declarative judgment $\Gamma \vdash t : A$. The route by which t was discovered is absent from that implication; it could have been produced by exhaustive search, a tactic, a language model, or a human. What remains outside the proposition is equally important: if t arose by elaborating richer surface syntax, kernel acceptance alone does not prove that the elaborator preserved the user's intended surface semantics. $\square$

13.7 Decidability

A type checker is most useful when it terminates. For simply typed lambda calculus with explicit annotations, type checking is decidable. For richer dependent theories, checking may invoke definitional equality, which in turn may require normalization.

If normalization is guaranteed and equality comparison is effective, decidable checking remains possible. If the theory admits unrestricted computation into definitional equality, the checker may inherit undecidability.

Thus the metatheory of computation is directly reflected in tooling behavior.

Formal Interlude XIII: Algorithmic soundness and completeness

An algorithmic checker should ideally satisfy two relationships to the declarative system.

Soundness:

$$\Gamma \vdash_{alg} t : A \implies \Gamma \vdash_{decl} t : A.$$

Completeness, up to annotations or elaboration:

$$\Gamma \vdash_{decl} t : A \implies \text{the algorithm accepts an appropriate elaboration of } t \text{ at } A.$$

Bidirectional systems may intentionally require annotations at points where a fully declarative derivation exists without them. The annotation discipline makes the algorithm predictable.

Theorem 13.2 (Soundness of the TinyTT bidirectional checker). *For a well-formed context Γ :*

- (a) *if $\text{synth}(\Gamma, t) = A$, then the corresponding declarative judgment $\Gamma \vdash t : A$ is derivable;*
- (b) *if $\text{check}(\Gamma, t, A)$ succeeds, then $\Gamma \vdash t : A$ is derivable.*

Proof sketch. Proceed by mutual induction on the recursive calls of **synth** and **check**. Each code branch is an oriented typing rule. Variable synthesis invokes context membership; annotations use the checking induction hypothesis; application synthesizes an arrow

and checks the argument; lambda checking extends the context and checks the body; products, sums, booleans, naturals, empty elimination, and the recursor reconstruct their matching declarative rules. The fallback from checking to synthesis is justified because exact type equality is required before success. $\square$

Remark 13.3. Completeness requires a qualification: bare lambdas, pairs, and injections do not synthesize in this implementation. A declaratively typable term may therefore need an expected type or an annotation before the algorithm has enough information to proceed.

Worked Example 13.1: Following information flow through the checker

To check $\lambda x.x$ against $\mathbb{N} \rightarrow \mathbb{N}$, the expected arrow type supplies information that the bare lambda lacks. The checker extends the context with $x : \mathbb{N}$ and checks the body against $\mathbb{N}$. The variable x synthesizes $\mathbb{N}$, which matches the expected type. In contrast, the application $f\ 0$ naturally synthesizes: synthesize the type of f , expose its domain and codomain, check 0 against the domain, and return the codomain. Bidirectionality is therefore an information-routing discipline, not merely a notation change.

Computational Laboratory 13: Build the bidirectional checker

The file `tinytt.py` contains `synth(ctx,t)` and `check(ctx,t,expected)`. Remove the provided implementation of lambda checking and reconstruct it from the rule above. Then add source-span information to AST nodes and propagate spans into type errors. The aim is to see a judgment derivation become a user-facing compiler service.

13.8 A minimal proof assistant architecture

Our tiny implementation now has four layers:

1. a syntax tree representing terms and types;
2. a context discipline for names and assumptions;
3. a normalization/equality engine;
4. a bidirectional kernel deciding typing judgments.

A real proof assistant adds universes, dependent functions and pairs, inductive families, equality types, elaboration metavariables, tactics, modules, termination checking, and libraries. But the architectural skeleton is already present.

The central judgment was never abstract ornament. It was an executable specification for a checker.

Exercises

Checkpoint.

1. Distinguish synthesis from checking.
2. Why do lambdas naturally check rather than synthesize in a basic bidirectional system?
3. What role does elaboration play between surface syntax and the kernel?

Core.

4. Write algorithmic rules for pair synthesis or checking.
5. Design a type error that reports a failed function application with context.
6. Explain why a small kernel can increase trust even if the elaborator is large.

Synthesis.

7. Compare proof-assistant architecture with a compiler frontend plus verified intermediate representation.
8. Explain how normalization affects the termination of dependent type checking.

Proof Workshop

- P1. Prove the application branch of Theorem 13.2 directly from the code contract of `synth` and the declarative application rule.
- P2. Give a declaratively typable bare lambda that TinyTT cannot synthesize. Add the smallest annotation that restores an algorithmic path.

Design Clinic

Add a derivation breadcrumb stack to the checker so a type mismatch reports the nested rule path that led to it. Demonstrate the output on an ill-typed application inside a pair.

Challenge.

9. Give a case where declarative typing is elegant but not syntax directed. Propose an annotation or bidirectional rule that restores a deterministic checking path.

Make search powerful; keep acceptance small

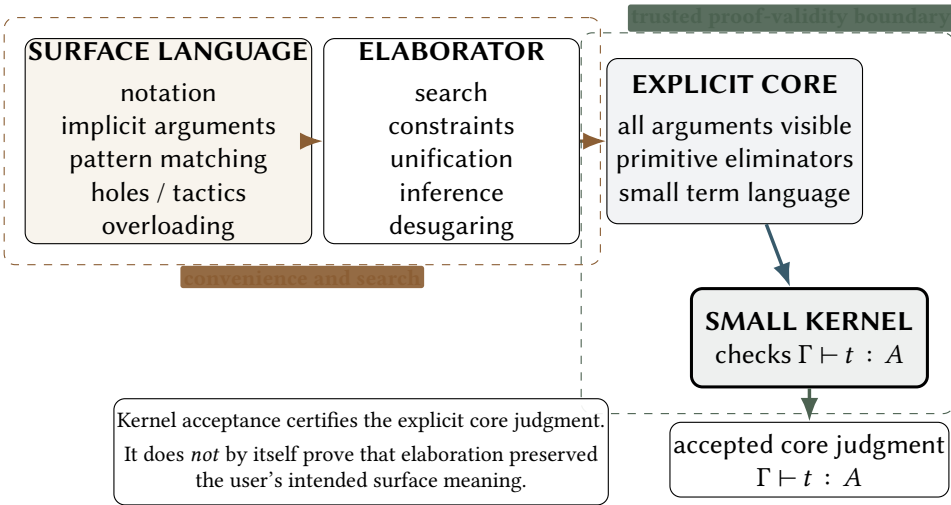


Figure 13.3: **Plate 35. Elaborator and trusted kernel.** A large elaborator may perform sophisticated, heuristic work while a smaller kernel verifies the explicit artifact it produces. This localizes trust for the formal typing claim: search need not itself be trusted if its output is independently checked. The boundary is not magical, however; faithful elaboration from surface syntax to core syntax is a separate semantic obligation.

Judgment as Communication: From Expressions to Protocol States

14.1 The turnstile as a communication boundary

We began with the question: what does it mean for an expression to make sense? We can now sharpen the answer. A judgment records that, in a particular informational situation, an expression may be transmitted, used, or relied upon according to a stated type.

This gives the turnstile a communicative reading:

$$\Gamma \vdash a : A.$$

The context is the shared state of understanding. The term is the payload or construction. The type is the protocol of admissible use. The judgment says the message is licensed in that state.

This reading is not meant to replace the logical or programming interpretation. It is another projection of the same formal skeleton.

14.2 A function call is a tiny protocol

The type

$$A \rightarrow B$$

encodes a two-stage interaction:

1. the caller supplies an A ;
2. the callee supplies a B .

The protocol is minimal because there is only one request and one response. Currying reveals multi-stage protocols:

$$A \rightarrow B \rightarrow C$$

can be read as “receive an A , then receive a B , then produce a C .”

Dependent functions make the later protocol state depend on earlier communication. Session types, developed later in the series, make this temporal reading explicit and symmetric across communicating parties.

14.3 Contexts as protocol states

Suppose a dialogue begins with no shared value of type A . After one party transmits $a : A$, the receiving side may extend its context:

$$\Gamma \rightsquigarrow \Gamma, x : A.$$

A subsequent message may have a type that mentions x . Thus communication changes not merely available data but the grammar of legal future communication.

This is dependency seen operationally.

Remark 14.1 (Contexts are not clocks). The protocol reading adds structure that ordinary typing contexts do not contain by themselves. A context records which assumptions are available for a judgment; it does not, merely by being written left to right, enforce who sent which message, when a message may arrive, whether a peer must eventually respond, or whether two endpoints agree on dual actions. Later protocol type systems add those temporal and relational constraints explicitly.

14.4 APIs as partial type theories

An API defines operations, parameter types, return types, state constraints, and often sequencing restrictions. In that sense, every serious interface is a partial theory of the computations it permits.

A typed API says:

If you can provide evidence of these preconditions, you may invoke this operation; afterward you may rely on these postconditions and capabilities.

Ordinary API types are often too weak to encode all state transitions. Richer type systems can make illegal states unrepresentable, track tpestate, enforce ownership, or describe communication protocols.

14.5 The limits of the closed-term worldview

A closed term can be evaluated without further input. But much of computation is open: it awaits messages, sensor readings, human decisions, or other agents.

After the machinery is earned, a judgment can be read as communication

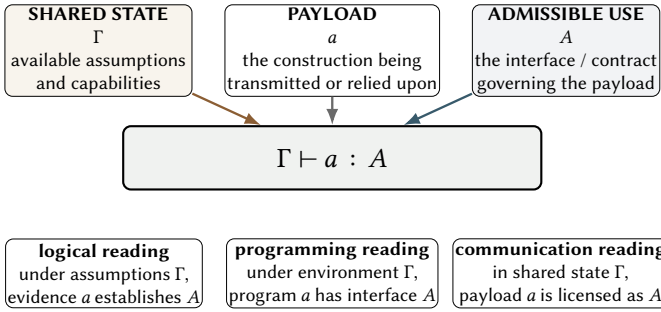
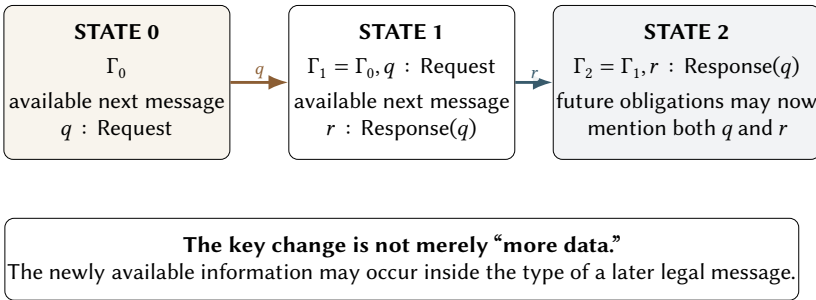


Figure 14.1: **Plate 37. Judgment as communication.** The communicative reading becomes precise only after contexts, substitution, constructors, eliminators, normalization, and checking have been developed. Here Γ is shared informational state, a is the construction under consideration, and A governs its admissible use. This is a reading of typing relative to information, not yet a full temporal protocol semantics; session structure arrives later in the series.

Communication can change the grammar of what may come next



A typing context records assumptions; a protocol semantics adds temporal transition rules. The picture combines them deliberately, but they are not identical notions.

Figure 14.2: **Plate 38. Context as protocol state.** After a request q is received, a later response contract may depend on that particular request, schematically $\text{Response}(q)$. Context extension therefore suggests an operational reading in which communication changes the space of legal continuations. The analogy has a boundary: ordinary typing contexts are not clocks, and context extension alone does not impose message order, liveness, duality, or any other temporal protocol law.

The central object for open computation may therefore be not a value but a transition relation, strategy, process, or protocol. Type theory remains relevant because types can classify those interactive structures, but normalization to a final value is no longer the universal story.

The grand-unification thesis must therefore evolve from

$$\text{computation} = \text{normalization}$$

toward

$$\text{computation} = \text{lawful evolution of typed possibilities under information.}$$

Volume I has built the vocabulary needed to make that stronger claim precise later.

14.6 Language is not merely representation

A language determines which distinctions can be stated succinctly and which guarantees can be enforced mechanically. Changing the type language changes the space of programs one can express and verify.

A language with only booleans and naturals cannot state “this list has the same length as that index.” A language with dependent types can. A language without linearity cannot directly express “this capability must be consumed exactly once.” A language without effect information cannot distinguish a pure computation from one that writes to disk.

Thus language is constitutive of computational possibility, not merely a notation placed over an already fixed universe of programs.

14.7 Collected comprehension principles

We can now revisit the thesis that type theories are collected comprehension principles.

Each type former licenses a new way to build a space of possible constructions:

Type former	Informational act	Computational reading
$A \rightarrow B$	hypothetical dependence	transformation
$A \times B$	combine both	paired information
$A + B$	choose one, record which	tagged alternative
1	trivial evidence	acknowledgement
0	no canonical evidence	impossibility
$\mathbb{N}$	finite generation	iteration/induction
$\Pi_{x:A} B(x)$	dependent uniformity	output contract varies with input
$\Sigma_{x:A} B(x)$	dependent package	witness plus certified payload

A type theory is, in this sense, a constitution of permissible world-building operations.

Governing Principle

Changing the collection of type formers changes which dependencies and invariants can be stated inside the language itself. Changing the computation and equality rules changes which of those statements the checker can discharge automatically. A type theory is therefore specified not by a vocabulary of type names alone, but by a coupled package of formation, construction, observation, computation, and equality principles.

14.8 What Volume I has established

The book began before programs. We can now state the chain with precision:

1. A formal system begins with judgments, not bare expressions.
2. Judgments are relative to contexts of available information.
3. Variables make hypothetical information locally usable.
4. Substitution turns hypothetical dependency into concrete specialization.
5. Type formers define new spaces of constructions by formation, introduction, elimination, and computation rules.
6. Function abstraction packages hypothetical derivations.
7. Products and sums describe elementary information composition.
8. Inductive types tie data generation to recursive and inductive computation.
9. Definitional equality makes computation visible to typing.
10. Normalization exposes canonical computational content in the pure fragment.
11. Curry-Howard identifies the same rules as both proof construction and program construction.
12. Algorithmic typing turns the judgment discipline into executable verification.

None of these steps required us to begin with a machine model. Machines become implementations of formally disciplined computational languages rather than the source of meaning itself.

The synthesis map is deliberately causal rather than chronological. The checker, for example, arrives late in the exposition but returns us to the first question: which judgments are licensed? Normalization is introduced after the basic type formers, yet its equality consequences feed backward into checking. A mature formal system is therefore not a one-way pipeline. It is a small network of mutually supporting invariants.

14.9 The next necessity: types that depend on information

The limitations of simple types are now visible everywhere. Martin-Löf’s constructive type theories make dependency foundational rather than ornamental [9, 10]. We want to state:

- a vector has length n ;
- a parser returns a syntax tree matching a grammar;
- a matrix multiplication output shape depends on input dimensions;
- a protocol’s legal next message depends on the previous message;
- a proof’s conclusion depends on the witness supplied;
- an API response schema depends on the request kind.

In each case, a type must depend on a term.

This is not an optional ornament. It completes the context and substitution machinery already developed.

Formal Interlude XIV: The threshold of dependent type theory

The decisive judgment is

$$\Gamma, x : A \vdash B(x) \text{ type.}$$

It says that after receiving information $x : A$, a new type $B(x)$ becomes meaningful. Two canonical comprehension principles then arise:

$$\Pi_{x:A} B(x) \quad \text{and} \quad \Sigma_{x:A} B(x).$$

The dependent product packages a construction that works uniformly for every x while delivering the corresponding $B(x)$. The dependent sum packages a particular x together with an inhabitant of its corresponding $B(x)$.

Volume II begins here.

Proposition 14.2 (The simple-arrow grammar cannot bind a term in its codomain). *In the simple type grammar of Volume I, forming $A \rightarrow B$ does not introduce a term variable that may occur in B . Consequently a nonconstant family $B(x)$ cannot be expressed directly as the codomain of an ordinary simple arrow whose input is $x : A$.*

Proof. Inspect the formation rule for the simple arrow: it requires $\Gamma \vdash A \text{ type}$ and $\Gamma \vdash B \text{ type}$, and concludes $\Gamma \vdash A \rightarrow B \text{ type}$. The context used to form B is still Γ ; no declaration $x : A$ has been introduced. Dependent product formation changes exactly this premise to $\Gamma, x : A \vdash B(x) \text{ type}$, thereby making the input available while the codomain is formed. $\square$

Worked Example 14.1: A result contract selected by an input

Suppose head should accept a vector whose length is known to be a successor:

$$\text{head} : \prod_{n:\mathbb{N}} \text{Vec}(A, \text{succ}(n)) \rightarrow A.$$

The vector type appearing after the first argument contains that argument. A simple arrow grammar can describe $\mathbb{N} \rightarrow C$ only when the codomain C is already a type independent of the supplied numeral. The example exposes the missing operation: substitution must act inside types, because incoming information determines which type is meaningful next.

Computational Laboratory 14: The threshold experiment

Modify the tiny checker so that a type annotation may mention a natural-number literal in a vector-like constructor `VecTy(elem, n)`. You do not need to implement full dependent typing. Instead, observe exactly where the current checker architecture breaks when the type of a result must substitute an input term. Write down the required changes. This failed extension is the intended result: it exposes why dependent substitution must operate at the type level.

14.10 The thesis after one volume

We can now defend a stronger statement than “types classify programs” but a weaker statement than the full series hypothesis:

Governing Principle

For a broad class of formal computations, a type theory specifies the grammar of meaningful constructions, the information under which they are meaningful, the ways they may be composed and observed, and the reductions by which their computational content becomes explicit.

The remaining volumes ask whether that grammar can be extended far enough to deserve the title *grand unified theory of computation*.

The series atlas makes the burden of that title explicit. Volume I establishes a disciplined grammar for pure typed construction. The later volumes must show what survives when computation becomes interactive, effectful, reflective, higher-dimensional, partial, or otherwise resistant to the closed normalizing picture. The synthesis is earned only if those extensions preserve enough common structure to justify speaking of one theory rather than a family resemblance.

Exercises

Checkpoint.

1. Give a communication reading of $\Gamma \vdash a : A$.
2. Why can $A \rightarrow B$ be read as a tiny protocol?
3. State one computational specification that simple types cannot express naturally but dependent types can.

Core.

4. Reinterpret context extension as a protocol-state transition.
5. Explain how an API's type language determines which misuse can be rejected before execution.
6. Explain the phrase “collected comprehension principles” using three type formers.

Synthesis.

7. Choose an interactive system – database, robot, browser, or agent – and identify which parts fit the closed function model and which require a richer protocol model.
8. Defend or criticize the claim that programming-language design is partly ontology design: choosing which computational distinctions exist in the language.

Proof Workshop

- P1. Prove Proposition 14.2 by writing the simple arrow formation rule and the dependent product formation rule side by side and isolating the changed context.
- P2. Give three nonconstant families $B(x)$ drawn from data shape, protocol state, and proof evidence. State exactly what information indexes each family.

Design Clinic

Specify a two-message request/response protocol in which the response type depends on the request tag. Explain why a simple sum of all responses loses information that a dependent response family retains.

Challenge.

9. Give the strongest version of the “type theory is the grammar of computation” thesis that you now consider defensible, and list three empirical or formal counterexamples that a complete theory would still have to answer.

A type theory is a collection of licensed world-building moves

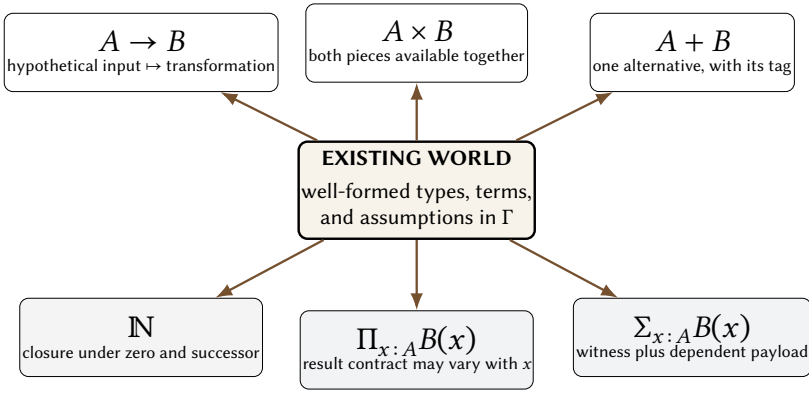


Figure 14.3: **Plate 39. Collected comprehension principles.** Each type former enlarges the language of admissible constructions in a characteristic way: arrows internalize hypothetical transformation, products coexistence, sums alternatives, naturals generated closure, and dependent products and sums information-indexed construction. The picture is a rule atlas, not an ontology of literal spatial worlds. The Π and Σ gates mark the threshold of Volume II rather than constructors already implemented by TinyTT.

Volume I forms a causal spine, then closes a verification loop

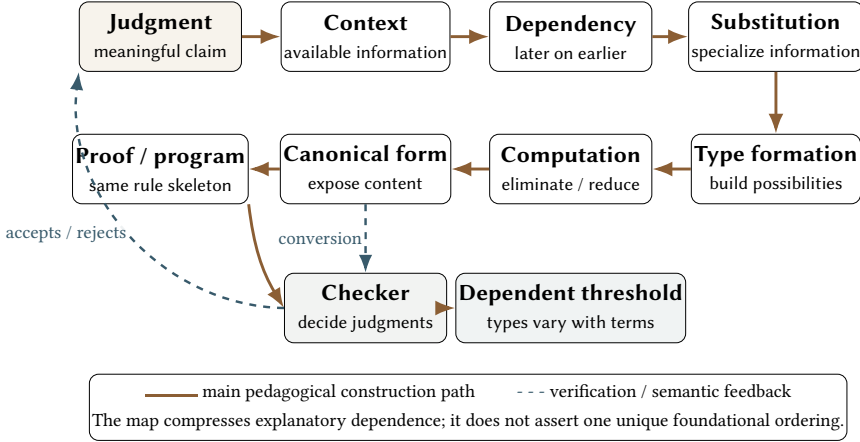


Figure 14.4: **Plate 40. Volume I synthesis.** The warm spine summarizes the book’s construction order: judgments become contextual, dependency becomes operational through substitution, type formation enables computation, normalization exposes canonical content, Curry–Howard re-reads the same rules, and an algorithmic checker returns to the original judgment before simple types reach their dependent boundary. Dashed arrows record feedback used in verification. The topology is pedagogical rather than a claim of unique foundational precedence.

Three different problems demand the same missing grammatical move

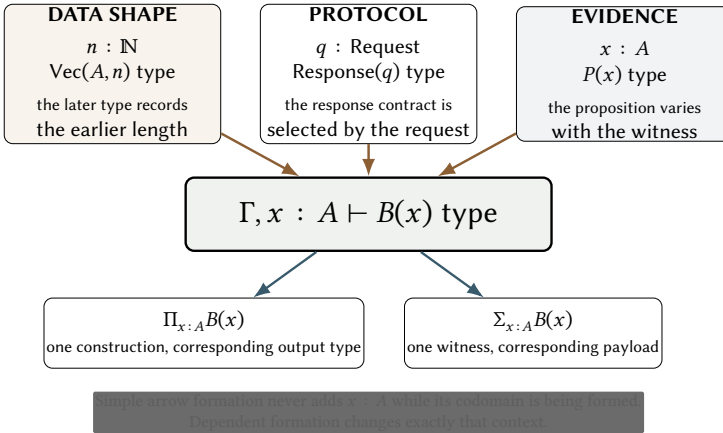


Figure 14.5: **Plate 41. Threshold to Volume II.** Length-indexed data, request-indexed responses, and witness-indexed propositions are structurally the same demand: after obtaining $x : A$, the language must be able to form a type $B(x)$ that mentions it. Dependent products and sums internalize two canonical ways of using such a family. The examples motivate the common judgment; they do not claim that vectors, protocols, and propositions have identical semantics.

The collection is one hypothesis examined from ten directions

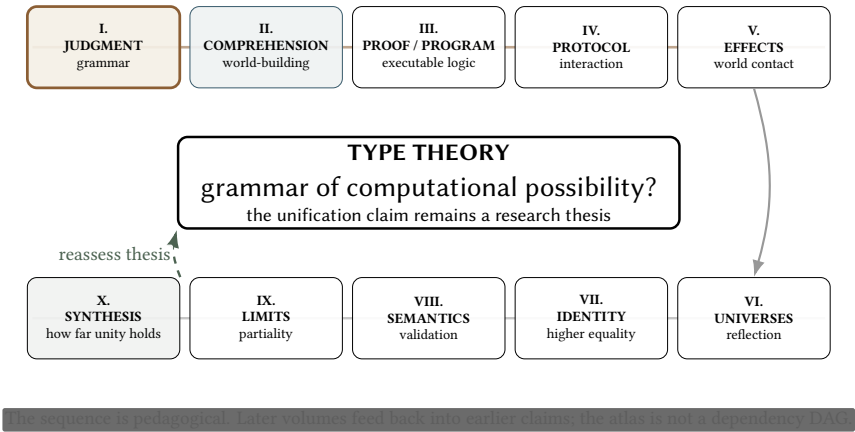


Figure 14.6: **Plate 42. Series atlas.** Volume I establishes the judgmental grammar and hands directly to comprehension in Volume II. The remaining monographs test the unification thesis against proof, interaction, effects, reflection, identity, semantics, and the limits of formal construction before attempting a synthesis. This is an editorial research map, not a theorem that the subjects form a strict hierarchy or a single linear prerequisite chain.

A Compact Formal Reference

This appendix collects the core syntax and rules used throughout the volume. It is intentionally smaller than a production language.

A.1 Notation guide

The notation of Volume I is deliberately small. The following table is the authoritative quick reference. When the same printed mark has an ordinary mathematical use outside the core calculus, the local judgment or grammar determines its meaning. Color is never semantic.

Notation	Meaning in this volume
Γ, Δ	Contexts: ordered collections of typed declarations used by a judgment.
$x : A$	A declaration that variable x is available at type A .
$\Gamma \vdash t : A$	Under context Γ , term t has type A .
$\Gamma \vdash A \text{ type}$	A is a well-formed type in context Γ . This becomes essential in dependent type theory; in the simple core, types do not depend on term variables.
$\Gamma \text{ ctx}$	Γ is a well-formed context.
$A \rightarrow B$	Function type: given an A , a computation may produce a B .
$A \times B$	Product type: an inhabitant contains both an A and a B .
$A + B$	Sum type: an inhabitant contains either a tagged A or a tagged B . Here $+$ is a type former, not arithmetic addition.
1, 0	Unit and empty types: respectively one canonical inhabitant and no canonical inhabitant.
$\mathbb{N}$	Natural-number type generated by 0 and succ.
$\lambda x. t$	Function abstraction binding x in t .
$\langle t, u \rangle$	Product introduction (pair).
$\text{inl}(t), \text{inr}(u)$	Left and right introductions for a sum.
$t[a/x]$	Capture-avoiding substitution of a for free occurrences of x in t .
$\text{FV}(t)$	Set of free variables of t .
$t \rightarrow u$	One computation/reduction step. This long arrow is distinct from the type-forming arrow $A \rightarrow B$.

Notation	Meaning in this volume
$t \rightarrow^* u$	Zero or more reduction steps (reflexive-transitive closure).
$\Gamma \vdash t \equiv u : A$	Definitional equality at type A , generated in this volume by the stated computation rules, congruence, equivalence closure, and alpha-renaming.
$\bar{n}$	The canonical numeral $\text{succ}^n(0)$.
$\Pi_{x:A} B(x), \Sigma_{x:A} B(x)$	Prospective dependent product and dependent sum notation. These are previewed at the Volume II threshold but are not part of TinyTT's simple core.

Overloading policy. The book uses familiar mathematical symbols only when their role is recoverable from the surrounding grammar. Thus $+$ between types means a sum type, while arithmetic operations are written as named or defined term-level programs. Binding is always determined structurally, not by color or typography. A symbol introduced only prospectively, such as Π or Σ , is marked as outside the executable TinyTT core.

A.2 Types and terms

Types:

$$A, B ::= \mathbb{N} \mid \text{Bool} \mid \mathbf{1} \mid \mathbf{0} \mid A \rightarrow B \mid A \times B \mid A + B.$$

Terms:

$$\begin{aligned}
t, u ::= & x \mid 0 \mid \text{succ}(t) \mid \text{true} \mid \text{false} \\
& \mid \text{if}(t; u; v) \mid \lambda x. t \mid t u \\
& \mid \langle t, u \rangle \mid \pi_1 t \mid \pi_2 t \\
& \mid \text{inl}(t) \mid \text{inr}(t) \\
& \mid \text{case}(t; x.u; y.v) \\
& \mid \star \mid \text{absurd}_A(t) \\
& \mid \text{rec}_{\mathbb{N}}(t; u; v).
\end{aligned}$$

A.3 Contexts

$$\Gamma ::= \cdot \mid \Gamma, x : A.$$

A variable may be used when declared in the context. Contexts are assumed well formed, and names are taken modulo alpha-renaming.

A.4 Selected typing rules

Variable:

$$\frac{x : A \in \Gamma}{\Gamma \vdash x : A}.$$

Function:

$$\frac{\Gamma, x : A \vdash t : B}{\Gamma \vdash \lambda x.t : A \rightarrow B}, \quad \frac{\Gamma \vdash f : A \rightarrow B \quad \Gamma \vdash a : A}{\Gamma \vdash fa : B}.$$

Product:

$$\frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B}{\Gamma \vdash \langle a, b \rangle : A \times B},$$

$$\frac{\Gamma \vdash p : A \times B}{\Gamma \vdash \pi_1 p : A}, \quad \frac{\Gamma \vdash p : A \times B}{\Gamma \vdash \pi_2 p : B}.$$

Sum:

$$\frac{\Gamma \vdash a : A}{\Gamma \vdash \text{inl}(a) : A + B}, \quad \frac{\Gamma \vdash b : B}{\Gamma \vdash \text{inr}(b) : A + B},$$

$$\frac{\Gamma \vdash s : A + B \quad \Gamma, x : A \vdash t : C \quad \Gamma, y : B \vdash u : C}{\Gamma \vdash \text{case}(s; x.t; y.u) : C}.$$

Booleans:

$$\overline{\Gamma \vdash \text{true} : \text{Bool}}, \quad \overline{\Gamma \vdash \text{false} : \text{Bool}},$$

$$\frac{\Gamma \vdash c : \text{Bool} \quad \Gamma \vdash t : A \quad \Gamma \vdash u : A}{\Gamma \vdash \text{if}(c; t; u) : A}.$$

Naturals:

$$\overline{\Gamma \vdash 0 : \mathbb{N}}, \quad \frac{\Gamma \vdash n : \mathbb{N}}{\Gamma \vdash \text{succ}(n) : \mathbb{N}}.$$

A.5 Core reductions

$$\text{if}(\text{true}; t; u) \rightarrow t, \quad \text{if}(\text{false}; t; u) \rightarrow u,$$

$$(\lambda x.t) u \rightarrow t[u/x],$$

$$\pi_1 \langle t, u \rangle \rightarrow t, \quad \pi_2 \langle t, u \rangle \rightarrow u,$$

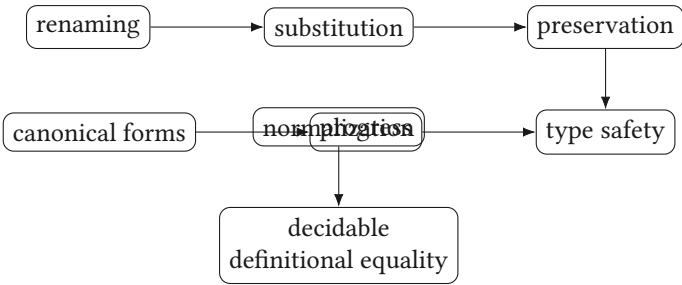
$$\text{case}(\text{inl}(t); x.u; y.v) \rightarrow u[t/x],$$

$$\text{case}(\text{inr}(t); x.u; y.v) \rightarrow v[t/y],$$

$$\text{rec}_{\mathbb{N}}(0; c_0; s) \rightarrow c_0,$$

$$\text{rec}_{\mathbb{N}}(\text{succ}(n); c_0; s) \rightarrow s(\text{rec}_{\mathbb{N}}(n; c_0; s)).$$

A.6 Metatheoretic dependency graph



Computational Laboratory Guide

B.1 Purpose

The laboratory code is deliberately small enough to read in one sitting. It is not a production parser, compiler, or proof assistant. Its purpose is to make the formal rules executable and to provide a controlled environment in which readers can break invariants and observe the consequences.

B.2 Recommended sequence

1. Parse and print ASTs without typing.
2. Implement context lookup and free-variable calculation.
3. Implement capture-avoiding substitution.
4. Implement one-step reduction and normalization.
5. Implement simple type synthesis for variables, annotated terms, and applications.
6. Implement checking for lambdas and constructor forms.
7. Add products and sums.
8. Add naturals and a recursor.
9. Add definitional equality by normalization.
10. Instrument errors with source spans and derivation traces.

B.3 Laboratory rubric

For each extension, the reader should answer four questions:

1. Which formation/introduction/elimination/computation rule is being implemented?
2. What invariant does the code assume about contexts and variables?
3. Which metatheoretic property would fail if the implementation were wrong?
4. What new class of programs becomes expressible after the extension?

B.4 Experiments worth keeping

Readers should preserve a notebook of counterexamples. The most educational entries are not successful programs but near misses: capture bugs, missing sum branches, non-function applications, mismatched projections, and attempted self-application. Each failure should be annotated with the exact judgment rule that blocks it.

Selected Exercise Notes and Solutions

The following are concise solution sketches for representative exercises. They illustrate proof patterns without reproducing the complete answer key. Release Candidate 1 is accompanied by a separate *Instructor and Self-Study Solutions Companion* containing keyed responses, hints, proof sketches, and design rubrics for all 168 chapter exercises.

C.1 Function inhabitants

An inhabitant of $A \rightarrow B \rightarrow A$ is

$$\lambda x. \lambda y. x.$$

The second input is permitted to go unused because ordinary contexts admit weakening.

An inhabitant of

$$(A \rightarrow B) \rightarrow (B \rightarrow C) \rightarrow A \rightarrow C$$

is

$$\lambda f. \lambda g. \lambda x. g(f(x)).$$

The type nearly dictates the program: the only available route from A to C is through B .

C.2 Commuting products and sums

For products:

$$\text{swap}_\times = \lambda p. \langle \pi_2 p, \pi_1 p \rangle.$$

For sums:

$$\text{swap}_+ = \lambda s. \text{case}(s; x.\text{inr}(x); y.\text{inl}(y)).$$

The difference is pedagogically important. Product elimination observes both components; sum elimination must branch because only one payload exists.

C.3 Why an arbitrary atomic A has no closed inhabitant

The introduction rules provide no closed canonical constructor whose conclusion is an arbitrary atomic type A . That observation alone is not yet a proof that no closed term has type A , because a closed term could begin as an elimination form. The missing bridge is normalization (or an equivalent logical-relations argument): if every closed well-typed term normalizes and canonical forms are type-directed, then a hypothetical closed inhabitant of atomic A would normalize to a canonical inhabitant of A , but no such canonical form exists. This distinction between “no constructor” and “no inhabitant” is exactly why metatheory matters.

C.4 The capture example

For

$$(\lambda y.x)[y/x],$$

first alpha-rename the binder to a fresh z :

$$\lambda z.x.$$

Then substitute:

$$\lambda z.y.$$

The free y remains free, preserving dependency structure.

C.5 Weakening: the application case

Suppose $\Gamma \vdash f : A \rightarrow B$ and $\Gamma \vdash a : A$. By the induction hypotheses for weakening, both premises remain derivable in any well-formed extension Γ' that preserves the relevant declarations. Reapplying function elimination yields $\Gamma' \vdash fa : B$. The proof is representative: structural metatheorems are propagated rule by rule.

C.6 Substitution: the binder case

For $\lambda y.u$, the proof cannot blindly push substitution under the binder. If $y \in \text{FV}(a)$, first alpha-rename y to a fresh z . Then the induction hypothesis applies to the body without capturing any dependency of a . This is the exact point at which the mathematical proof and the implementation’s `fresh` operation express the same invariant.

C.7 Progress for applications

If $\vdash fa : B$, inversion gives $\vdash f : A \rightarrow B$ and $\vdash a : A$. By induction, f either steps or is a value. If it steps, the application steps on the left. If it is a value, the canonical-forms lemma for arrows says it is a lambda. Then either a steps, or a is a value and beta reduction applies. Thus a closed well-typed application cannot be stuck.

C.8 Bidirectional soundness: application

If $\text{synth}(\text{ctx}, \text{App}(f, a))$ returns B , the code first synthesizes $A \rightarrow B$ for f and then checks a at A . By the mutual induction hypotheses these correspond to declarative derivations of $\Gamma \vdash f : A \rightarrow B$ and $\Gamma \vdash a : A$. The declarative application rule concludes $\Gamma \vdash fa : B$.

Illustration Register and Production Status

D.1 Visual doctrine

Every major plate must carry a specific explanatory burden. The visual language should resemble an engineer's notebook crossed with a mathematical atlas: restrained linework, generous negative space, warm paper, dark ink, and one or two semantic accent colors. Decorative complexity is prohibited unless it reveals dependency, flow, construction history, or observational structure.

Each plate has four layers available to the designer:

1. **Phenomenon**: the ordinary-language situation.
2. **Structure**: the diagrammatic abstraction.
3. **Judgment**: the formal notation.
4. **Failure mode**: a nearby invalid construction showing why the distinction matters.

The preferred visual progression is phenomenon → structure → notation, never notation → unexplained decoration.

Governing Principle

Plates 1–42 are now finished vector illustrations embedded at their first pedagogically relevant location in the manuscript. The complete canonical visual programme for Volume I is therefore closed for this edition. A plate is marked finished only when it compiles in the book, has a caption that states the limit of its analogy, remains legible in grayscale, and survives full-page render inspection.

No.	Plate	Pedagogical burden	Composition specification
1	Before execution	Separate physical symbol manipulation from meaningful formal use.	Split page: left, arbitrary keypress/symbol stream; right, same marks organized as parse tree and then licensed judgment. Highlight the extra layer rather than a “bad vs good” moral.
2	Anatomy of a judgment	Make every component of $\Gamma \vdash a : A$ visually legible.	Central judgment with four call-outs: context, turnstile, term, type. Bottom row gives programming, logical, and communication readings.
3	Marks / expressions / judgments	Prevent object-language/metalevel confusion.	Three stacked transparent sheets: bytes/tokens; AST; derivability claim. Arrows only upward, emphasizing added structure.
4	A rule manufactures a judgment	Explain inference rules as licenses.	Mechanical press metaphor rendered abstractly: premises feed through a rule bar and produce conclusion; beside it, the formal fraction notation.
5	Derivation tree for two	Show proof provenance.	Tree for $\text{succ}(\text{succ}(0)) : \mathbb{N}$ with each rule named in the margin; root at bottom to match inference-rule reading.
6	Context as a room	Introduce contexts as informational worlds.	Architectural plan with declarations placed as available tools; a target term outside is connected only to tools it uses. Avoid literal people.
7	Context extension	Make comprehension concrete.	Nested frames $\Gamma \subset \Gamma, x : A \subset \Gamma, x : A, y : B(x)$; later frame visibly depends on earlier declaration.
8	Structural rules as resource policy	Reveal weakening/contraction/exchange as computational assumptions.	Three mini-panels: discard token, duplicate token, reorder independent tokens; crossed-out variants preview linear systems.
9	Bound and free variables	Clarify scope.	Lambda expression with translucent scope brackets and colored tether from each occurrence to its binder or external context.
10	Alpha-equivalence	Separate names from binding identity.	Two differently named lambda trees overlaid to identical binding arcs; a third capture example visibly changes an arc.

No.	Plate	Pedagogical burden	Composition specification
11	Type family as fibers	Prepare dependency.	Horizontal base A with several points; vertical fibers $B(x_i)$ of different shapes. Caption: “information chooses the space in which the next object must live.”
12	Substitution as information flow	Make substitution the central dynamic.	Dependency graph with open variable source x feeding term and type nodes; concrete a arrives and reroutes all edges.
13	Capture-avoiding substitution	Explain why substitution is not search-and-replace.	Before/naive/correct triptych. Naive middle panel shows a free-variable tether accidentally attached to inner binder.
14	Beta reduction	Show application as closing a hypothetical world.	Lambda enclosure holding an open socket labeled $x : A$; argument plugs in; enclosure unfolds into specialized body. Formal equation below.
15	Formation/introduction/elimination/computation	Establish the four-question template.	Four-quadrant compass around a type former. Each quadrant includes one question and an icon: legitimacy, construction, observation, reduction.
16	Harmony	Show constructor/eliminator fit.	Pair constructor deposits two components in a sealed object; projections recover exactly those components. A forbidden extra outlet marked as extracting information never introduced.
17	Function as staged capability	Read $A \rightarrow B$ behaviorally.	Left-to-right handshake: supply A , activate suspended lambda body, receive B . A smaller logical caption gives implication reading.
18	Higher-order composition	Visualize functions as movable objects.	Three typed pipes $A \rightarrow B, B \rightarrow C$, composed into $A \rightarrow C$; composition term written as wiring diagram rather than plumbing realism.
19	Product	Both pieces exist simultaneously.	Pair container with two observable ports π_1, π_2 ; no branch structure.
20	Sum	Exactly one tagged alternative exists.	Y-shaped constructor paths entering a single sum capsule; eliminator splits back into exhaustive left/right cases.

No.	Plate	Pedagogical burden	Composition specification
21	Algebra of data	Show sums-of-products.	A small AST datatype decomposed into alternative constructors, each constructor expanded into a product of fields.
22	Distributivity as program	Turn algebraic identity into computation.	Animated-style sequence from $A \times (B + C)$ to $(A \times B) + (A \times C)$ using tag-preserving rewiring.
23	Natural numbers as generated structure	Tie constructors to induction.	Spiral or ladder beginning at 0, each successor visibly one constructor layer; side bracket marks structural depth.
24	Recursor follows construction history	Explain structural recursion.	Input numeral shown as stack of successor shells; evaluator peels one shell per recursive call until zero.
25	Induction fibers	Distinguish recursion from dependent induction.	Naturals on base line, predicate fibers $P(n)$ above; induction step is a bridge carrying evidence from one fiber to the next.
26	Equality spectrum	Prevent collapse of different equality notions.	Horizontal spectrum: textual identity, alpha, definitional, propositional, extensional, contextual. Each has a distinct “who decides?” annotation.
27	Confluence diamond	Explain evaluation-path independence.	Standard diamond with two reduction paths reconverging; accompany with a real lambda term containing independent redexes.
28	Equality budget	Connect expressiveness to checker cost.	Balance diagram: automatic equality rules on one side, algorithmic predictability on the other; no claim of a strict zero-sum law.
29	Canonical forms	Show type-directed final shapes.	Gallery: function $\rightarrow$ lambda, product $\rightarrow$ pair, sum $\rightarrow$ tagged injection, natural $\rightarrow$ numeral.
30	Divergence boundary	Contrast typed normalization with untyped self-application.	Computed Ω self-loop on one side; finite simple typing constraints on the other prevent the self-application wiring.
31	Progress and preservation	Visualize type safety.	Typed reduction track. Progress gives value-or-step; preservation keeps the type invariant along legal movement.

No.	Plate	Pedagogical burden	Composition specification
32	Curry-Howard correspondence	Show rule-level, not slogan-level, correspondence.	Three aligned columns: logic, types, computation. Arrows connect implication introduction to lambda abstraction and elimination to application, beta to proof detour removal.
33	Inhabitation as search	Type as a program specification.	Target type at top, available hypotheses as inventory, valid constructor choices as branching search tree; one successful term highlighted.
34	Bidirectional typing	Explain synth/check information flow.	Two opposing arrows: term $\Rightarrow$ type and term $\Leftarrow$ expected type. Place variables/application on synth side, lambdas on check side.
35	Elaborator and kernel	Separate convenience from trust.	Funnel from rich surface language through elaborator into compact core term, then small kernel stamp. Automation shown outside trusted boundary.
36	Error as failed derivation	Make diagnostics structural.	Derivation tree with one branch turning red at exact mismatched premise; corresponding source span connected by a thin line.
37	Judgment as communication	Reinterpret the central form after the technical development.	Context as shared state, term as transmitted payload, type as protocol envelope; the same $\Gamma \vdash a : A$ centered.
38	Context as protocol state	Show information-dependent future.	Sequence of context frames after successive messages; available next-message types narrow or change after each event.
39	Collected comprehension principles	Summarize type formers as world-building operations.	Radial atlas: central existing world, outward gates labeled $\rightarrow, \times, +, \Pi, \Sigma$, each opening a differently structured space.
40	Volume I synthesis	Compress the whole book into one causal map.	Judgment $\rightarrow$ context $\rightarrow$ dependency $\rightarrow$ substitution $\rightarrow$ type formation $\rightarrow$ computation $\rightarrow$ canonical form $\rightarrow$ proof/program $\rightarrow$ checker $\rightarrow$ dependent threshold. Use one warm accent for the main path and cool accent for feedback arrows.

No.	Plate	Pedagogical burden	Composition specification
41	Threshold to Volume II	Make dependency feel inevitable.	Three concrete examples - vector length, protocol response, existential witness - each projected onto the same formal schema $x : A \vdash B(x)$ type.
42	Series map	Situate Volume I in the full collection.	Ten-volume orbit/atlas, with Judgment highlighted and arrows showing later themes: proof/program, protocol, effects, universes, identity, semantics, limits, synthesis.

D.2 Figure-production constraints

- Every plate must remain intelligible in grayscale; color is semantic reinforcement, never the sole carrier of meaning.
- Mathematical labels must use the same typography as the manuscript.
- Avoid pseudo-3D, glossy gradients, generic stock iconography, and ornamental circuit-board motifs.
- Where motion is represented, use sequential states or directional arrows rather than blur.
- No plate may introduce notation that has not appeared in the adjacent text.
- Every visual metaphor must include a caption marking where the metaphor stops being exact.
- Small diagrams should be reproducible as vector line art for print; large conceptual plates may use richer hand-drawn texture but must preserve the same logical topology.

Glossary of Core Terms

Alpha-equivalence

Equality of binding structure up to consistent renaming of bound variables.

Canonical form

A recognizable introduction form appropriate to a type, such as a lambda for a function or pair for a product.

Context

An ordered record of assumptions, variables, types, and potentially capabilities under which a judgment is made.

Definitional equality

Equality recognized directly by the computational and congruence rules of the type theory.

Elimination rule

A rule specifying how an inhabitant of a type may be used or observed.

Formation rule

A rule specifying when a type expression is legitimate.

Introduction rule

A rule specifying how canonical inhabitants of a type are constructed.

Judgment

A formal assertion licensed by the rules of a theory, such as $\Gamma \vdash a : A$.

Normalization

Reduction of a term toward a form containing no further relevant computational redexes.

Preservation

The property that reduction preserves typing.

Progress

The property that a closed well-typed term is a value or can reduce.

Substitution

Capture-avoiding replacement of a variable by a term, propagating supplied information through dependency.

Type former

A comprehension principle that constructs new types from existing data or types.

Weakening

Structural permission to carry an additional unused assumption.

Further Reading: A Guided Path

This volume is designed to stand alone, but its claims sit inside a deep literature. The references below are not a claim of exhaustive priority; they are a deliberately small path through primary landmarks and modern syntheses.

For the simply typed lambda-calculus, Church’s 1940 paper is a primary historical anchor, while Barendregt supplies the mature lambda-calculus background [2, 1]. For programming-language metatheory, Pierce gives an especially clear operational route through typing, preservation, progress, polymorphism, and implementation [12]; Harper develops a broad language-design framework organized around precise statics and dynamics [7].

For natural deduction and proof-theoretic meaning, begin with Gentzen’s introduction of natural-deduction systems and Prawitz’s normalization-centered development [5, 13]. For the proof/program correspondence, read Curry’s early functionality work together with Howard’s formulae-as-types paper and a modern systematic account such as Sørensen and Urzyczyn [4, 8, 14].

For constructive dependent type theory, Martin-Löf’s 1975 paper and the 1984 lecture notes remain foundational reference points [9, 10]. The Calculus of Constructions provides a historically important bridge between higher-order logic, dependent types, and proof-assistant architecture [3].

For normalization and the proof-theoretic reading of typed lambda calculus, Girard, Lafont, and Taylor remain unusually effective at showing proofs and programs as one normalization phenomenon [6]. Milner’s 1978 polymorphism paper is a primary landmark for algorithmic type inference in programming languages [11].

For dependent type theory and proof assistants, compare several ecosystems rather than identifying the subject with one implementation. Lean, Rocq/Coq, Agda, Idris, and related systems make different engineering choices about elaboration, universes, equality, termination, tactics, and trusted kernels. The contrasts reveal which parts of “type theory” are structural and which are design policy.

For categorical semantics, begin after the operational and proof-theoretic material is comfortable. Cartesian closed categories provide a semantic home for simply typed lambda calculus; locally cartesian closed categories illuminate dependent types. For homotopy type theory, first understand identity types and the distinction between definitional and propositional equality; only then does the geometric language become explanatory rather than slogan-like.

Finally, read alternatives and critics. Untyped lambda calculus, Turing machines, domain theory, process calculi, automata, probabilistic programming, linear logic, game semantics, and coalgebra expose phenomena that any grand-unification thesis must account for. The purpose of this series is not to win a naming contest; it is to discover which structural claims survive comparison.

Bibliography

- [1] H. P. Barendregt. *The Lambda Calculus: Its Syntax and Semantics*. Revised ed., Studies in Logic and the Foundations of Mathematics 103. North-Holland, 1984.
- [2] Alonzo Church. “A Formulation of the Simple Theory of Types.” *Journal of Symbolic Logic* 5(2):56–68, 1940. doi:10.2307/2266170.
- [3] Thierry Coquand and Gérard Huet. “The Calculus of Constructions.” *Information and Computation* 76(2–3):95–120, 1988. doi:10.1016/0890-5401(88)90005-3.
- [4] Haskell B. Curry. “Functionality in Combinatory Logic.” *Proceedings of the National Academy of Sciences* 20(11):584–590, 1934. doi:10.1073/pnas.20.11.584.
- [5] Gerhard Gentzen. “Untersuchungen über das logische Schließen I–II.” *Mathematische Zeitschrift* 39:176–210 and 405–431, 1935. doi:10.1007/BF01201353 and doi:10.1007/BF01201363. English translation: “Investigations into Logical Deduction,” in M. E. Szabo (ed.), *The Collected Papers of Gerhard Gentzen*, North-Holland, 1969, pp. 68–131.
- [6] Jean-Yves Girard, Yves Lafont, and Paul Taylor. *Proofs and Types*. Cambridge Tracts in Theoretical Computer Science 7. Cambridge University Press, 1989.
- [7] Robert Harper. *Practical Foundations for Programming Languages*. 2nd ed. Cambridge University Press, 2016.
- [8] William A. Howard. “The Formulae-as-Types Notion of Construction.” In J. P. Seldin and J. R. Hindley (eds.), *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, Academic Press, 1980, pp. 479–490. Based on notes privately circulated in 1969.
- [9] Per Martin-Löf. “An Intuitionistic Theory of Types: Predicative Part.” In H. E. Rose and J. C. Shepherdson (eds.), *Logic Colloquium ’73*, Studies in Logic and the Foundations of Mathematics 80, 1975, pp. 73–118. doi:10.1016/S0049-237X(08)71945-1.
- [10] Per Martin-Löf. *Intuitionistic Type Theory*. Notes by Giovanni Sambin. Bibliopolis, Naples, 1984.

- [11] Robin Milner. “A Theory of Type Polymorphism in Programming.” *Journal of Computer and System Sciences* 17(3):348–375, 1978. doi:10.1016/0022-0000(78)90014-4.
- [12] Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, 2002.
- [13] Dag Prawitz. *Natural Deduction: A Proof-Theoretical Study*. Acta Universitatis Stockholmiensis, Stockholm Studies in Philosophy 3. Almqvist & Wiksell, 1965.
- [14] Morten Heine Sørensen and Paweł Urzyczyn. *Lectures on the Curry–Howard Isomorphism*. Studies in Logic and the Foundations of Mathematics 149. Elsevier, 2006.

Instructor and Self-Study Roadmap

G.1 Fourteen-week course

A one-semester course can follow the chapters almost one per week. Weeks 1–3 establish judgments and contexts. Weeks 4–6 develop variables, substitution, and rule design. Weeks 7–9 cover functions, algebraic data, and induction. Weeks 10–11 develop equality and normalization. Week 12 treats Curry-Howard. Week 13 builds the checker. Week 14 is reserved for synthesis and the dependent threshold.

The computational laboratories should not be postponed until after the theory. They are designed to occur in parallel. By week 5, students should already have implemented capture-avoiding substitution. By week 11, they should be able to run reduction traces and inspect preservation failures introduced deliberately into a broken checker.

G.2 Mastery criteria

A reader has mastered Volume I when they can do all of the following without relying on memorized slogans:

1. Read and construct typing derivations for the core calculus.
2. Explain the conceptual role of a context and why dependency makes order matter.
3. Implement capture-avoiding substitution and explain why it is the engine beneath beta reduction.
4. Derive the formation/introduction/elimination/computation pattern for a new simple type former.
5. Explain products, sums, functions, and naturals simultaneously as data structures, logical connectives, and communication shapes.

6. Distinguish syntactic, definitional, propositional, extensional, and contextual notions of equality.
7. State and sketch proofs of substitution, progress, and preservation.
8. Build or audit a small bidirectional type checker.
9. Explain precisely why dependent types require substitution at the level of types.
10. State a careful, non-exaggerated version of the thesis that type theory is a grammar of computation.

G.3 Capstone project

The recommended capstone is a tiny proof-programming environment with:

- variables and annotations;
- functions;
- products and sums;
- unit and empty types;
- natural numbers and structural recursion;
- normalization and definitional equality;
- bidirectional checking;
- derivation-aware diagnostics;
- a pretty-printer that makes binding scope unambiguous.

The written report must connect each implementation component to its judgment rule and identify which metatheoretic invariant it is intended to preserve.

Closing Note: The Small Sentence

We end where we began:

$$\Gamma \vdash a : A.$$

At first it looked like a compact type annotation. It now carries an entire architecture.

Γ is an informational world. It records what has already been made available and therefore what may be said next.

a is a construction. It may be data, a function, evidence, a suspended computation, or a component of a larger protocol.

A is a discipline of possible use. It tells us what observations, transformations, and compositions are licensed.

$\vdash$ records that the passage from world to construction is justified by explicit rules.

And computation itself appears when these constructions meet their eliminators, when hypothetical variables receive concrete information, when redexes expose canonical content, and when contexts evolve under communication.

The sentence is small because the theory has compressed a great deal into it.

The next volume asks what happens when the right-hand type itself is allowed to move with the information on the left:

$$\Gamma, x : A \vdash B(x) \text{ type.}$$

That single extension changes everything.

Index

- algebraic data type, 53
- alpha-equivalence, 23
- API, 99

- beta reduction, 32, 44
- bidirectional, 90
- bidirectional typing, 90
- bound variable, 22

- canonical form, 10, 60, 72
- canonicity, 60
- capture-avoiding substitution, 30
- comprehension principle, 101
- computation rule, 10
- confluence, 66
- context, 15
- context formation, 15
- contraction, 18
- Curry–Howard correspondence, 81

- decidability, 94
- definitional equality, 65
- dependent product, 103
- dependent type, 103
- derivation tree, 10

- elaboration, 93
- elaborator, 93
- elimination rule, 10
- exchange, 18

- free variable, 22
- function type, 43

- induction, 58

- inductive type, 57
- inhabitation, 84
- introduction rule, 10

- judgment, 1

- kernel, 93

- meaning explanation, 2

- natural-number recursor, 57
- noncanonical term, 10
- normal form, 72
- notation, 109

- preservation, 33, 76
- product type, 50
- program extraction, 84
- progress, 33, 74
- proof erasure, 84
- proof relevance, 84
- proof search, 84
- proof-theoretic harmony, 37
- protocol state, 99

- recursion, 57

- strong normalization, 73
- strongly normalizing, 73
- structural rule, 18
- substitution lemma, 32
- sum type, 51
- syntax, 1

- trusted computing base, 93

type checking, 90
type family, 103
type synthesis, 90
type system, 2

value, 72
variable rule, 17

weakening, 17